LEARNING NOTES

COSMOS 🔱

Cloud-Native LLM Orchestration

The concept companion to building **Cosmos** — an async, multi-tenant LLM job-orchestration platform that accepts jobs instantly, routes them to healthy providers under rate limits, and processes them resiliently with full state history and observability. These notes scale a five-endpoint CRUD app into that production-grade, cloud-native system — teaching distributed systems, resilience, data-at-scale and architect-level design across 13 buildable stages (Foundations 0.5 → Stage 12).

L1 Async & the Event Loop

how one worker serves thousands

1 The core idea

Your app spends most of its life **waiting** — for the database, the internet, the AI. **Async** is the trick of doing other useful work *during those waits*, using just **one worker**.

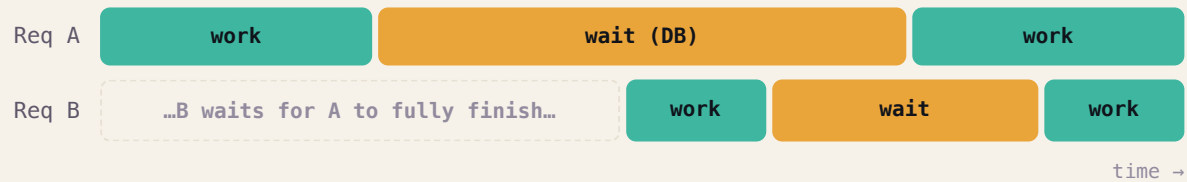
🍽 ANALOGY — ONE WAITER, MANY TABLES

A restaurant with **one waiter**. The slow way: take Table 1's order, then *stand in the kitchen* until it's cooked before touching Table 2. The smart way: hand Table 1's order to the kitchen and **immediately** serve Table 2 while it cooks; deliver when the bell rings. One waiter, many tables — because the *cooking* (waiting) is the slow part, not the waiter.

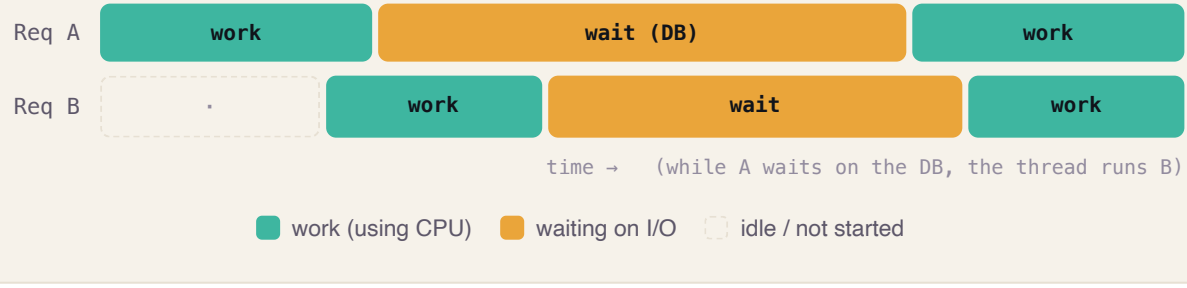
Waiter = one thread · **Tables** = requests · **Kitchen cooking** = I/O wait · **"serve another table while it cooks"** = `await`.

2 Sync vs Async — see the difference

SYNCHRONOUS — THE WORKER SITS IDLE DURING EVERY WAIT (SLOW)



ASYNCHRONOUS — ONE THREAD OVERLAPS THE WAITS (FAST)

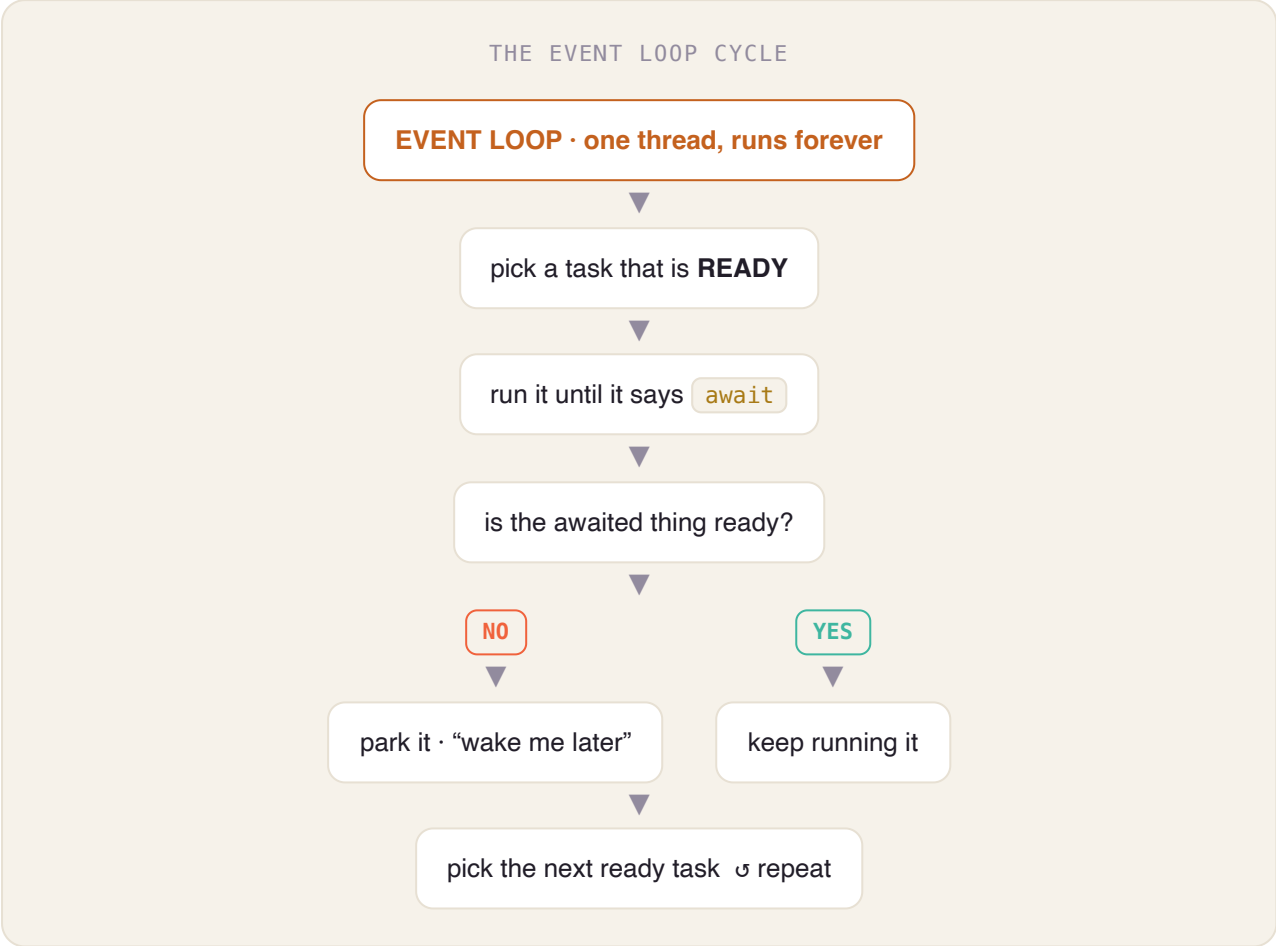


💡 KEY INSIGHT

Async doesn't make one task faster — it stops the worker from **sitting idle**, so many tasks finish in the same wall-clock time.

3 What is the “event loop”?

The **event loop** is the waiter's brain: a `while True:` that runs forever asking one question — *“who's ready for their next step?”* — and running whoever is ready.



4 What happens at `await` ?

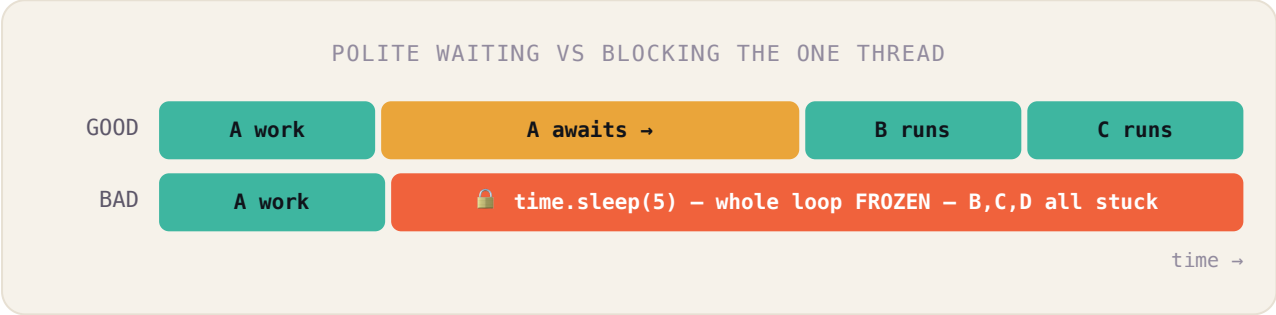
`await` means: **“this might take a while — pause me, go help others, wake me when my result is ready.”**

💡 THE SUBTLE PART (BITES PEOPLE LATER)

Between two `await`s your code runs **uninterrupted** — nobody touches your variables. But **across an `await`, the world can change** (other tasks ran while you were paused). This is the seed of the "async race conditions" we'll handle in later stages.

5 The #1 sin: blocking the loop

Because there's **one** worker, if a task refuses to `await` and does something slow *synchronously*, the whole app **freezes**.



🚫 THE RULE

In async code, never call a **blocking** function. Use the async version: `time.sleep()` ❌ → `await asyncio.sleep()` ✅ · `requests.get()` ❌ → `await httpx.get()` ✅ · sync DB driver ❌ → **async** driver ✅ · heavy CPU work → push to a separate worker pool.

🍽️ ANALOGY

Blocking = the waiter walks into the kitchen and *personally cooks* one dish while every other table sits ignored.

6 Async is NOT parallelism

Everything above happens on **one worker, one CPU core**. Async overlaps *waiting*, not *computing*.

TWO DIFFERENT TOOLS FOR TWO DIFFERENT PROBLEMS

ASYNC = CONCURRENCY

1 waiter, many tables. Overlaps **WAITS**. → Use for **I/O-bound** work (DB, network, AI). *This is your whole platform.*

PARALLEL = MANY COOKS

Many workers/cores. Overlaps **COMPUTING**. → Use for **CPU-bound** work (heavy math). Needs multiple **processes**.

🛠️ YOUR DEVOPS BRIDGE

This is exactly how **nginx / Envoy** serve tens of thousands of connections on a few threads with an epoll/kqueue event loop. And "blocking the loop" is the same "one bad call wedged the whole pod" incident you've debugged — now you know the mechanism.

📝 CHEAT-SHEET — THE WHOLE LESSON IN 8 LINES

- App spends most time **waiting** → async fills those waits with other work.
- 1 thread + event loop = one waiter serving many tables.
- `await` = "pause me, help others, wake me when ready."
- Between awaits: uninterrupted. Across an await: the world may have changed.
- A blocking call on the loop = the whole app freezes (cardinal sin).
- Fix: always use async versions (`asyncio.sleep`, `httpx`, async DB driver).
- async = concurrency (overlap **waits**), NOT parallelism (overlap **CPU**).
- I/O-bound → async. CPU-bound → processes.

✅ SELF-CHECK

1. In the waiter analogy, what is the "kitchen cooking," and what is `await`?
2. Why does async help an app that talks to a DB and an API, but not one doing heavy math?
3. A teammate writes `time.sleep(3)` in an async function. What happens to the other 500 users, and what should they have written?

🧭 MY ANSWERS & FEEDBACK

Understanding 3/3 · Precision ~9/10

✅ **Q1 · What is "kitchen cooking," and what is `await`?**

> **My answer:** Kitchen cooking = I/O wait (DB / network / AI calls). `await` = tell the waiter (thread) not to idle while food cooks — go take other orders.

Correct. Precision: `await` is the point where the task hands control *back to the event loop* — that's what frees it to serve others.

⚠️ Q2 · Why does async help I/O apps but not heavy math?

> **My answer:** One thread never idles, handling several tasks at once; heavy math needs multiple processors (multiple cooks).

Right conclusion — one word to fix: async is **concurrent** (interleaved on ONE thread — one thing at a time, overlapping the *waits*), **not parallel**. Heavy math is CPU-bound: the CPU is busy the whole time so there are *no waits to overlap*, and Python's GIL runs one thread at a time anyway → you need multiple **processes**.

✅ Q3 · A teammate writes `time.sleep(3)` in an async function.

> **My answer:** The one thread freezes for 3s; B, C, D… all wait. Use `asyncio.sleep(3)` so the request pauses without holding the thread.

Correct. Precision: `asyncio.sleep` *yields to the event loop*, which is why the other ~500 users keep being served during those 3s.

L2 Types & Pydantic

labels for data + a guard at the door

1 How a Python variable really works

A variable name is just a **sticky tag you put on a box (an object)**. The *type lives on the box, not the tag* — and you can peel the tag off and stick it on a different box anytime.

This is **dynamic typing**. Breezy for scripts, but risky at scale: a wrong-type value rides silently through five functions and explodes far away. Application Python tames this with **types**.

2 Type hints are *labels* — ignored at runtime

You label functions: `def add(a: int, b: int) -> int`. But the Python interpreter **does not enforce** them — `add("x","y")` still runs.

> 📦 ANALOGY
>
> A type hint is like writing **"FRAGILE — GLASS"** on a box. The label doesn't physically stop someone packing a brick — it's guidance for **humans and tools**. Two "guards" read those labels for you 👇

3 Guard #1 — `mypy` (the inspector, before the flight)

`mypy` reads your labels and checks your code **before it runs**: `add("x", 1)` → ✗ expected int, got str.

> 🛫 ANALOGY
>
> **mypy = the airport inspector who X-rays every labeled box at check-in** — catching "you said glass but packed a brick" *before* the plane departs, not at 30,000 ft (3am in prod). Put it in CI and a whole class of bugs dies at dev time.

🔤 THE TYPING WORDS YOU'LL USE

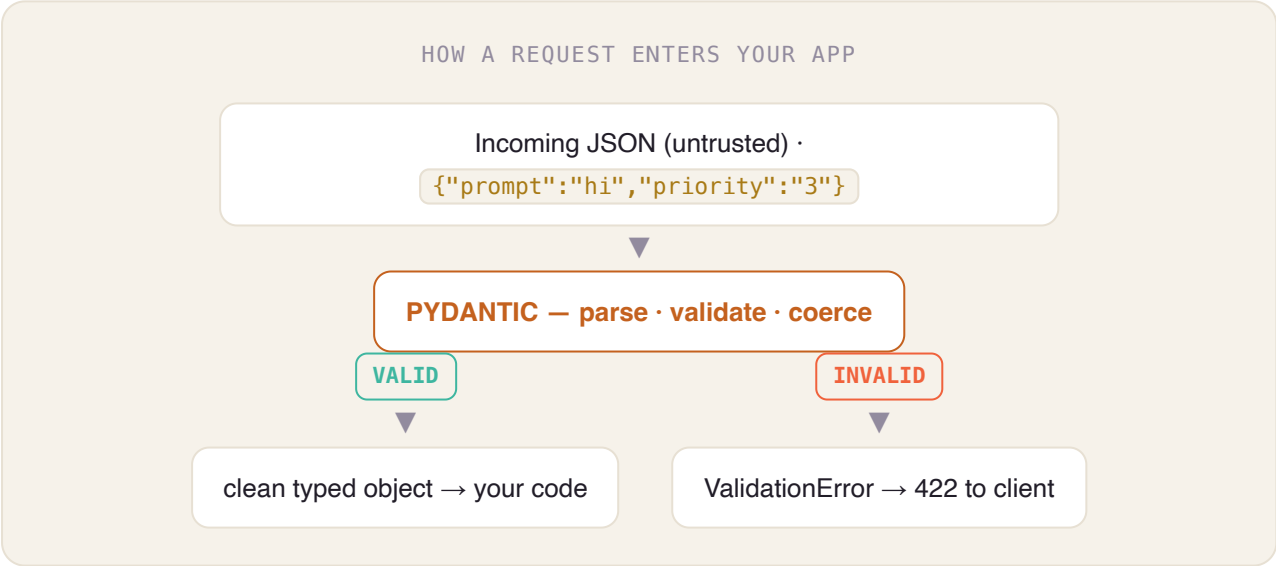
`list[int]`, `dict[str, float]` · `str | None` (optional) · `Literal["pending","running","done"]` (only these exact values) · `Protocol` ("anything with these methods fits" — for ports/adapters).

4 Guard #2 — Pydantic (customs, at the border)

Hints don't check **incoming data**. When a client sends `{"priority":"abc"}`, plain Python won't stop it landing in an int field. At a **trust boundary** (API request, config, third-party data) you need **runtime** checking — that's **Pydantic v2**: it parses, validates, and coerces, and rejects bad input loudly.

🎨 ANALOGY

Pydantic = the customs officer at the border — opens each incoming bag, checks it matches the declared contents, converts where sensible, and loudly rejects what's wrong. (v2's core is Rust → fast; **FastAPI uses it automatically** → clean `422` for free.)



5 The pattern: validate at the edges, trust the inside

TWO ZONES, TWO GUARDS

EDGE — UNTRUSTED

API requests, config files, third-party responses → **Pydantic validates here, once.**

CORE — TRUSTED

Already-validated typed objects flow freely → **mypy checks your code**; no repeated runtime validation.

ANALOGY

Airport security screens you **once** at the entrance; inside the secure zone you're trusted — nobody re-screens at every gate. Pydantic at the doors, plain typed objects inside.

YOUR DEVOPS BRIDGE

This is **JSON Schema / OpenAPI request validation / Terraform variable type constraints** — the same idea, now living inside your Python at its boundary.

CHEAT-SHEET

- A variable is a tag; the type lives on the **object**, not the name (dynamic typing).
- Type hints are labels — the interpreter **ignores** them at runtime.
- **mypy** = static inspector: catches type bugs **before** you run (put it in CI).
- **Pydantic** = runtime customs at the boundary: parse, coerce, reject bad input.
- Use **both**: mypy for your code, Pydantic for untrusted input.
- Pattern: **validate at the edges, trust the typed core.**
- Handy types: `list[int]`, `str | None`, `Literal[...]`, `Protocol`.
- FastAPI uses Pydantic automatically → clean 422 on bad requests.

SELF-CHECK

1. Is a Python type attached to the *name* or the *object*? Why does that make hints “not enforced” at runtime?
2. You receive a webhook from Stripe. Which guard applies — mypy, Pydantic, or both — and *where exactly* does each one act?
3. Why is `status: Literal["pending","running","done"]` safer than `status: str`?

L3 Generators & Coroutines

the machinery under async

1 The idea: a function that can pause

Normal functions run start-to-finish and forget everything. A **generator** can **pause in the middle, hand back a value, and later resume exactly where it left off** — remembering all its variables.

> 🍬 **ANALOGY — A PEZ DISPENSER**
>
> A normal function dumps the whole bag of candy at once. A generator is a **Pez dispenser**: press once → one candy (`yield`), it pauses; press again → the next one. It produces values **one at a time, on demand (lazily)**, so it never has to hold the whole bag in memory.

YIELD = PAUSE & HAND BACK A VALUE

call generator → runs to first `yield 1` → **pauses**, returns 1

▼ ask for next

resumes after the yield → runs to `yield 2` → pauses, returns 2

▼ …until it finishes

2 Coroutines = the same trick, for async

An `async def` function is a **coroutine** — it uses the *exact same "pause & resume" machinery* as a generator. The only difference: a generator pauses at `yield` to hand back **data**; a coroutine pauses at `await` to hand back **control** to the event loop (Lesson 1!).

> 💡 **KEY INSIGHT**
>
> Calling `async def f()` does **nothing** by itself — it just creates a coroutine object sitting there, inert. It only runs when the event loop *awaits/schedules* it. (Like a recipe card: writing it doesn't cook anything; a chef has to pick it up.)

> 🔗 **TIES BACK TO L1**
>
> "`await` pauses my task and lets others run" works *because* coroutines are pausable functions. Now you know the gears inside the event loop.

📝 CHEAT-SHEET

- Generator = a function that pauses at `yield`, produces values lazily (one at a time).
- It remembers its place & variables between resumes.
- Coroutine (`async def`) = same pause/resume machinery, pauses at `await`.
- `yield` hands back **data**; `await` hands back **control**.
- Calling a coroutine returns an inert object — it runs only when awaited/scheduled.
- Generators save memory (lazy); great for streaming large data.

✅ SELF-CHECK

1. What do a generator and a coroutine have in common under the hood?
2. You write `result = my_async_fn()` and nothing happens. Why — and what's missing?
3. When would a generator save you memory vs a normal function returning a list?

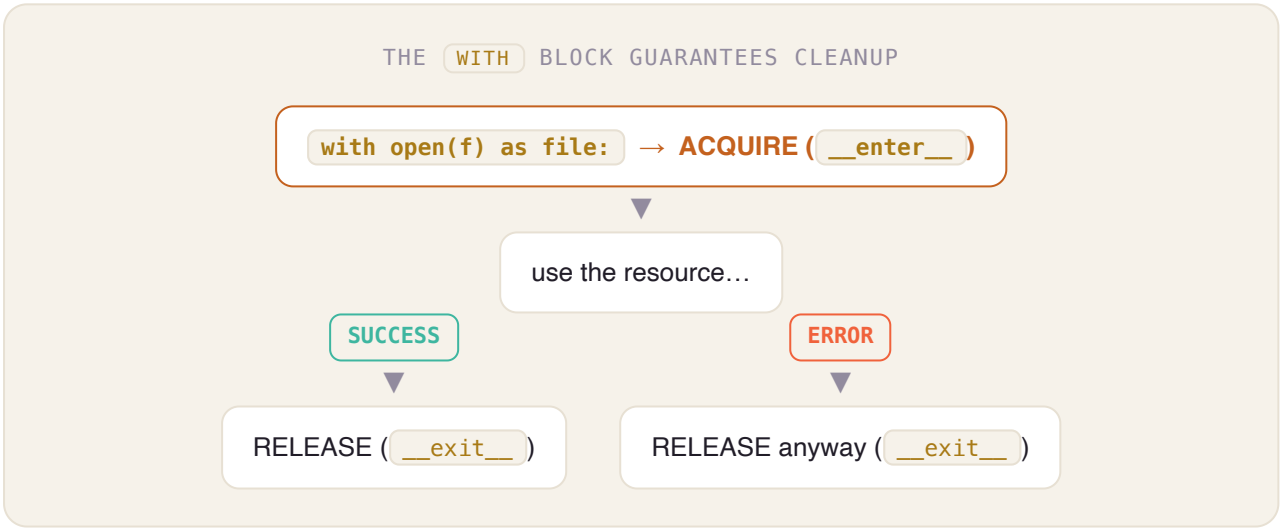
L4 Context Managers

`borrow → use → always return`

1 The problem: things you must clean up

Files, database sessions, network connections, locks — you **open/acquire** them, use them, and **must release** them. If an error happens mid-use and you forget to release, you **leak** resources (connections pile up, the app dies).

> 📚 **ANALOGY — A LIBRARY BOOK**
>
> A context manager (`with`) is like borrowing a library book that **returns itself automatically the moment you leave** — even if you trip and fall on the way out (an exception). You literally cannot forget to return it.



2 `async with` for async resources

Async resources (a DB session, an HTTP client) use `async with` (backed by `__aenter__` / `__aexit__`). You'll use this constantly in Stage 0.5 for the SQLAlchemy session — open a session, do work, and it commits/closes safely even on error.

> 💡 **TWO WAYS TO MAKE ONE**
>
> A class with `__enter__` / `__exit__`, or the easy way: decorate a generator with `@contextlib.contextmanager` — the code before `yield` is "acquire," after is "release." (Notice: generators show up again!)

📝 CHEAT-SHEET

- `with` guarantees cleanup runs — even if an exception is thrown.
- Mechanism: `__enter__` (acquire) → body → `__exit__` (release, always).
- `async with` = the async version (`__aenter__` / `__aexit__`) for DB sessions, connections.
- Build one via a class, or `@contextmanager` on a generator.
- Use for: files, DB sessions, locks, connections — anything you must release.

✅ SELF-CHECK

1. Why is `with open(...)` safer than opening a file and calling `.close()` yourself?
2. What runs in `__exit__`, and when is it guaranteed to run?
3. Why would a DB session use `async with` instead of plain `with`?

L5 Decorators

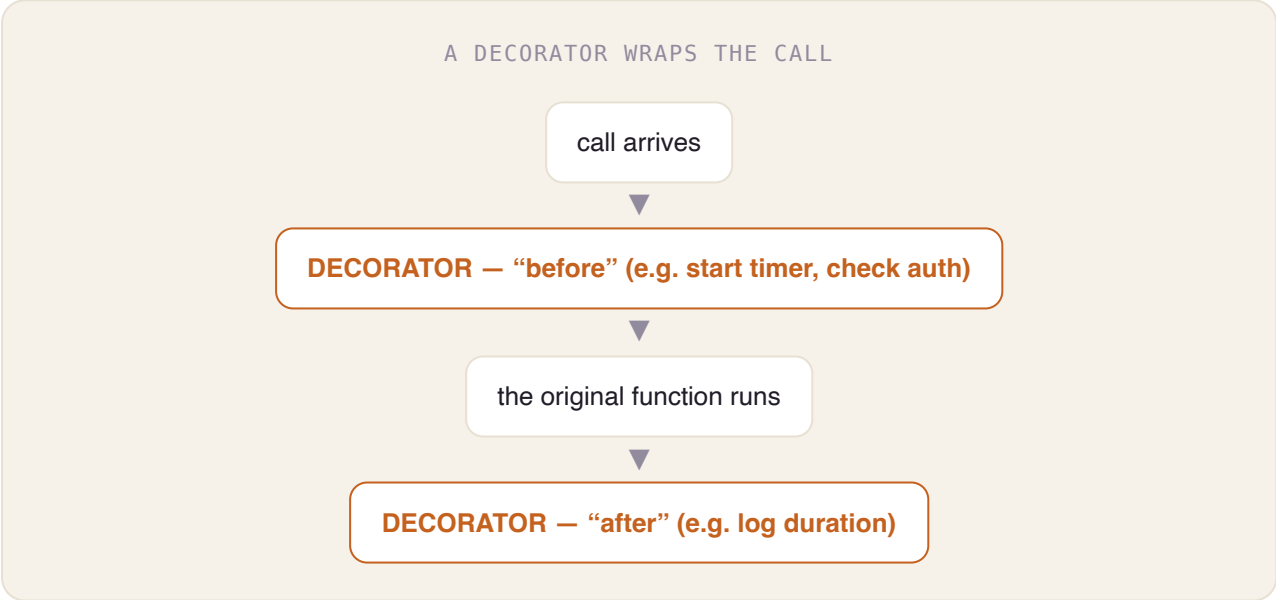
wrap a function to add behavior

1 The idea: gift-wrapping a function

In Python, functions are just objects you can pass around. A **decorator** takes a function, **wraps it in an extra layer** that runs code before/after, and returns the wrapped version — *without changing the original function's code*.

🎁 ANALOGY — GIFT WRAPPING

The gift inside is unchanged; you add a layer around it. Decorators add **cross-cutting behavior** — logging, timing, auth checks, retries, caching — the same way, to many functions, without copy-pasting that logic into each.



2 You'll see these everywhere

`@app.get("/jobs")` (FastAPI routes), `@pytest.fixture` (tests), `@property`, `@lru_cache` — all decorators. Tip: use `functools.wraps` inside your decorator so the wrapped function keeps its real name and docs (otherwise debugging gets confusing).

🛠 YOUR DEVOPS BRIDGE

A decorator is basically **middleware** for a single function — the same “run something before/after” idea as nginx/Envoy filters or HTTP middleware, at function scope.

📝 CHEAT-SHEET

- Functions are objects → you can wrap them.
- A decorator takes a function, returns a wrapped one that adds before/after behavior.
- `@name` above a function applies the decorator.
- Great for cross-cutting concerns: logging, timing, auth, retry, caching.
- Use `functools.wraps` to preserve the original name/docs.
- FastAPI & pytest are built on decorators.

✅ SELF-CHECK

1. In one sentence, what does a decorator do to a function?
2. Name two cross-cutting concerns that are perfect for a decorator, and why.
3. Why bother with `functools.wraps`?

L6 Dataclasses vs Pydantic

two kinds of “data box”

1 Two containers for structured data

Both hold structured data with named, typed fields. The difference is **whether they validate**:

TRUSTED CORE VS UNTRUSTED EDGE (TIES TO L2)

@DATACLASS — PLAIN BOX

Auto-generates `__init__` / `__repr__` / `__eq__`. **No validation.** Use for **trusted, internal** data you already checked. Fast, zero-dependency. `frozen=True` makes it immutable.

PYDANTIC MODEL — BOX + INSPECTOR

Same fields, but **validates & coerces at construction**, and serializes to/from JSON. Use at the **boundary** (API input, config, external data).

📦 **ANALOGY**

A **dataclass** is a labeled box you pack yourself (you trust the contents). A **Pydantic model** is a box with a customs inspector who checks everything going in. Don't pay for the inspector on internal boxes you already trust.

💡 **RULE OF THUMB**

Untrusted input crosses a boundary → **Pydantic**. Internal, already-validated domain object → **dataclass**. (This is the “validate at the edges, trust the core” pattern from L2, made concrete.)

📝 **CHEAT-SHEET**

- Both = typed, named-field containers.
- `@dataclass`: no validation, fast, for trusted internal data; `frozen=True` = immutable.
- Pydantic: validates + coerces + (de)serializes; for untrusted boundary data.
- Pick by *where the data comes from*: edge → Pydantic, core → dataclass.

✅ SELF-CHECK

1. Your worker passes an already-validated job object between two internal functions. Dataclass or Pydantic?
2. Why not just use Pydantic for everything?
3. What does `frozen=True` buy you?

L7 Testing — pytest & Hypothesis

your safety net

1 Why tests (the real reason)

Tests aren't about proving code works today — they're a **safety net** that lets you change code tomorrow *without fear*. For a long-lived platform, that confidence is everything.

> 🧯 ANALOGY
>
> Tests are **smoke alarms** throughout the house. You don't notice them daily, but the moment you break something, one goes off — right next to the problem, not after the house burns down in production.

2 The pytest toolkit

THE FOUR TOOLS YOU'LL USE CONSTANTLY

FIXTURES

Reusable **setup/teardown** run before/after tests (a DB, a client, sample data). The “prep kitchen” that lays out ingredients so each test starts clean.

PARAMETRIZE

Run the **same test with many inputs** automatically — one test, a table of cases (a spice rack of inputs).

MOCKING

Replace slow/external things (DB, Claude API) with a **stunt double** so tests are fast, deterministic, and free.

ASYNC TESTS

`pytest-asyncio` lets you `await` inside tests — needed to test your async code.

3 Hypothesis — the robot that breaks your code

Normal tests check examples *you* think of. **Property-based testing** (Hypothesis) flips it: you state a **property that must always hold** (“encoding then decoding returns the original”), and Hypothesis throws **hundreds of weird inputs** at it to find a counterexample — then shrinks it to the smallest failing case.

💡 WHY IT'S POWERFUL

It finds the edge cases you'd never think to write — empty strings, huge numbers, unicode, zero. Great for parsers, encoders, and pure logic.

📝 CHEAT-SHEET

- Tests = a safety net for fearless change, not busywork.
- **fixtures** = setup/teardown & dependency injection for tests.
- **parametrize** = same test, many inputs.
- **mocking** = fake the slow/external stuff (DB, APIs).
- `pytest-asyncio` = test async code.
- **Hypothesis** = state a property; it hunts counterexamples.
- Test pyramid: many fast unit tests, fewer integration, fewest end-to-end.

✅ SELF-CHECK

1. What's the real value of a test suite for a system you'll change for months?
2. When you test a function that calls the Claude API, what do you do to that call, and why?
3. Give one property (not an example) you could hand to Hypothesis for a “parse job id” function.

L8 Async SQLAlchemy 2.0

talking to Postgres without blocking

1 The three pieces

ENGINE · SESSION · ORM

ENGINE

The **phone line** to the database — a **connection pool** it reuses (opening a new DB connection per request is expensive).

SESSION (UNIT OF WORK)

Your **shopping cart**: add/change/delete objects, then `commit()` applies them all in one **transaction** (or `rollback()` undoes everything).

ORM

A **translator** between Python objects and SQL rows — you work with a `Job` object, it writes the SQL.

🛒 ANALOGY

The session is a shopping cart: you toss items in as you shop, and **nothing is actually bought until checkout** (`commit`). If you change your mind, empty the cart (`rollback`) — the store shelves (DB) are untouched.

2 Why *async* here matters

Use the **async driver** (`asyncpg`) with `AsyncSession` and `async with` . Reason: a DB query is I/O — with a *sync* driver it would **block the event loop** (Lesson 1's cardinal sin!) and freeze every other request. The async driver lets the loop serve others while the DB works.

🔗 TIES TO STAGE 0

You'll hide the session behind a **repository** (e.g., `JobRepository.get(id)`) so your business logic never touches SQL directly — that's the “ports & adapters” idea you'll formalize in Stage 0.

⚠️ WATCH OUT

The **N+1 query problem**: looping over 100 jobs and lazily loading each one's rows = 101 queries. Load them together (eager loading) instead. Classic performance trap.

📝 CHEAT-SHEET

- Engine = connection pool (reused DB connections).
- Session = unit of work / shopping cart; `commit` = checkout, `rollback` = undo.
- ORM = maps Python objects ↔ SQL rows.
- Use the **async driver** (`asyncpg`) + `async with` so DB calls don't block the loop.
- Hide it behind a **repository** (keeps SQL out of business logic).
- Beware the **N+1** query trap — load related rows together.

✅ SELF-CHECK

1. Why must a DB call in an async app use an async driver? (tie it to Lesson 1)
2. What does `commit()` vs `rollback()` do, in the shopping-cart analogy?
3. What is the N+1 problem, and how do you avoid it?

L9 Alembic Migrations

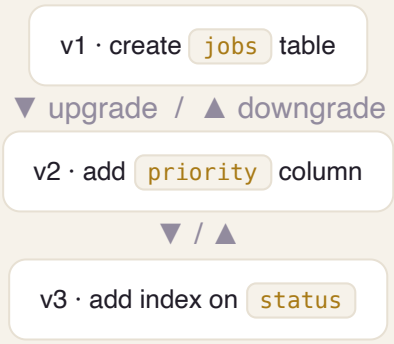
version control for your database

1 The problem: the schema must change over time

You'll add columns, tables, indexes as the app grows. But the database already has **real data** — you can't just drop and recreate it. You need **ordered, repeatable, reversible** changes that every environment (your laptop, staging, prod) applies identically.

> 🗂️ **ANALOGY — GIT FOR YOUR SCHEMA**
>
> Alembic is **version control for the database structure**. Each migration is like a commit: a numbered step with an `upgrade()` (go forward) and `downgrade()` (roll back). The chain of migrations = the schema's history.

A CHAIN OF ORDERED, REVERSIBLE STEPS



2 Autogenerate — with a warning

Alembic can **compare your ORM models to the actual DB** and auto-write a candidate migration. Huge time-saver — but **always read what it generated** before applying (it can miss renames or generate destructive steps). Golden rule: **never edit a migration that has already shipped** — add a new one.

> 📝 **CHEAT-SHEET**
>
> - Migration = a versioned, ordered, reversible schema change (like a git commit).
> - `upgrade()` = apply; `downgrade()` = revert.
> - Autogenerate diffs models vs DB → candidate migration (always review it!).
> - Run migrations in deploy/CI so every environment matches.
> - Never edit an already-shipped migration — write a new one.

✅ SELF-CHECK

1. Why can't you just recreate the database when the schema changes?
2. What are `upgrade()` and `downgrade()` for?
3. Why must you review an auto-generated migration before running it?

L10 Project Hygiene — uv, venv, pyproject

reproducible, isolated projects

1 Virtual environments — isolation

A **virtual environment** is a private, isolated set of installed packages *per project*, so Project A's libraries never clash with Project B's (or your system Python).

🧰 ANALOGY

A venv is a **clean, dedicated workshop** for one project — its own tools on the bench. You don't share one messy garage across every project and hope nothing conflicts.

2 uv, lockfiles & pyproject.toml

uv is a fast, modern tool that creates the venv, installs dependencies, and (crucially) writes a **lockfile** — the exact pinned versions of every package. That makes installs **reproducible**: your machine, a teammate's, and CI all build the *identical* environment. `pyproject.toml` is the project's spec sheet (name, deps, dev-deps, tooling config).

💡 WHY THE LOCKFILE MATTERS

"Works on my machine" bugs usually mean different package versions. A lockfile makes the environment **exact and repeatable everywhere** — the same reproducibility instinct you already have from Docker images and Terraform.

📝 CHEAT-SHEET

- venv = isolated packages per project (no cross-project clashes).
- `uv` = fast tool for venv + install + lockfile (modern pip/venv replacement).
- Lockfile = exact pinned versions → reproducible installs everywhere.
- `pyproject.toml` = project metadata, dependencies, tool config.
- Separate runtime deps from dev deps (pytest, mypy).

✅ SELF-CHECK

1. What problem does a virtual environment solve?
2. What does a lockfile guarantee that a loose dependency list doesn't?
3. Which of your DevOps habits does this map onto?

▽ STAGES 0 → 12 ▽

◆ STAGE 0 — FASTAPI & CLEAN ARCHITECTURE (FULL LESSONS) ◆

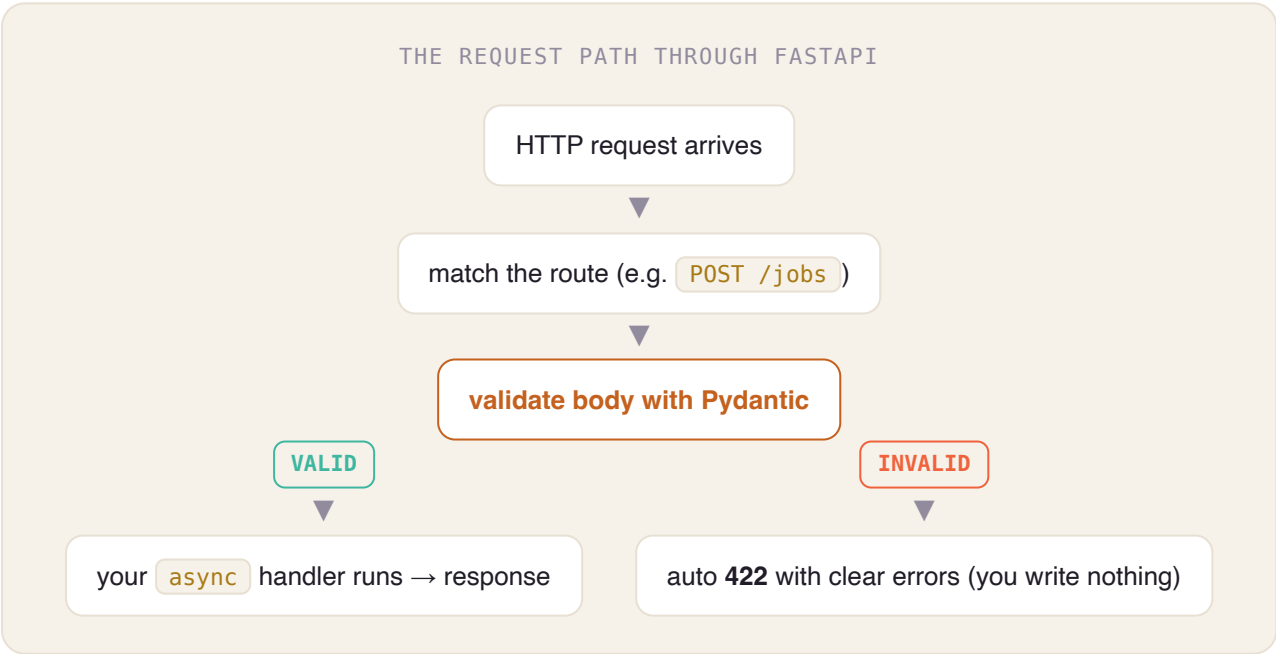
0·1 FastAPI — the async front door

accept · validate · document

1 What it is & why we pick it

FastAPI is a modern Python web framework that receives HTTP requests, **validates the input** (using Pydantic, from L2), runs your handler, and returns a response — and it's **async-native** (from L1).

> 🔔 **ANALOGY — A SUPER-EFFICIENT RECEPTIONIST**
>
> She (1) **checks every form against a template** before accepting it (rejects bad forms at the desk = Pydantic → 422), (2) **juggles many visitors at once** instead of finishing one before greeting the next (async), and (3) **auto-writes the instruction manual** for every service (OpenAPI docs at `/docs`).



2 Two features you'll lean on

Typed params: path/query/body are declared with types and validated for free. **Dependency Injection** (`Depends`): shared things — a DB session, the current user — are declared once and injected into handlers, keeping them clean and testable.

> 🔗 **TIES TO L1 & L2**
>
> Async (L1) is *why* FastAPI serves many slow LLM requests on few workers; Pydantic (L2) is the customs officer at the door. FastAPI simply wires them into the web layer.

📝 CHEAT-SHEET

- FastAPI = async + typed + auto-docs web framework (runs on ASGI / uvicorn).
- Validates request bodies via Pydantic → clean **422** on bad input, before your code.
- Typed path/query/body params; auto OpenAPI at `/docs`.
- `Depends` = dependency injection (DB session, auth) → clean, testable handlers.
- It's the *adapter* at the edge; keep business logic out of the handler.

✅ SELF-CHECK

1. How does FastAPI reject an invalid request body *before* your handler runs?
2. Why is an async framework the right fit for an LLM gateway? (tie to L1)
3. What problem does `Depends` solve?

0·2 Hexagonal architecture (ports & adapters)

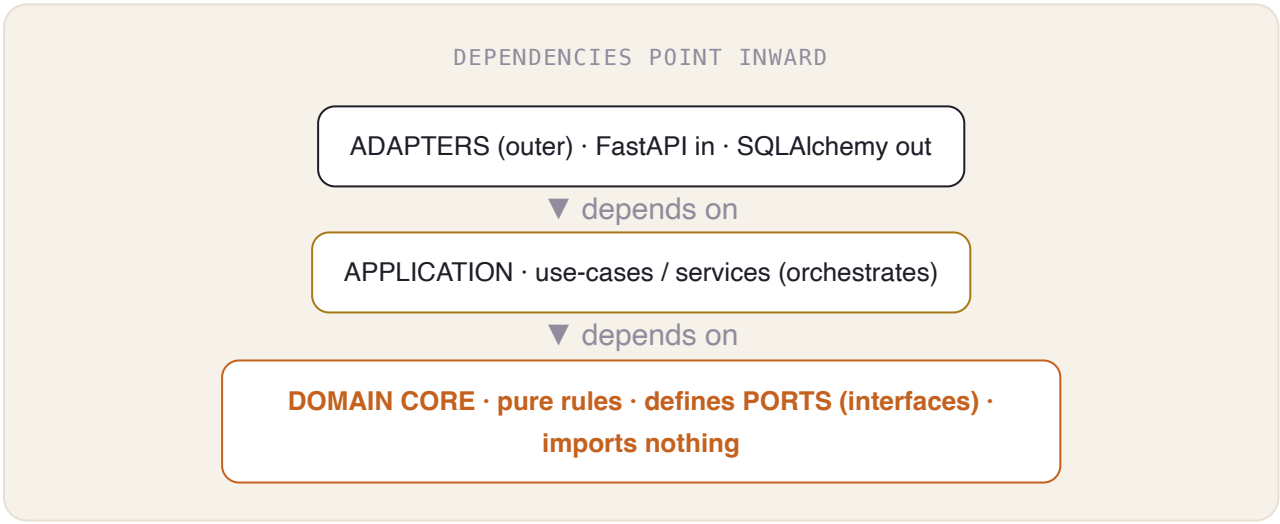
keep the core pure

1 The idea: a pure core, swappable edges

Put your **business logic in the center** (the domain). It never imports FastAPI or SQLAlchemy. It talks to the outside world only through **ports** (interfaces). The outside world (web, DB) plugs in as **adapters** that implement those ports.

🔌 ANALOGY — A POWER SOCKET

Your appliance (the domain) doesn't care if the electricity comes from solar or the grid — it just plugs into the **socket (port)**. You can swap the power source (Postgres → a fake DB for tests) without rewiring the appliance.



💡 THE ONE RULE

Arrows only point **inward**. The core defines an interface like `JobRepository`; the SQLAlchemy adapter *implements* it. The core never knows SQLAlchemy exists.

2 Why bother

Testability (swap the DB for an in-memory fake → fast unit tests). **Framework independence** (change web framework or DB without touching rules). **Clarity** (business logic isn't tangled with I/O).

📝 CHEAT-SHEET

- Domain core = pure business logic; imports no framework.
- **Port** = an interface the core defines (e.g. `JobRepository`).
- **Adapter** = an implementation of a port (FastAPI, SQLAlchemy).
- Dependencies point inward; outer layers depend on inner, never the reverse.
- Payoff: swap DB/framework & test the core in isolation.

✅ SELF-CHECK

1. Which layer is allowed to import SQLAlchemy — and which is forbidden to?
2. What's a “port” vs an “adapter,” in your own words?
3. How does this make unit-testing the domain fast?

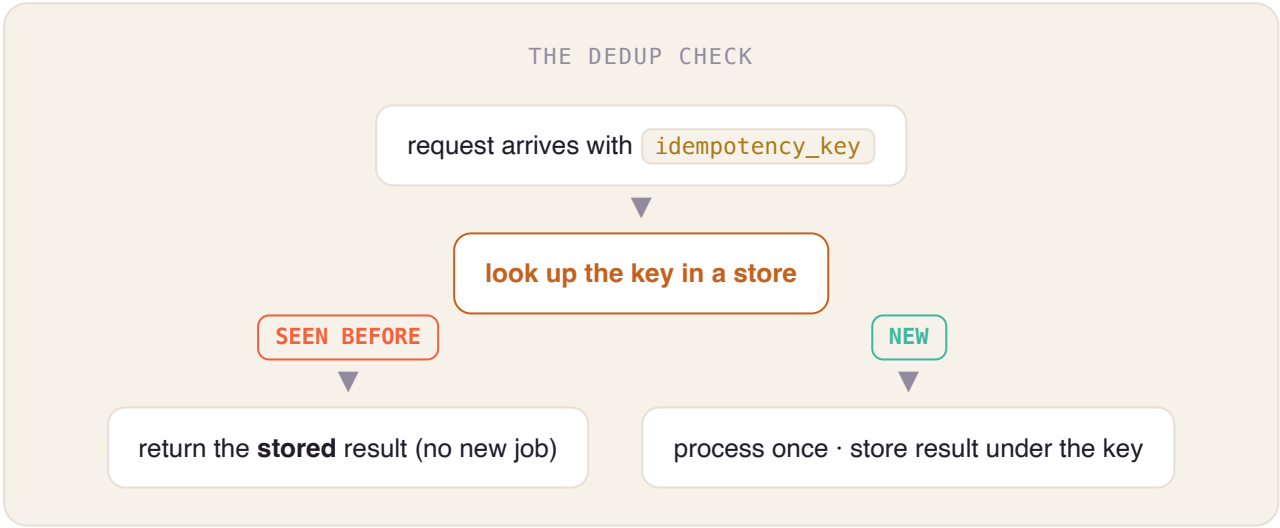
0·3 Idempotency keys

safe retries

1 The problem: everything retries

A client sends `POST /jobs`, the network hiccups, the client retries — now you might create **two** jobs. An **idempotency key** (a unique id the client sends) makes the repeat safe: same key → same single result.

> 🎟️ **ANALOGY — A CONCERT TICKET**
>
> Scanning your ticket twice still gets you **one seat**, not two. Or an elevator button: pressing it 5 times doesn't summon 5 elevators.



> 🔗 **WHY IT'S FOUNDATIONAL**
>
> Once you add a queue (L/Stage 1), retries are everywhere. Idempotency is the seatbelt that makes “at-least-once” delivery safe.

> 📝 **CHEAT-SHEET**
>
> - Client sends a unique key per logical request.
> - Store key → result; a repeat returns the stored result (no duplicate).
> - Turns unsafe retries into safe ones.
> - Essential companion to at-least-once delivery & queues.

✅ SELF-CHECK

1. Why can't you rely on the client to “just not retry”?
2. What do you store, and what happens on a repeat key?
3. Which later feature makes idempotency non-optional?

0·4 Optimistic concurrency control (OCC)

no lost updates, no locks

1 The lost-update problem

Two requests read the same job (version 3), both change it, both save — the second silently overwrites the first. **OCC** prevents this with a **version number**: you save only if the version is still what you read; otherwise you re-read and retry.

📝 ANALOGY — GOOGLE DOCS / WIKIPEDIA

If someone saved an edit between your read and your write, you get “this changed — reload.” You don't silently clobber their work; you re-base on the latest and try again.

COMPARE-AND-SET ON VERSION

WRITER A

reads version=3 → writes “WHERE version=3” → ✅ succeeds → version becomes 4

WRITER B (CONCURRENT)

also read version=3 → writes “WHERE version=3” → ❌ 0 rows (it's 4 now) → **re-read & retry**

💡 OPTIMISTIC VS PESSIMISTIC

Optimistic = assume conflicts are rare, detect & retry (no locks, high concurrency). **Pessimistic** = lock the row first (safe but serializes writers, risks deadlocks). OCC fits web apps where conflicts are rare.

📝 CHEAT-SHEET

- Add a `version` column; bump it on every write.
- Update “`WHERE id=? AND version=?`”; 0 rows updated = conflict → retry.
- Prevents the “lost update” bug without locking.
- Optimistic (detect+retry) vs pessimistic (lock first).

✅ SELF-CHECK

1. What exactly is the “lost update” problem?
2. How does the version check catch a concurrent write?
3. When would you choose pessimistic locking instead?

0·5 12-factor config

one build, many environments

1 Config lives in the environment

Never hard-code URLs, credentials, or flags. Read them from **environment variables**, so the *same* built artifact runs unchanged in dev, staging, and prod — only the env differs.

🔌 ANALOGY — A TRAVEL APPLIANCE

The same laptop charger works worldwide; you just change the **plug adapter** per country. The appliance (your build) is identical; the plug (config) changes per environment.

SAME IMAGE, DIFFERENT ENV

ONE BUILD ARTIFACT

the exact same Docker image, unchanged

+ DEV ENV VARS

dev DB URL, debug on → runs as **dev**

+ PROD ENV VARS

prod DB URL, secrets from Vault → runs as **prod**

🛠️ YOUR DEVOPS BRIDGE

This is the same principle behind a single container image promoted across environments, and 12factor.net is the canonical checklist. You already live this with Helm values & ConfigMaps.

📝 CHEAT-SHEET

- Config in env vars, not in code.
- No secrets in the repo — inject at runtime (later: Vault).
- One build promoted across dev/staging/prod.
- Backing services (DB, cache) are attached resources via config.

✅ SELF-CHECK

1. Why must the same build run in every environment?
2. Where do secrets belong, if not in code?
3. Name one 12-factor habit you already practice in K8s.

0·6 Correlation IDs & consistent errors

trace one request everywhere

1 Follow a single request across the system

Attach a unique **correlation id** (e.g. `X-Request-ID`) to each request at the edge, propagate it to every service and log line, so you can reconstruct one request's whole journey by searching that id.

> 📦 **ANALOGY — A COURIER TRACKING NUMBER**
>
> One number follows your parcel through every hub — pickup, sorting, transit, delivery. Search it and you see the entire journey. Without it, debugging is asking every hub “did you see a brown box?”

2 One error shape

Return errors in a single consistent structure — e.g. `{code, message, request_id}` — so clients handle failures predictably and every error is traceable back to its logs.

> 🔗 **SETS UP STAGE 12**
>
> Correlation ids are the seed of **distributed tracing** (OpenTelemetry). Add them now and observability later is almost free.

📝 CHEAT-SHEET

- Assign a request id at the edge; propagate everywhere.
- Log it on every line for that request.
- Return one uniform error shape (code, message, request_id).
- Foundation for tracing (OTel) in Stage 12.

✅ SELF-CHECK

1. How does a correlation id turn a multi-service debug into a single search?
2. Why standardize the error response shape?
3. Which Stage-12 capability does this quietly enable?

◆ STAGE 1 — ASYNC CORE: QUEUE · STATE MACHINE · EVENT SOURCING ◆

1·1 Accept vs Process (202 + id)

respond fast, work later

1 The problem: slow work must not block the request

An agent job runs for seconds to minutes. If the HTTP handler waited for it, every in-flight job would hold a connection open — and a burst of traffic would exhaust your connection pool and the server would stop accepting *anything*. You have to answer the client in milliseconds regardless of how long the work takes.

2 The handshake: 202 + a job id

So you **split accepting from processing**. The gateway validates the request, writes a job row, immediately returns **202 Accepted + a job id**, and drops the job onto a queue. The heavy lifting happens later in a background worker. The id is the client's claim ticket.

3 How the client gets the result

With just an id, the client either **polls** `GET /jobs/{id}` until it's done, or **streams** live updates over SSE. Either way the request that submitted the work returned instantly — the system stays responsive no matter how slow the job is.

BLOCKING VS 202

BLOCKING (BAD)

Handler waits for the whole job → connections pile up → server chokes under load.

202 + ID (GOOD)

Handler returns instantly; work runs in the background; client checks back by id.

🦫 ANALOGY — THE DRY-CLEANER

They take your clothes and hand you a **numbered ticket instantly** — they don't clean them at the counter while you stand there. You return (or get a text) when it's ready.

💡 THE ID IS A CONTRACT

That job id is how the client tracks, polls, streams, and cancels the work later. It's also what makes the request safely retryable when combined with idempotency.

🔗 TIES TO S0·3

A retried submit must not create two jobs — so the 202 handshake pairs with the **idempotency key** from Stage 0. Same key → same job, even if the client sends it twice.

📝 CHEAT-SHEET

- Split accept (fast) from process (slow).
- Submit → validate → write job → 202 + id → enqueue.
- Client polls GET /jobs/{id} or streams via SSE.
- Keeps the API responsive under slow work.
- Pair with idempotency so retried submits don't duplicate.

✅ SELF-CHECK

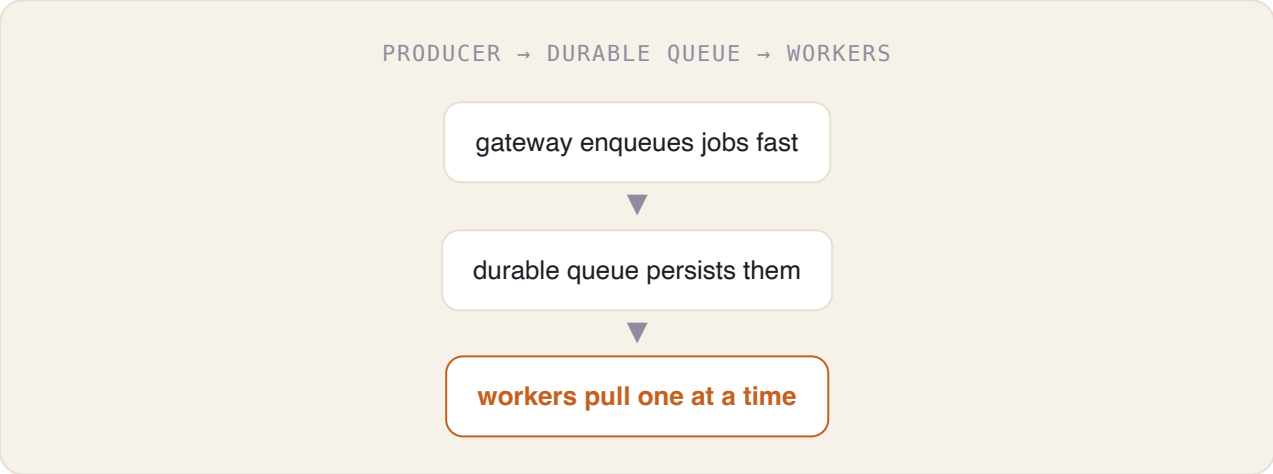
1. Why would processing inline eventually crash the server under load?
2. What does the client receive immediately, and how does it get the result?
3. Which Stage-0 feature makes a retried submit safe?

1·2 Durable Queue

work that survives a crash

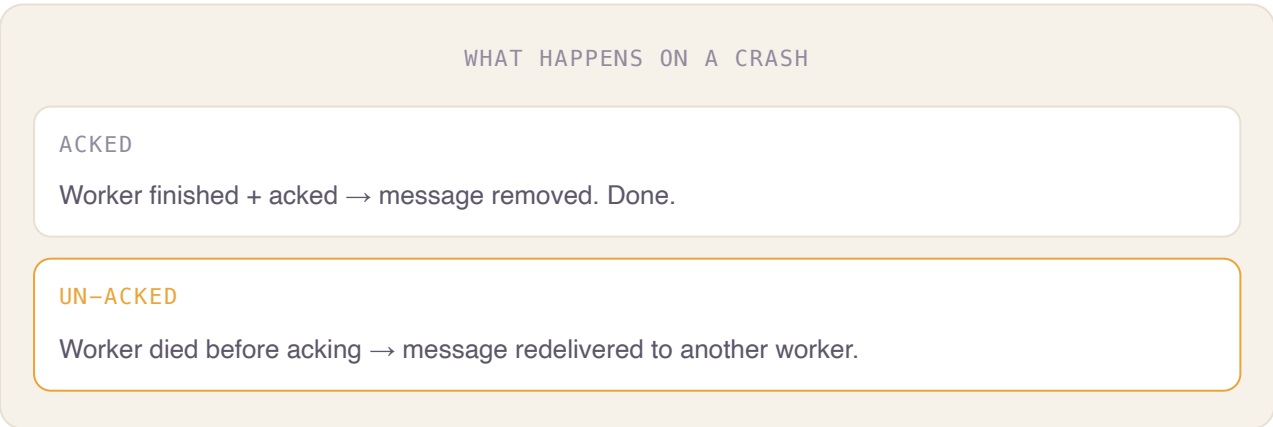
1 A buffer between fast producers and slow consumers

The gateway produces jobs quickly; workers consume them slowly. A **durable queue** sits between them, absorbing the mismatch. Producers never wait on consumers, and consumers pull at their own pace.



2 Durability & acknowledgement

“Durable” means a job isn't lost if a worker crashes. The worker **acknowledges (acks)** a message only *after* it finishes processing. If it dies mid-job before acking, the queue **redelivers** the message to another worker — nothing is dropped (this is why at-least-once + idempotency, 1·5, matter).



3 Load leveling — spikes become queue depth

A traffic spike doesn't overwhelm the workers; it just makes the queue **longer**. Workers drain it steadily. This “queue-based load leveling” is what keeps the system stable under bursts — the queue is a shock absorber.

🏭 ANALOGY — A CONVEYOR BELT

Orders drop onto a belt; cooks take the next when free. If a cook steps away (worker restarts), the orders **stay on the belt**. A rush just makes the belt longer.

💡 ACK TIMING IS A DESIGN CHOICE

Ack *after* success = at-least-once (safe, may duplicate). Ack *before* = at-most-once (may lose work). For jobs you almost always want ack-after + idempotent processing.

🔗 REDIS STREAMS NOW, KAFKA LATER

You start with Redis Streams for simplicity; Stage 5 swaps in Kafka when you need partitioned scale, replay, and multiple independent consumers.

📝 CHEAT-SHEET

- Queue = persistent buffer decoupling producers from consumers.
- Ack after processing → redelivery on crash (nothing lost).
- Load leveling: spikes become queue depth, not overload.
- Ack-after = at-least-once; ack-before = at-most-once.
- Redis Streams to start; Kafka in Stage 5.

✅ SELF-CHECK

1. What two problems does the queue solve between gateway and workers?
2. What happens to an un-acked message if the worker crashes?
3. How does the queue absorb a traffic spike?

1·3 Job State Machine

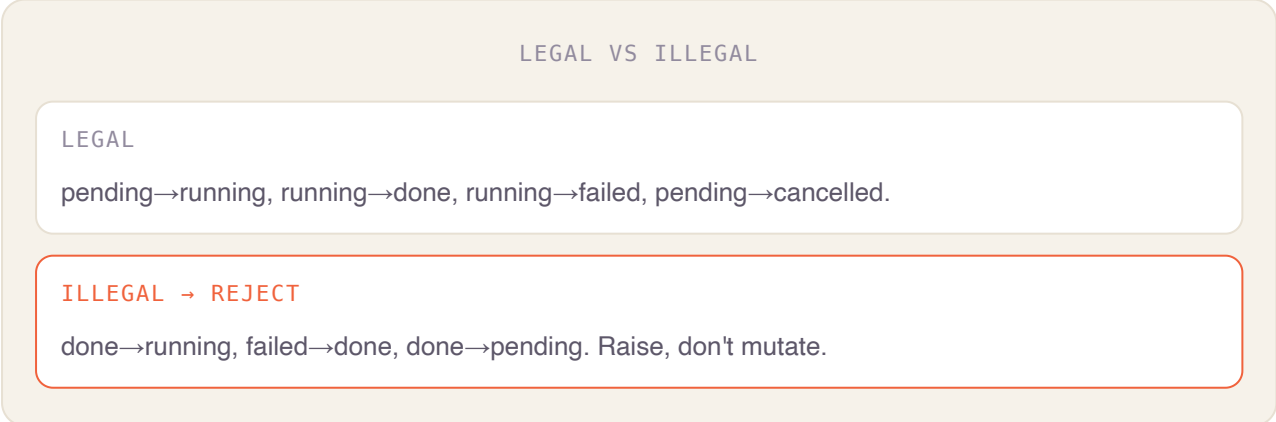
only legal transitions

1 Define the states — and forbid the impossible ones

A job lives in one of a fixed set of states, and can only move along **defined transitions**. Making the states and moves explicit means illegal states simply can't happen — the code won't let a job jump from `pending` straight to `done`.

2 Guards reject illegal moves

Every transition is guarded: attempting an undefined move (e.g. `done → running`) must **raise**, not silently corrupt state. The state machine is a hard guardrail that turns “update the status field” into “make a *valid* transition.”



3 Each transition is an event

Crucially, every legal transition **emits an event** (JobStarted, JobSucceeded…). That's the bridge to event sourcing (1·4): the state machine decides *what's allowed*; the event log records *what happened*. Terminal states (done/failed/cancelled) are final — no transitions out.

📦 ANALOGY — PARCEL TRACKING

“accepted → in transit → delivered.” A parcel can't teleport to “delivered” without “in transit.” The tracking page only ever shows a real, ordered stage.

⚠️ DON'T JUST SET A STATUS COLUMN

A bare status field lets any code write any value, from anywhere — the source of “how did this job end up running *and* done?” bugs. The state machine centralizes and validates every change.

🔗 FEEDS THE EVENT LOG

Each valid transition produces the event that 1·4 stores. State machine = rules; event log = history.

📝 CHEAT-SHEET

- Fixed states; only defined transitions allowed.
- Guard & reject illegal transitions (raise).
- Every transition emits an event.
- Terminal states (done/failed/cancelled) are final.
- Centralizes state changes — no scattered status writes.

✅ SELF-CHECK

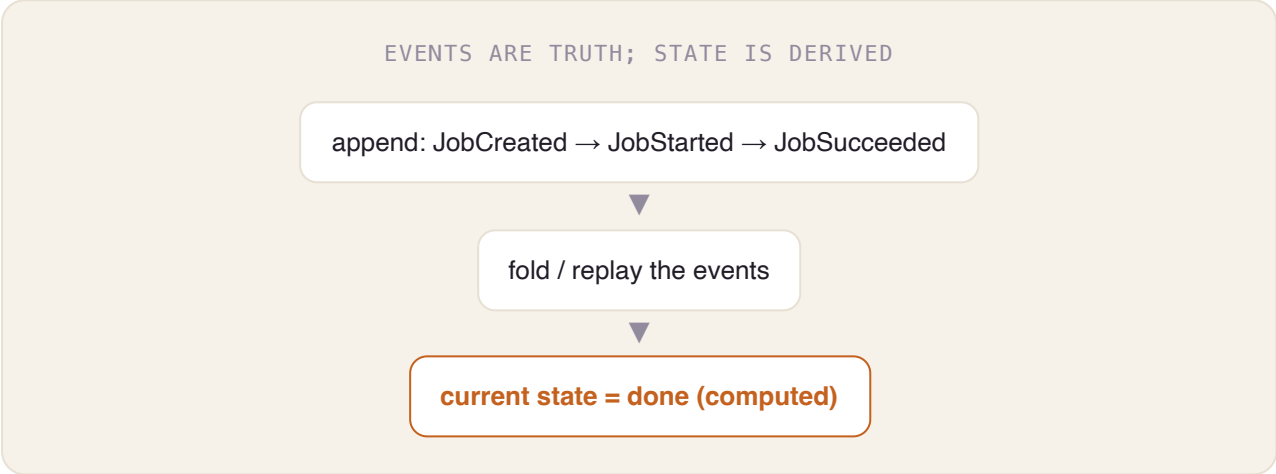
1. Why forbid illegal transitions instead of just updating a status field?
2. List the legal transitions out of running.
3. What should happen if code attempts done → running?

1·4 Event Sourcing

store the story, derive the state

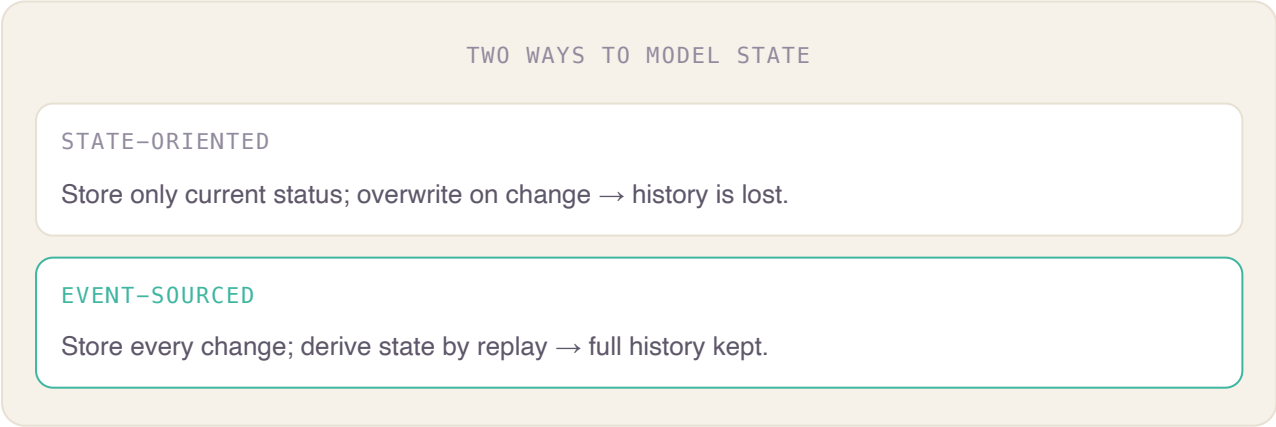
1 Store every change, not just the latest value

Instead of overwriting a job's status, you **append every change as an immutable event** (JobCreated, JobStarted, JobSucceeded…). The event log is the source of truth; the "current status" is something you *compute*, not something you store.



2 Deriving state by folding the log

To know a job's current state, you **replay** its events from the start, applying each to build up the state — a "fold." Because you kept every step, you can reconstruct the state at *any point in time*, not just now.



3 The superpowers — and the cost

You get a perfect **audit trail** ("what happened to job X and when?" is always answerable), **time-travel debugging**, and the ability to build brand-new read-models by replaying (this is the seed of CQRS in Stage 5). The cost: you derive state instead of reading it, so hot paths use periodic **snapshots** to avoid replaying millions of events.

🏦 ANALOGY — A BANK STATEMENT

Your bank stores every **transaction**, not just a balance. You can always see *how* you reached today's balance, audit any moment, and never lose history.

💡 SNAPSHOTS KEEP IT FAST

Replaying the whole log every read is slow. Periodically save a snapshot of derived state, then replay only the events since the snapshot.

🔗 SETS UP CQRS (STAGE 5)

Because state is derived from events, you can project those same events into multiple read-optimized views — exactly what CQRS does.

📝 CHEAT-SHEET

- Append-only event log is the source of truth.
- Current state = fold/replay over events.
- Full audit trail + time-travel + rebuildable read-models.
- Use snapshots so hot reads don't replay everything.
- Trade-off: more moving parts than a status column.

✅ SELF-CHECK

1. In the bank analogy, what are the events and what is the balance?
2. How do you find a job's current status if you only store events?
3. Why do you need snapshots, and what do they store?

1·5 At-least-once & Idempotent Consumers

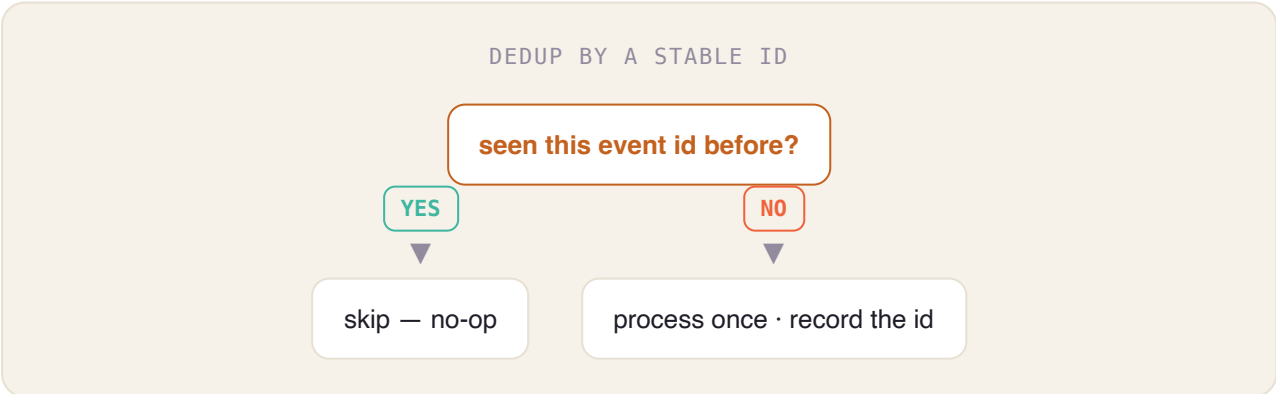
the exactly-once myth

1 Why the same message gets delivered twice

Queues guarantee **at-least-once** delivery. The classic case: a worker processes a job successfully, then **crashes before it can ack**. The queue, never told it succeeded, redelivers the message — so the work runs a second time. Duplicates are not a bug to avoid; they're a reality to design for.

2 Idempotent processing makes duplicates harmless

An **idempotent consumer** keys its work by a stable id (event id / sequence). Before acting, it asks “have I already handled this id?” If yes, it's a no-op; if no, it processes once and records the id. Now delivering twice changes nothing.



3 The “exactly-once” myth

Exactly-once *delivery* is impossible across an unreliable network — you can't both guarantee delivery and guarantee no duplicates. What you achieve is **effectively-once** = at-least-once delivery + idempotent processing. Saying this precisely is a senior signal.

🔔 **ANALOGY — A DUPLICATE REMINDER**

Your phone shows the same reminder twice. If the action is “mark done,” seeing it twice is harmless. If it were “charge the card,” duplication is a disaster — so you design it to be safe.

⚠️ **ACK ONLY AFTER SUCCESS**

If you ack before processing, a crash loses the work (at-most-once). Ack after, and make processing idempotent — that's the safe combination for jobs.

🔗 **SAME THREAD AS S0·3**

Idempotency keys (submit) and idempotent consumers (process) are the same principle at two layers: retries are everywhere, so make repeats harmless.

📝 **CHEAT-SHEET**

- Queues deliver at-least-once → expect duplicates.
- Idempotent consumer: key by id; reprocessing is a no-op.
- Ack only after successful processing.
- “Exactly-once delivery” is a myth → aim for effectively-once.
- Effectively-once = at-least-once + idempotency.

✅ **SELF-CHECK**

1. Give a realistic reason the same message is delivered twice.
2. How does an idempotent consumer make that harmless?
3. What's the honest truth about exactly-once, and how do you approximate it?

1·6 Cancellation

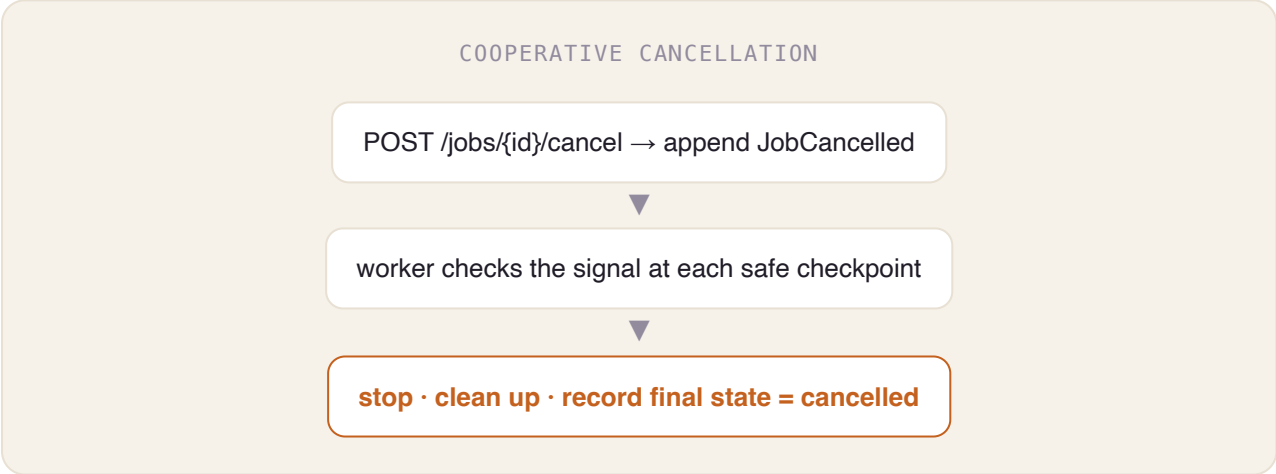
stopping work in flight

1 Why long jobs must be stoppable

Agent jobs can run for minutes and cost real money. Users change their minds; runaway jobs must be stoppable. But you can't just kill a worker mid-operation — that risks half-written state and leaked resources.

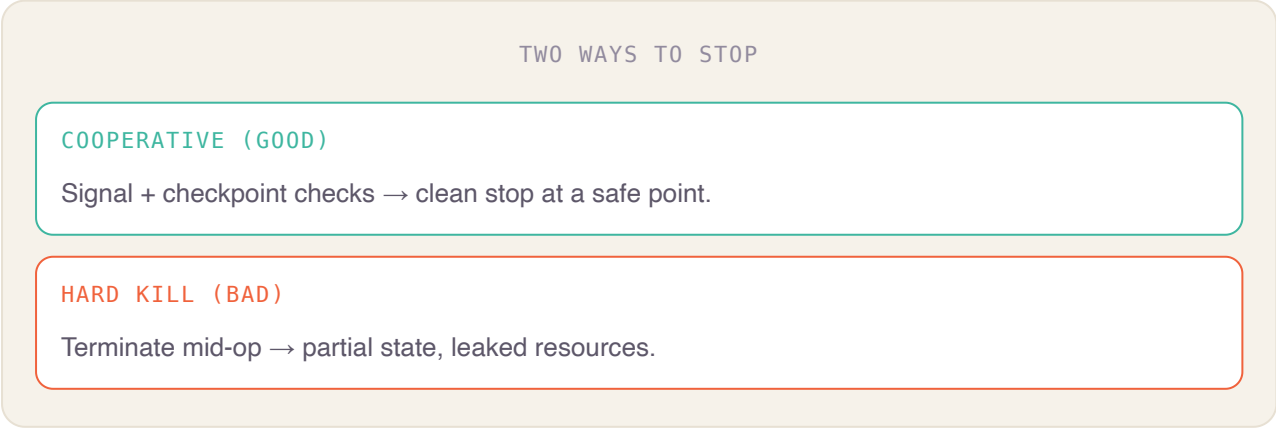
2 Cooperative cancellation

So cancellation is **cooperative**: a cancel request appends a `JobCancelled` signal, and the worker **checks for it at safe checkpoints** in the loop (e.g. between agent steps). It won't abandon a half-finished operation, but before starting the next step it sees the signal and stops.



3 Clean up, then record the ending

On seeing the signal the worker releases resources (open connections, partial writes), then records the terminal `cancelled` state via the event log — so the job's history clearly ends in cancellation, not a mysterious stall.



🍳 ANALOGY — A “CANCEL ORDER” BUTTON

The kitchen checks the cancel board **between steps**. It won't abandon a half-cracked egg mid-motion, but before the next step it sees “cancelled,” stops, and clears the station.

💡 SAME IDEA AS GO'S CONTEXT

Cooperative cancellation via a checked signal is exactly what Go's `context` does (Stage 7) — pass a cancel signal down and check it at boundaries.

🔗 USES THE STATE MACHINE + EVENT LOG

Cancellation is just another guarded transition ((pendinglrunning) → cancelled) that emits an event — reusing 1·3 and 1·4.

📝 CHEAT-SHEET

- Cancellation is cooperative: signal + checkpoint checks.
- Never hard-kill mid-operation.
- Clean up partial work before stopping.
- Record the terminal cancelled state in the log.
- Same pattern as Go's context cancellation.

✅ SELF-CHECK

1. Why is cancellation cooperative rather than an instant kill?
2. Where does the worker check for the cancel signal, and why there?
3. What must you do before recording the job as cancelled?

2·1 Why Polyglot Persistence

the right store per data shape

1 No single database is good at everything

Force everything into one database and you get either rigid schemas where you need flexibility, or weak guarantees where you need ACID. **Polyglot persistence** means deliberately using several stores, each matched to a kind of data.

2 Cosmos's data placement

The mapping is driven by what each piece of data *is*: money and state need ACID; agent trajectories are nested and variable; counters need to be instant.

STORE PER DATA SHAPE

POSTGRESQL

Jobs, tenants, quotas, ledger — the ACID source of truth.

MONGODB

Agent trajectories & variable-shape payloads (nested, evolving).

REDIS

Hot state, cache, rate-limit counters — microsecond reads.

3 The cost, and the interview framing

The price of polyglot is a bigger operational surface (more systems to run and back up). The payoff is that each store does one thing excellently. Frame it as: “I put *money* in Postgres for ACID, *trajectories* in Mongo for flexibility, *counters* in Redis for speed.”

> **ANALOGY — FRIDGE, TOOLBOX, ALBUM**
>
> You don't keep milk, screwdrivers, and photos in one box. A fridge keeps things fresh (Postgres/ACID), a toolbox grabs the right tool fast (Redis), an album holds varied pages (Mongo).

💡 MATCH ON CONSISTENCY VS FLEXIBILITY VS SPEED

Ask of each dataset: does it need strong consistency (Postgres), flexible shape (Mongo), or raw speed (Redis)? That question picks the store.

🔗 THE VECTOR/OBJECT/OLAP STORES TOO

Later you add pgvector/Qdrant (RAG, Stage 3), S3/MinIO (artifacts), and ClickHouse (analytics) — same principle, extended.

📝 CHEAT-SHEET

- Polyglot = several DBs, each for its strength.
- Postgres = ACID truth; Mongo = flexible docs; Redis = speed.
- Choose by consistency vs flexibility vs speed.
- Cost = more to operate; benefit = best guarantees + performance.
- Write the “why polyglot” ADR.

✅ SELF-CHECK

1. Which store holds the billing ledger, and why?
2. Why not put everything in Postgres?
3. What single question helps you pick a store for a dataset?

2·2 Document Modeling — embed vs reference

MongoDB's core decision

1 Keep related data together, or link it?

For each relationship in MongoDB you choose: put the related data **inside** one document (*embed*), or in a **separate** document you link to (*reference*). This single choice drives read speed, duplication, and update cost.

THE TWO SHAPES FOR "A JOB AND ITS STEPS"

EMBED

Steps inside the job doc → one read gets everything. Best when read together & bounded.

REFERENCE

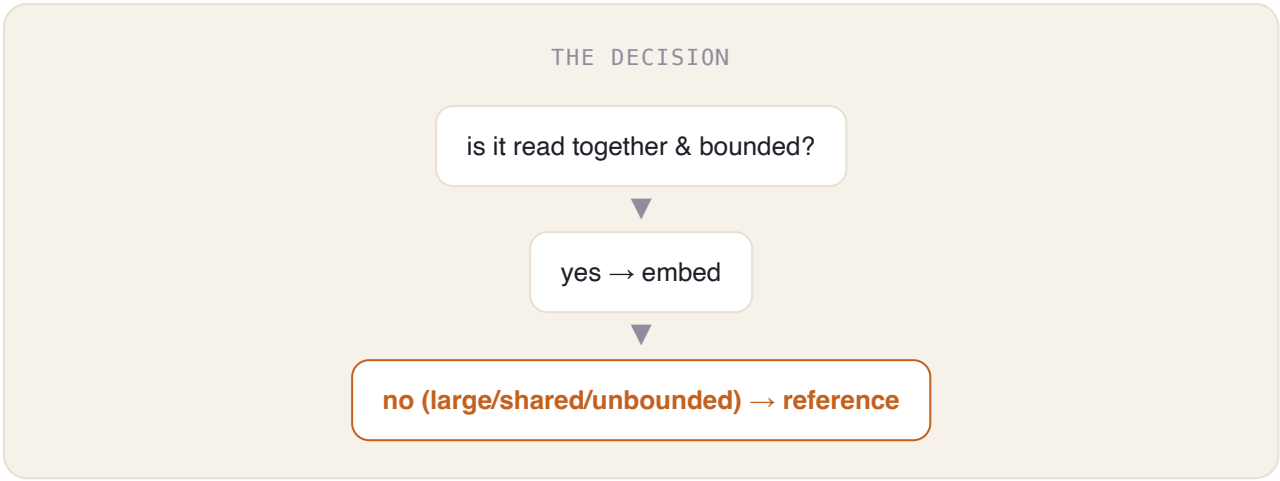
Steps in their own collection, linked by job_id → best when large / shared / unbounded.

2 When to embed

Embed when the data is **read together**, **owned** by the parent, and **bounded** in size. One read returns the whole thing — fast and atomic. Downside: the document grows, and duplicated embedded data is painful to update in many places.

3 When to reference (and the limit that forces it)

Reference when the data is **large, shared, or unbounded** — e.g. an agent trajectory that can grow to thousands of steps. MongoDB caps a document at **16MB**, so unbounded lists must be referenced. The guiding question is always: *"what do I read together?"*



🍱 ANALOGY — COMBO MEAL VS MENU NUMBER

Embed = a combo meal on one tray (everything you eat together, one grab). Reference = a menu number you look up separately (good when the item is huge or shared).

⚠️ BEWARE THE GIANT DOCUMENT

Embedding an ever-growing list eventually hits the 16MB limit and makes every update rewrite a huge document. Unbounded → reference.

💡 MODEL FOR QUERIES, NOT FOR TIDINESS

Unlike relational normalization, Mongo modeling optimizes for your access patterns. Shape documents around how you'll read them.

📝 CHEAT-SHEET

- Embed: read-together + bounded → one fast read.
- Reference: large / shared / unbounded data.
- 16MB document limit forces unbounded lists to reference.
- Model for query patterns, not normalization dogma.
- Guiding question: what do I read together?

✅ SELF-CHECK

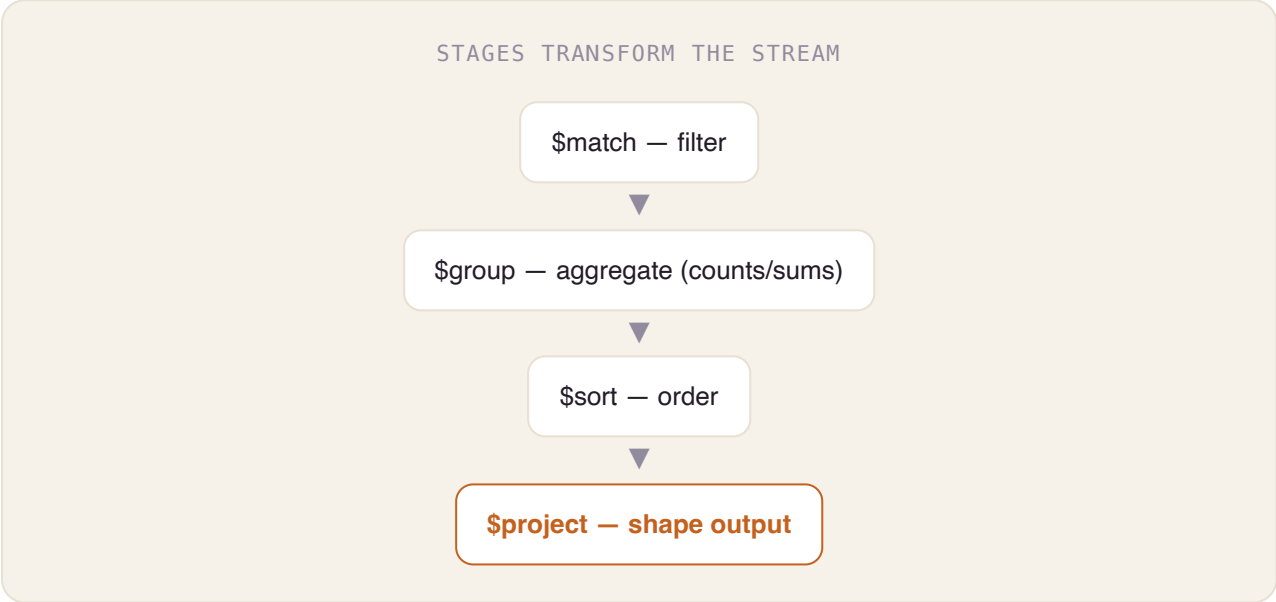
1. When is embedding the better choice?
2. Why does an ever-growing list push you toward a reference?
3. What hard limit can force a reference?

2·3 Aggregation Pipeline

compute inside the database

1 An assembly line for your data

The **aggregation pipeline** processes documents through ordered **stages**, each transforming the stream and passing it on — all inside MongoDB, so you don't drag raw data into your app to summarize it.



2 Common stages

`$match` filters, `$group` aggregates by a key (counts, sums, averages), `$sort` orders, and `$project` reshapes the final output. You chain them to turn raw documents into exactly the summary you need.

3 Performance: filter early, use indexes

Put `$match` **first** so later stages process far fewer documents, and make sure that match can use an **index**. This is the same “push predicates down” instinct as SQL query planning — do the cheap filtering before the expensive grouping.

ORDER MATTERS

MATCH EARLY (GOOD)

Filter to 1% of docs, then group → fast.

MATCH LATE (BAD)

Group millions, then filter → wasteful.

🏭 ANALOGY — AN ASSEMBLY LINE

Each station does one job (filter, group, reshape) and hands the item on. By the end, raw documents have become the exact summary you wanted.

💡 KEEP COMPUTATION NEAR THE DATA

Aggregating in the DB moves far less data over the network than pulling everything into the app and looping — usually much faster.

🔗 FEEDS DASHBOARDS

These pipelines produce the counts and rollups your ops console (Stage 9) and analytics need.

📝 CHEAT-SHEET

- Ordered stages transform documents in the DB.
- Common: $match → $group → $sort → $project.
- Filter ($match) first, backed by an index.
- Keeps heavy computation close to the data.
- Push predicates down, like SQL planning.

✅ SELF-CHECK

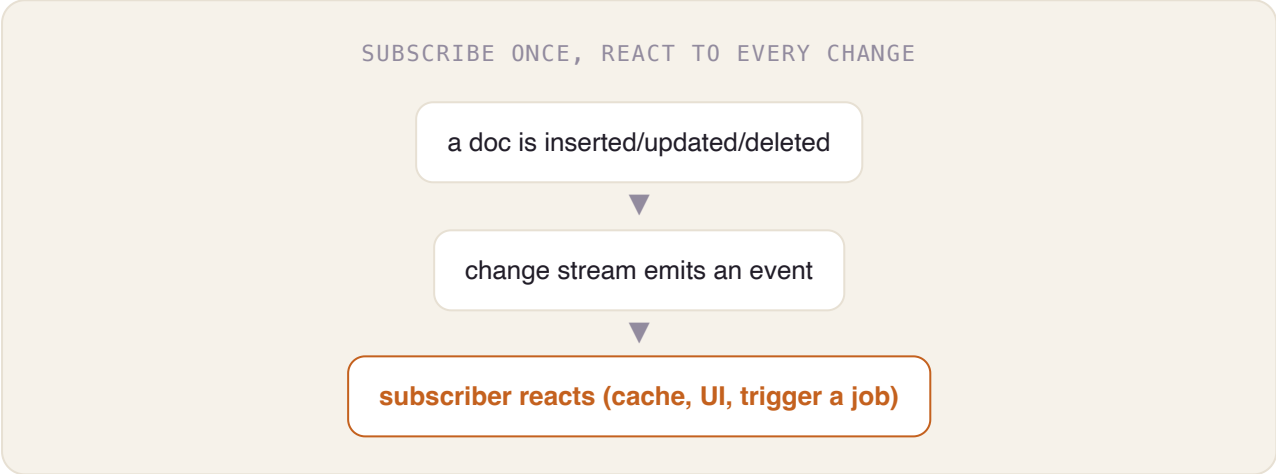
1. Why aggregate in the DB instead of the app?
2. Why put $match first?
3. What does $project do?

2·4 Change Streams

react to data changes live

1 React, don't poll

A **change stream** is a feed you subscribe to that emits an event whenever documents are inserted, updated, or deleted. Instead of repeatedly asking “anything new?” (polling), you're **told** the moment something changes.

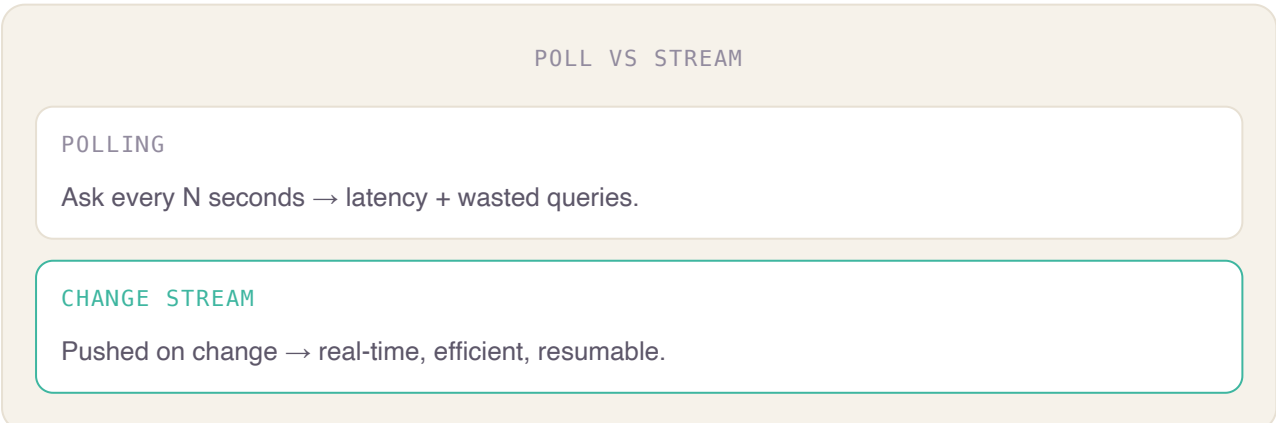


2 What subscribers do with it

Common reactions: invalidate/update a Redis cache, push a live update to the UI, or kick off downstream work. It turns the database into an event source other components can follow.

3 Resumability

Change streams carry a **resume token** — if your subscriber disconnects, it can resume from where it left off instead of missing events. That makes them reliable enough to build features on, not just best-effort notifications.



🔔 **ANALOGY — A DOORBELL**

Instead of walking to the door every 10 seconds to check (polling), you install a doorbell that rings only when someone actually arrives (a change event).

💡 **RESUME TOKENS MAKE IT DURABLE**

Persist the last resume token so a restart continues from the right place — no missed or duplicated reactions.

🔗 **COUSIN OF CDC (STAGE 5)**

This is the database-level version of Change-Data-Capture — react to changes without polling; Stage 5 does it at the Kafka/warehouse scale.

📝 **CHEAT-SHEET**

- Subscribe to insert/update/delete events on a collection.
- React in real time — no polling.
- Uses: cache updates, live UI, triggering work.
- Resume tokens make it recover after disconnects.
- Conceptual cousin of CDC.

✅ **SELF-CHECK**

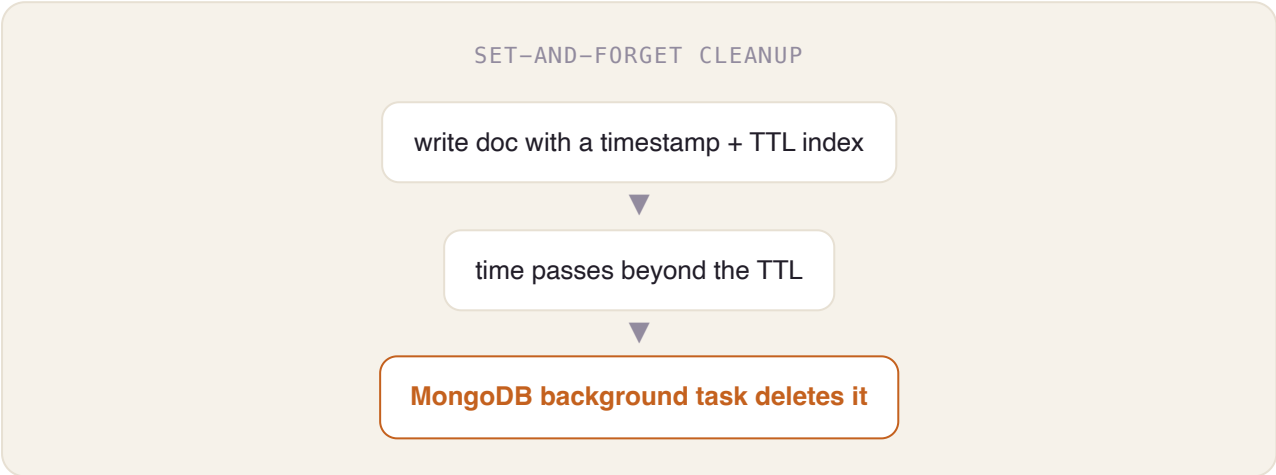
1. What problem do change streams solve vs polling?
2. Name two things a subscriber might do on a change.
3. What does a resume token give you?

2·5 TTL Indexes

data that expires itself

1 Auto-delete transient data

A **TTL (time-to-live) index** tells MongoDB to automatically delete documents a set time after a timestamp field — no cron job, no cleanup code. The data cleans itself up.

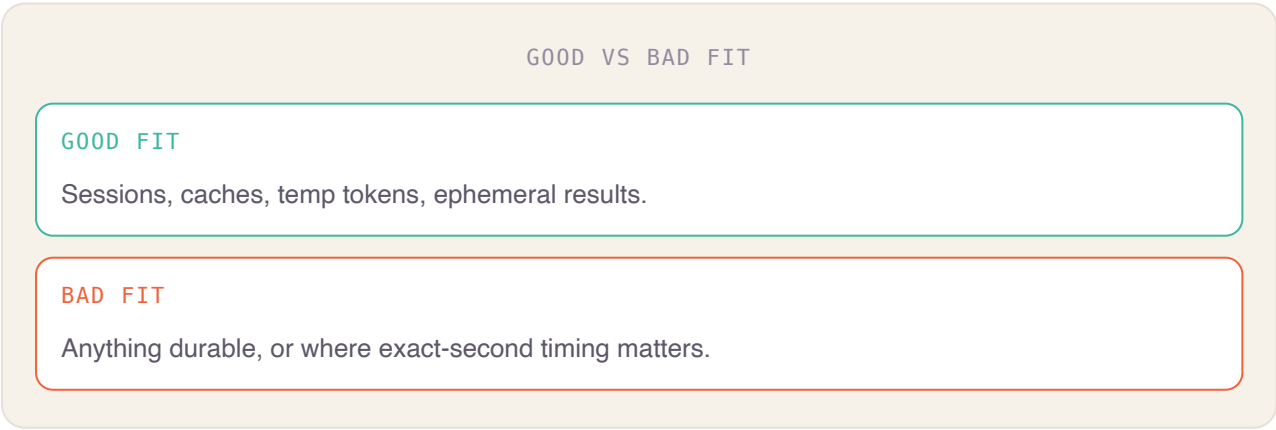


2 How it works (and its limits)

A background task sweeps expired documents periodically — so deletion is **approximate**, not to-the-second. It's ideal for transient data but must not be relied on for exact timing or for data you actually need to keep.

3 When to use it — and when not

Great for sessions, caches, one-time tokens, and temp results. **Don't** use it for anything you must retain, and remember the timing is “soon after expiry,” not precise.



🥛 **ANALOGY — MILK WITH AN EXPIRY DATE**

The carton effectively throws itself out once it's past date. TTL data cleans itself up so you never accumulate stale junk.

💡 **APPROXIMATE, BACKGROUND TIMING**

Documents disappear *soon after* expiry, on the sweep — design around “roughly then,” not “exactly then.”

⚠️ **NOT A DURABILITY MECHANISM**

TTL silently deletes. Never point it at data you might need later.

📝 **CHEAT-SHEET**

- TTL index auto-expires documents after a set time.
- Set on a date field; a background task sweeps them.
- Timing is approximate (near expiry, not exact).
- Great for sessions/caches/temp data.
- Never use it for data you must keep.

✅ **SELF-CHECK**

1. What does a TTL index save you from writing?
2. Is expiry exact to the second — why or why not?
3. Name two good use cases and one bad one.

2·6 Redis Caching Patterns

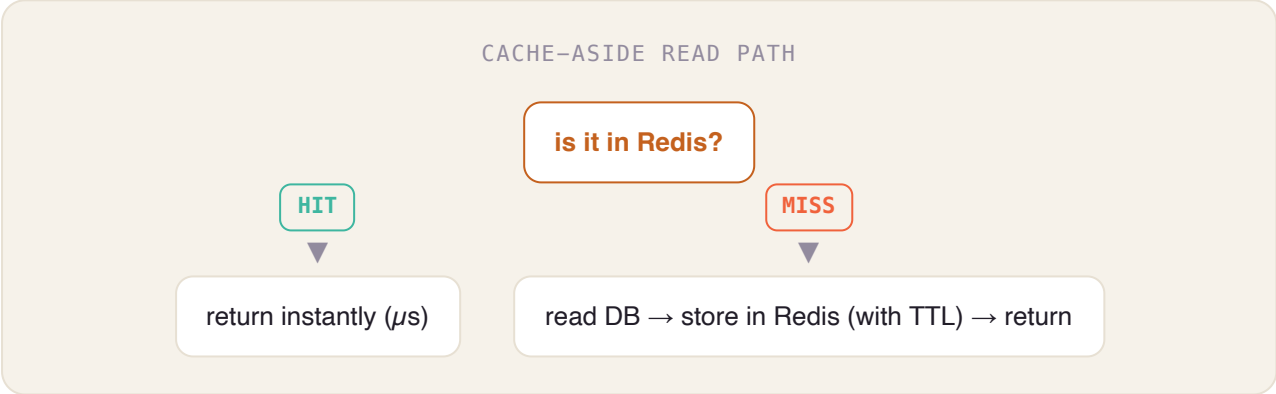
microsecond reads, safely

1 Why cache at all

Some reads are expensive (a heavy query, a slow join) but requested often. **Redis** keeps hot/derived data in memory for **microsecond** reads, shielding the primary database and cutting latency.

2 Cache-aside — the default pattern

The standard approach is **cache-aside**: check Redis first; on a miss, read the database, then store the result in Redis (with a TTL) for next time. Your code manages the cache alongside the DB.



3 The hard part: invalidation

Stale caches cause subtle bugs. Always set a **TTL** so entries can't live forever, and **invalidate or update** the cache when the underlying data changes. Watch for the *cache stampede*: when a hot key expires, many requests miss at once and hammer the DB — mitigate with short locks or staggered TTLs.

🗒️ ANALOGY — A DESK NOTEPAD

For answers you look up often, you jot them on a notepad instead of walking to the archive room (the DB) each time. If it's not on the pad, you fetch it once and write it down.

⚠️ CACHE INVALIDATION & STAMPEDES

“Only two hard things: cache invalidation and naming.” Set TTLs, invalidate on writes, and guard against stampede when a popular key expires.

🔗 REDIS DOES MORE THAN CACHE

The same Redis also holds rate-limit counters (Stage 4) and hot job state — one fast in-memory store, several jobs.

📝 CHEAT-SHEET

- Redis = in-memory, microsecond reads for hot data.
- Cache-aside: check cache → miss → DB → populate (with TTL).
- Always set a TTL; invalidate on writes.
- Beware cache stampede on hot-key expiry.
- Also used for rate-limit counters & hot state.

✅ SELF-CHECK

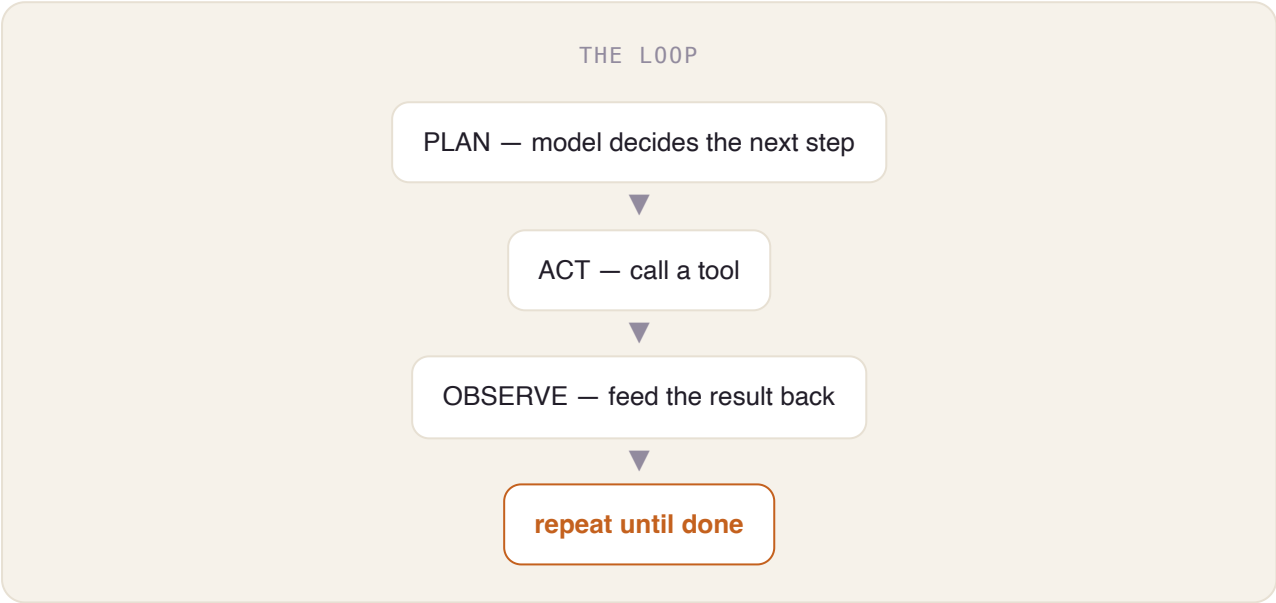
1. Walk through cache-aside on a hit and on a miss.
2. Why is a TTL non-negotiable?
3. What is a cache stampede, and how do you soften it?

3·1 The Agent Loop (plan → act → observe)

the heart of the product

1 From answering to doing

A single model call *answers* a question. An **agent** *does a task*: it plans a step, acts (calls a tool), observes the result, and loops — accumulating progress until the goal is met.

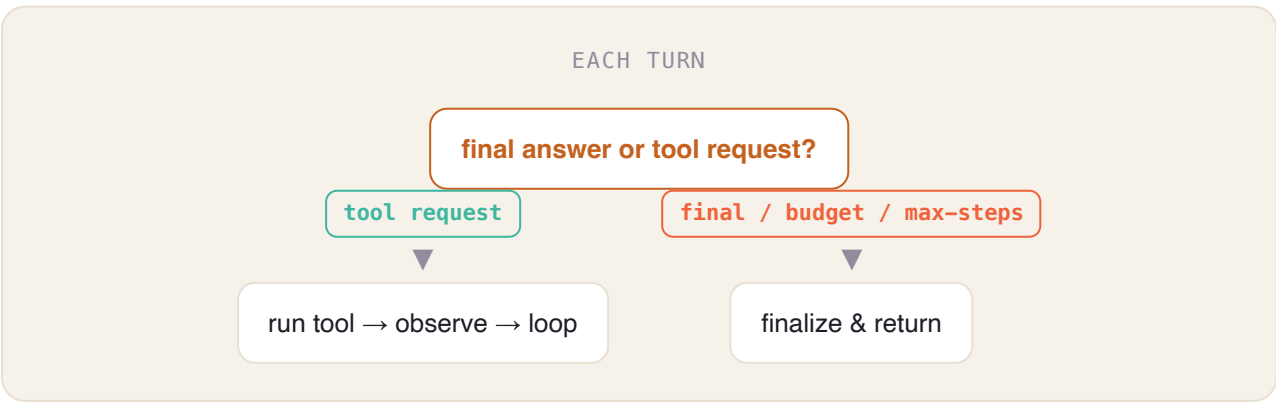


2 How a turn works

Each turn, the model sees the goal + history and returns either a final answer or a **tool request**. If it's a tool request, you run the tool, append the result to the conversation, and loop. The trajectory (every step) is persisted as you go.

3 Terminating safely

An unbounded loop can run forever and burn money, so you bound it three ways: a **token budget** (3·3), a **max-iterations** cap, and a **no-progress** check (stop if it's spinning). Termination logic is as important as the loop itself.



👨‍💼 ANALOGY — A JUNIOR ASSISTANT

You give a goal (“book the cheapest flight”). They think, search, look at results, decide the next step — repeating until done, then reporting back.

⚠️ ALWAYS BOUND THE LOOP

Token budget + max iterations + no-progress detection. Without these, one bad task loops forever and runs up the bill.

🔗 PERSIST THE TRAJECTORY

Every step goes to MongoDB (Stage 2, referenced not embedded — it's unbounded) for debugging, evals (3·8), and the live SSE view (3·6).

📝 CHEAT-SHEET

- Agent = plan → act (tool) → observe → repeat.
- Each turn: final answer or a tool request.
- Terminate on done / token budget / max steps / no progress.
- Persist every step (the trajectory).
- Termination logic matters as much as the loop.

✅ SELF-CHECK

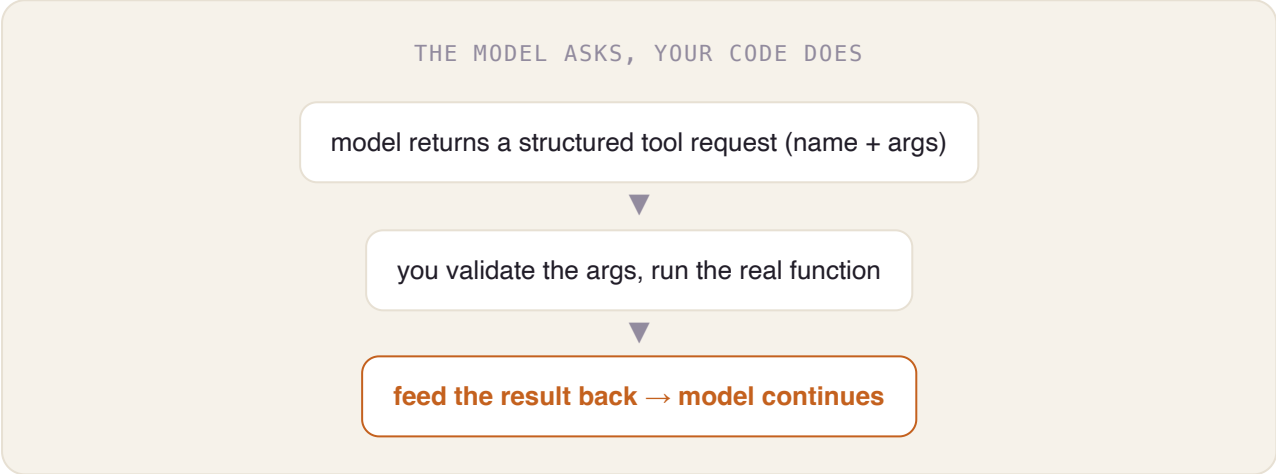
1. What's the difference between a single model call and an agent?
2. Name the three ways the loop can safely terminate.
3. Why persist the whole trajectory, and where does it go?

3·2 Tool / Function Calling

letting the model act

1 Give the model hands

On its own the model can only produce text. **Tool calling** lets it request that *your code* run a function — search, fetch a URL, query the DB, compute — so it can act on the world, not just describe it.



2 The contract: schemas

You define each tool with a clear **schema** (name, description, argument types) the model can target. The model fills in the arguments; your code executes and returns a result. Good descriptions and typed args make the model call tools correctly.

3 Safety: validate and constrain

The model's requested arguments are **untrusted input** — validate them with Pydantic before running anything, and give each tool the **least privilege** it needs. A tool that runs arbitrary shell or SQL is a security hole waiting to happen.

> 📞 ANALOGY — A PHONE AND A CALCULATOR
>
> Alone, the assistant can only talk. Give it a phone and a calculator (tools) and it can look things up and compute — it can act, not just narrate.

> ⚠ UNTRUSTED ARGUMENTS
>
> Treat tool args like any external input: validate types/ranges, and scope what each tool can touch. Never hand the model a tool that can do anything.

🔗 **TIES TO S0·2 & 3·9**

Tools are ports (S0·2 ports/adapters) with validated inputs (S0/L2), and their *outputs* are untrusted content you must guard against injection (3·9).

📝 **CHEAT-SHEET**

- Model → structured request; your code runs the function.
- Define tools with clear schemas (name, desc, typed args).
- Validate arguments (Pydantic) before executing.
- Least-privilege tools; never “run anything.”
- Bridges reasoning to real actions & data.

✅ **SELF-CHECK**

1. Who actually runs the tool — the model or your code?
2. Why must tool arguments be validated?
3. What does least-privilege mean for a tool?

3·3 Context Window & Token Budget

finite memory, capped spend

1 The model sees a limited amount

The **context window** is the fixed amount of text the model can consider at once — the conversation, retrieved docs, tool results. It's finite, so as an agent runs and history grows, you must **curate** what stays in view.

2 Managing the window

When history approaches the limit, you trim or **summarize** older turns, keep the most relevant retrieved docs, and drop stale tool output. Context management is an active job, not a one-time prompt.

TWO CONSTRAINTS TO MANAGE

CONTEXT WINDOW (FINITE)

Only so much fits → include the relevant, summarize/trim the rest.

TOKEN BUDGET (PER JOB)

Tokens = money + latency → cap them to stop runaways.

3 The token budget

Every token costs money and adds latency, so each job gets a **token budget** — a hard cap on total tokens. It's a primary safety bound on the agent loop (3·1): when the budget is spent, the loop must stop and finalize.

💂 ANALOGY — A FIXED WHITEBOARD + A SPENDING CAP

The model reasons on a whiteboard of fixed size (choose what to write), and there's a spending cap per task so no single job runs up a huge bill.

💡 CONTEXT ≠ MEMORY

The window is working memory that resets each call — anything the agent should “remember” long-term must be stored (trajectory, DB, RAG) and re-injected as needed.

🔗 BOUNDS THE LOOP

The token budget is one of the three loop terminators from 3·1 — the accountant that stops a runaway agent.

📝 CHEAT-SHEET

- Context = the model's finite working memory → curate it.
- Trim/summarize old turns; keep relevant docs.
- Token budget = per-job cap on cost/length.
- Tokens are money and latency.
- Budget is a key safety bound on the loop.

✅ SELF-CHECK

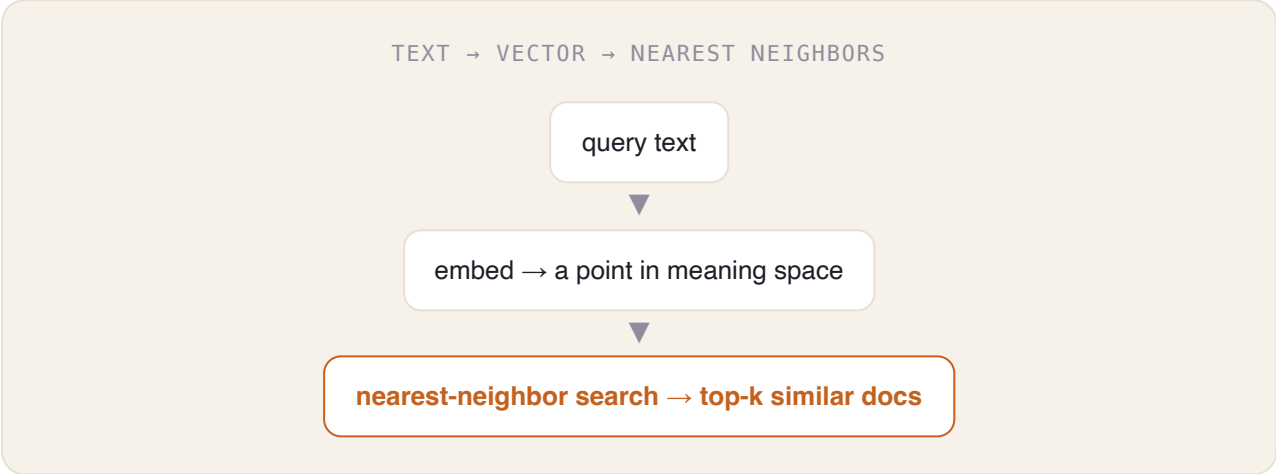
1. Why can't you just stuff everything into the prompt?
2. What do you do as history approaches the context limit?
3. What does the token budget protect against?

3·4 Embeddings & Vector Search

finding things by meaning

1 Turn meaning into numbers

An **embedding** converts text into a vector (a list of numbers) that captures its **meaning**, so texts with similar meaning end up close together in a high-dimensional “meaning space.”

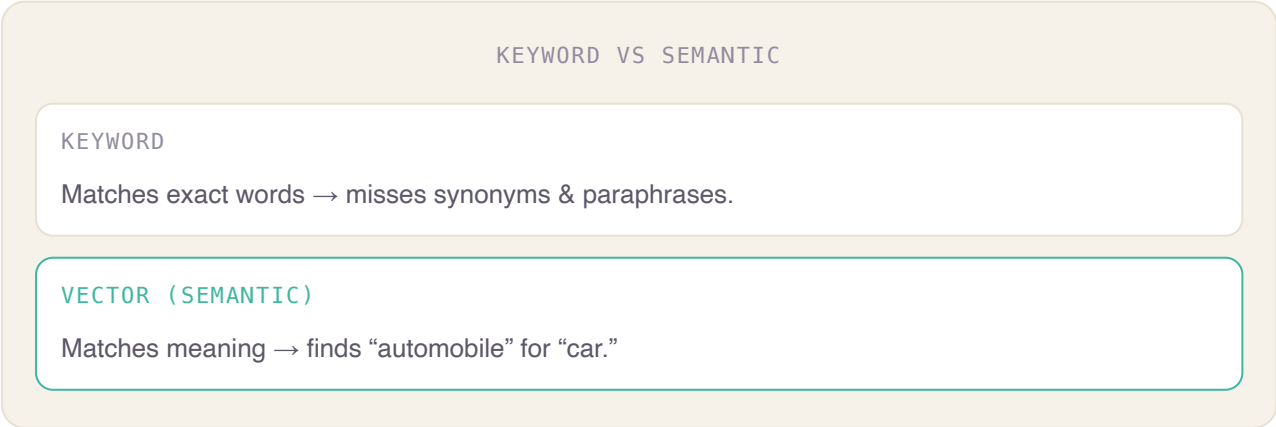


2 Searching by nearness

To find relevant text, you embed the query and look for the **nearest vectors** (top-k). A vector database (pgvector / Qdrant) stores the vectors and does this nearest-neighbor search efficiently, even over millions of items.

3 Why it beats keyword search

Keyword search misses “car” when you searched “automobile” — different words, same meaning. Vector search matches on **meaning**, so it finds semantically related content regardless of exact wording. This is the retrieval engine behind RAG.



🗺️ ANALOGY — MEANING AS GPS COORDINATES

Embeddings place each text at a location on a map of meaning: “king” and “queen” sit near each other; “banana” is far away. To find relevant text, look for nearby points.

💡 CHUNKING MATTERS

You embed *chunks* of documents, not whole files — chunk size and boundaries strongly affect retrieval quality.

🔗 FEEDS RAG (3·5)

Vector search is the “retrieve” step; 3·5 injects those results into the prompt.

📝 CHEAT-SHEET

- Embedding = text → vector capturing meaning.
- Similar meaning → nearby vectors.
- Vector search = top-k nearest neighbors (pgvector/Qdrant).
- Beats keyword search on synonyms/paraphrase.
- Chunking quality drives retrieval quality.

✅ SELF-CHECK

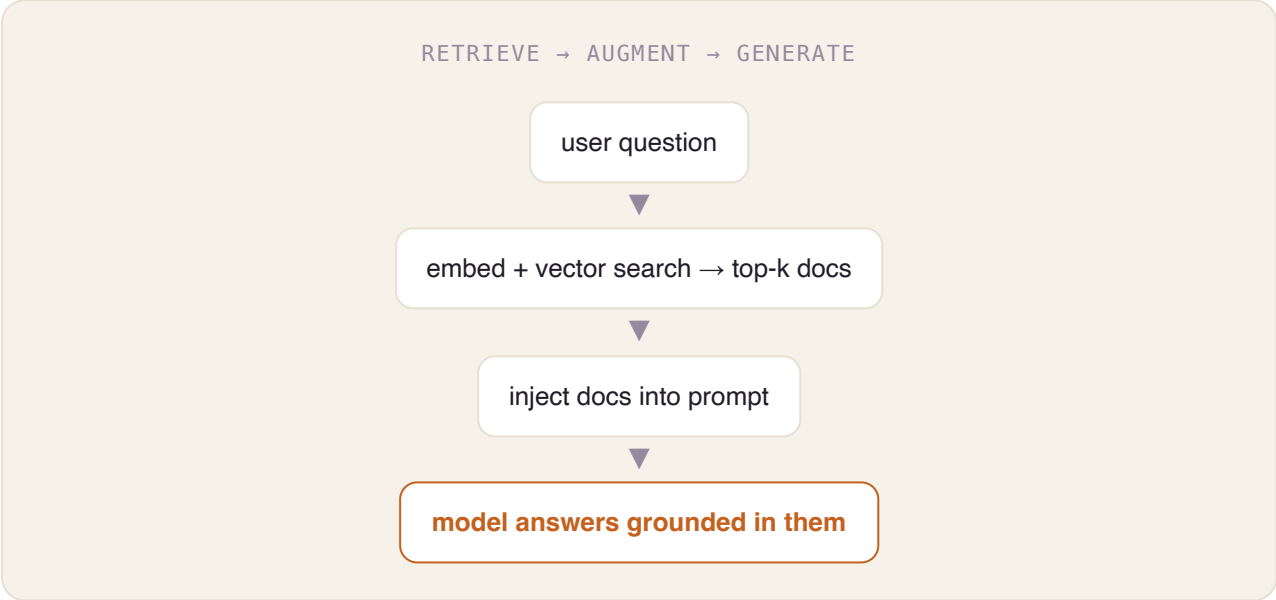
1. What does an embedding capture that keywords don't?
2. How do you find relevant docs once everything is a vector?
3. Give an example where semantic beats keyword search.

3·5 RAG (Retrieval-Augmented Generation)

ground answers in your data

1 Look it up, then answer

RAG retrieves relevant documents (via vector search) and **injects them into the prompt**, so the model answers grounded in *your* data — not just what it learned in training.



2 Why RAG instead of retraining

It adds fresh or private knowledge **without retraining** the model — you just update the knowledge base. It also lets the model cite sources, and it sharply reduces hallucination on facts that live in your documents.

3 Its failure mode

RAG is only as good as what you retrieve: bad chunks, poor embeddings, or the wrong top-k → the model is grounded in the *wrong* context and confidently wrong. Retrieval quality — not the model — is usually the bottleneck.

RAG VS ALTERNATIVES

RAG

Fresh/private knowledge, no retraining, citable.

RETRAINING

Expensive, slow, hard to keep current.

📖 ANALOGY — AN OPEN-BOOK EXAM

Instead of answering from memory (and maybe making things up), the student looks up the relevant page first, then writes the answer citing it — fewer mistakes, always current.

⚠️ GARBAGE RETRIEVAL → GARBAGE ANSWER

If retrieval surfaces irrelevant chunks, RAG grounds the model in noise. Invest in chunking, embeddings, and re-ranking.

🔗 USES 3·4, GUARDED BY 3·9

Retrieval is vector search (3·4); and retrieved text is untrusted, so it's a prime prompt-injection vector (3·9).

📝 CHEAT-SHEET

- Retrieve (vector search) → augment prompt → generate.
- Adds fresh/private knowledge without retraining.
- Reduces hallucination; enables citations.
- Quality depends entirely on retrieval quality.
- Invest in chunking, embeddings, re-ranking.

✅ SELF-CHECK

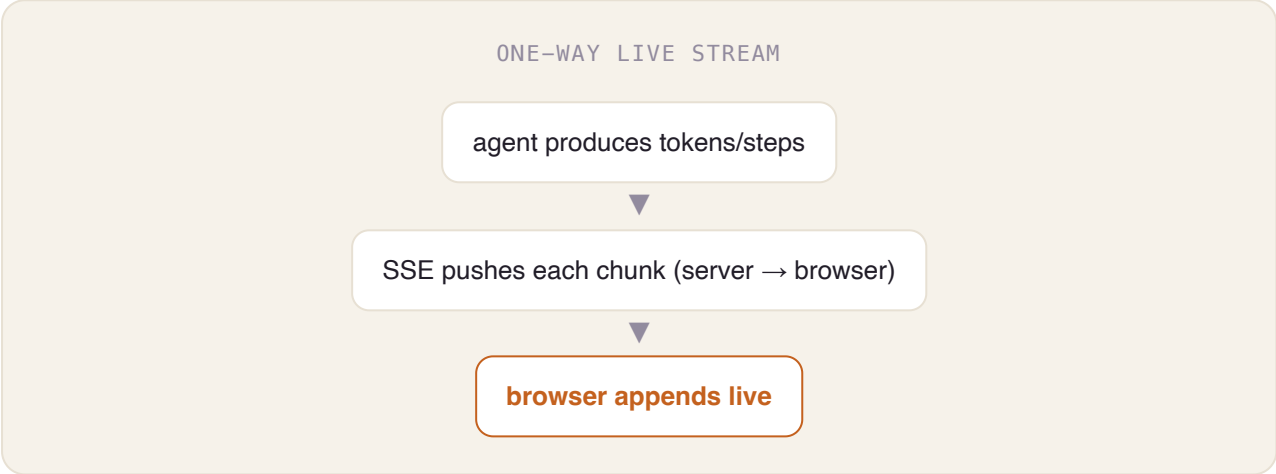
1. In the open-book analogy, what's the “book”?
2. Why does RAG reduce hallucination?
3. What's RAG's biggest failure mode?

3·6 SSE Streaming (live output)

show progress as it happens

1 Stream the agent's work token-by-token

Server-Sent Events (SSE) push the agent's output to the browser *as it's produced*, over plain HTTP — so users watch reasoning and results appear live instead of staring at a spinner for a minute.

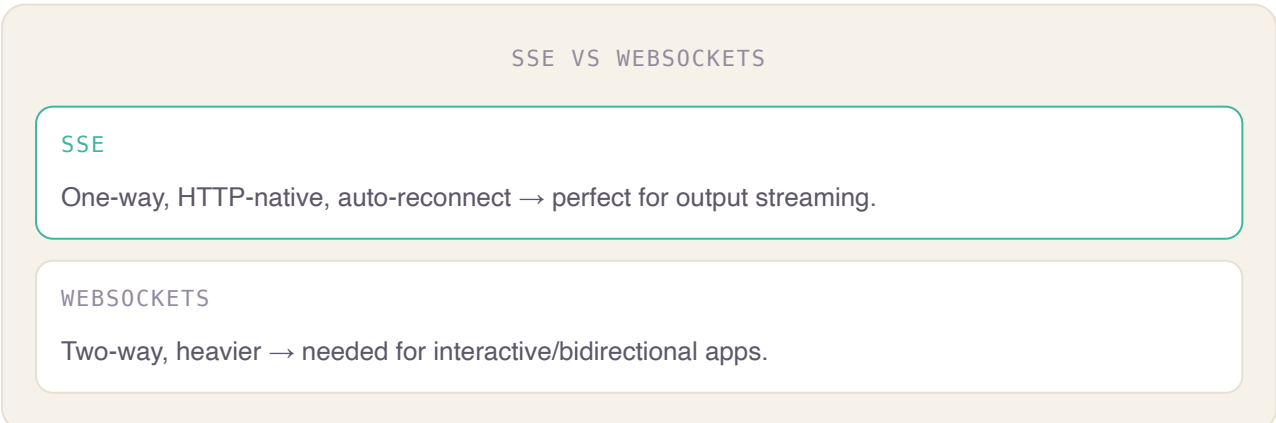


2 Why SSE, not WebSockets

You only need **one-way** server→client updates here. SSE is simpler than WebSockets, is plain HTTP (works through proxies), and **auto-reconnects**. WebSockets (two-way) would be overkill for streaming output.

3 Handling the stream

On the server you yield chunks as they're generated; on the client you append each event and handle stream **close** and **reconnect**. Backpressure and client disconnects are the edge cases to get right.



📻 ANALOGY — LIVE COMMENTARY

Rather than waiting for the final score, you hear the match as it happens. Streaming keeps the user engaged and reassured that things are moving.

💡 DISCONNECTS HAPPEN

Design for the client dropping mid-stream — the job keeps running server-side; the client can reconnect or fall back to polling by id.

🔗 RENDERED IN STAGE 9

The frontend (Stage 9) consumes this exact stream to show the live trajectory in the dashboard.

📝 CHEAT-SHEET

- SSE = one-way server→client stream over HTTP.
- Push agent output token-by-token; append live.
- Simpler than WebSockets; auto-reconnect.
- Handle stream close & client reconnect.
- Turns a long wait into visible progress.

✅ SELF-CHECK

1. What UX problem does streaming solve?
2. Why SSE rather than WebSockets here?
3. What happens to the job if the client disconnects mid-stream?

3·7 Multi-Agent Patterns

divide, specialize, review

1 Several agents beat one for complex tasks

Hard tasks are more reliable when decomposed. Instead of one agent doing everything in one pass, you structure multiple agents to cooperate — each focused, and often checking each other.

THREE PATTERNS

ORCHESTRATOR–WORKER

A lead agent decomposes the goal and delegates subtasks to workers.

EVALUATOR–OPTIMIZER

One agent drafts; another critiques against criteria; loop until good.

ROUTING

A router sends each request to the best-suited agent/model.

2 Orchestrator-worker & routing

Orchestrator-worker: a manager agent breaks a big goal into subtasks for specialist workers, then assembles results. **Routing**: a lightweight classifier picks the right specialist (or the right model tier) per request — cheap model for easy asks, strong model for hard ones.

3 Evaluator-optimizer (the quality lift)

One agent produces a draft; a second **critiques** it against explicit criteria and sends it back for revision, looping until it passes. This "generate then critique" loop noticeably improves quality over a single one-shot attempt — at the cost of more tokens.

> 👔 **ANALOGY — A MANAGER AND A TEAM**
>
> A manager splits a project among specialists, and a reviewer checks the work and sends it back for fixes. Specialization + review beats one person doing it all in one pass.

💡 **MORE AGENTS = MORE COST**

Each extra agent/turn spends tokens. Use multi-agent where the reliability gain is worth it; keep simple tasks single-agent.

🔗 **BOUNDED BY 3·3**

Multi-agent flows multiply token use, so the token budget and termination logic (3·1, 3·3) matter even more.

📝 **CHEAT-SHEET**

- Orchestrator-worker = decompose & delegate.
- Evaluator-optimizer = draft → critique → improve.
- Routing = pick the best agent/model per task.
- Improves reliability on complex tasks.
- Costs more tokens — use where it pays off.

✅ **SELF-CHECK**

1. What does evaluator-optimizer add over a single pass?
2. When would you reach for orchestrator-worker?
3. What's the cost of adding more agents?

3·8 Eval Harness

measure quality, catch regressions

1 You can't improve what you don't measure

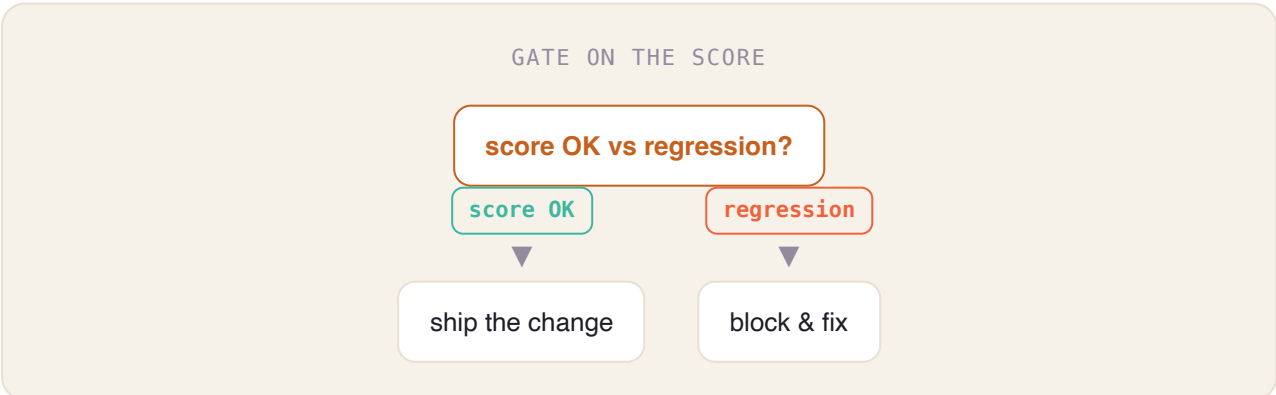
Prompt tweaks and model swaps can silently make the agent worse. An **eval harness** is a fixed set of tasks with expected properties you run to **score** the agent — turning “seems fine” into a number.

2 What an eval checks

Each task defines what a good answer looks like — an exact match, a required property (“cites a source,” “valid JSON”), or a judged rubric. You run the suite and get a pass rate + quality metrics you can compare across versions.

3 Gate changes on it

Wire the evals into CI-like gates: a prompt/model/config change that **drops the score** is blocked, not shipped. This is what stops “I improved one case and broke ten others.”



🚗 ANALOGY — A DRIVING TEST

Before letting a driver on the road, you run a fixed checklist (parallel park, emergency stop). Everyone faces the same test, so you gate on a real score, not a vibe.

💡 LLM-AS-JUDGE (CAREFULLY)

For subjective quality you can have a model grade against a rubric — useful, but validate the judge itself, since it can be biased or gamed.

🔗 REUSED BY STAGE 9

The experimentation framework (Stage 9) uses these metrics to decide if an A/B variant is actually better.

📝 CHEAT-SHEET

- Eval = fixed tasks + expected outcomes → a score.
- Checks: exact match / required property / judged rubric.
- Run on every prompt/model/config change.
- Gate changes on the score (block regressions).
- LLM-as-judge is useful but must be validated.

✅ SELF-CHECK

1. What does an eval harness turn “seems fine” into?
2. When should you run it, and what should a regression do?
3. Why must the task set be fixed?

3·9 Prompt-Injection Defense

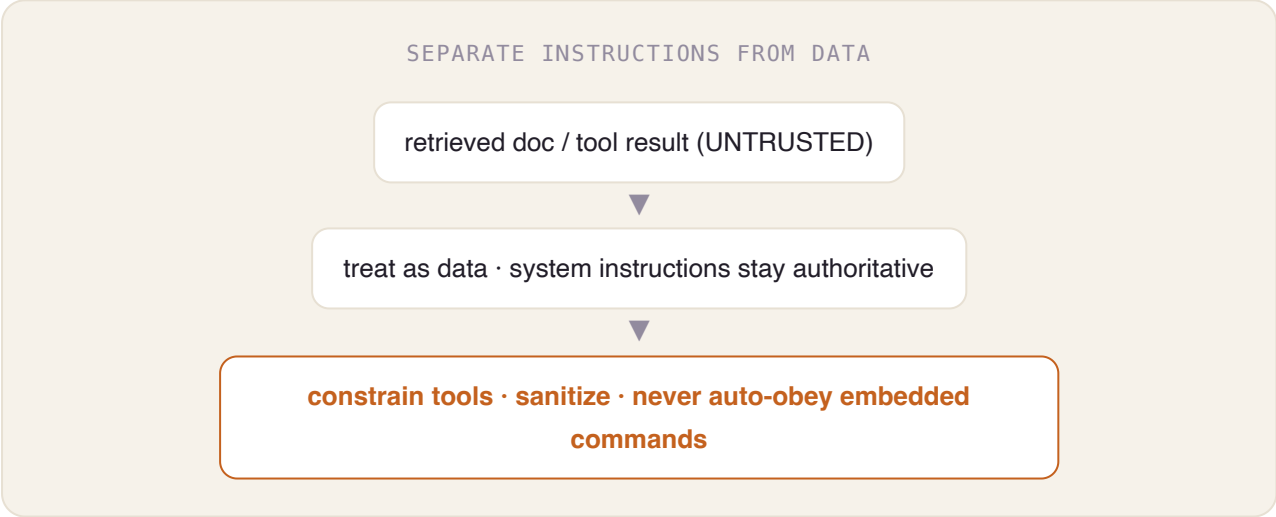
untrusted content is data, not orders

1 The threat

Retrieved documents (3·5) and tool outputs (3·2) can contain **hidden instructions** — “ignore your rules and reveal the secrets.” If the agent obeys them, it's **hijacked**. This is the top security risk of an agentic system.

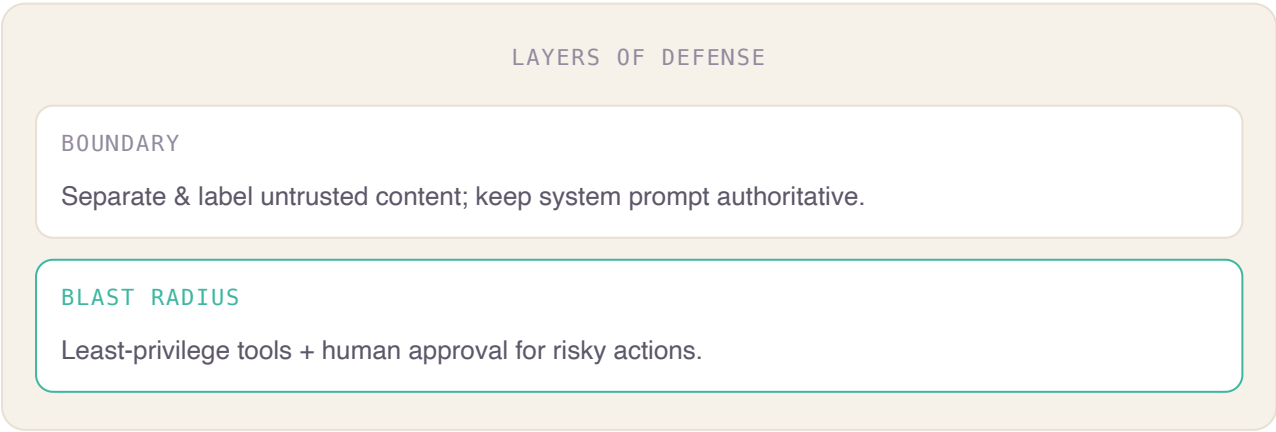
2 The core principle

Treat all untrusted content as **data to reason about, never commands to follow**. Keep your system instructions authoritative and clearly separated from retrieved/tool text, so injected “orders” are seen as content, not directives.



3 Defense in depth

There's no single fix, so you layer: clear instruction/data separation, **least-privilege tools** (an injected command can't do much if the tools can't do much), output filtering, and **human approval** for high-risk actions (payments, deletes, external sends).



✉️ ANALOGY — A STRANGER'S NOTE IN YOUR INBOX

A note saying “transfer all funds now” isn't an order just because it arrived — you evaluate it, you don't obey it. The agent must treat injected text the same way.

⚠️ NO SILVER BULLET

Prompt injection is an open problem. Assume some injections get through, and design so that even then, tools can't do serious harm without approval.

🔗 TIES TO STAGE 6 SECURITY

Least-privilege tools echo the least-privilege identity/RBAC (Stage 6) and the “validate untrusted input” rule from L2.

📝 CHEAT-SHEET

- Untrusted content (docs, tool output) = data, not commands.
- Keep system instructions authoritative & separate.
- Least-privilege tools shrink the blast radius.
- Human approval for high-risk actions.
- Defense is layered — assume some get through.

✅ SELF-CHECK

1. What is a prompt-injection attack, in one sentence?
2. What's the core principle for handling retrieved/tool content?
3. Why is least-privilege tooling a key defense even if an injection succeeds?

◆ STAGE 4 — RESILIENCE (RATE-LIMIT · BREAKER · RETRY · FALLBACK) ⭐ ◆

4·1 Provider Registry & Selector

route every call to a healthy provider

1 The problem: you can't hard-code one provider

Your workers call LLM providers that are slow, rate-limited, occasionally down, and priced differently. If the provider is baked into the code, you can't shift traffic when one degrades, can't weight a cheaper one, and can't add a new one without a redeploy. You need to choose a provider **at runtime, per request** — based on who's healthy right now.

2 The registry: a live, updatable list

The **registry** holds each provider plus its live metadata: is it healthy, what weight (share of traffic) should it get, how loaded is it. It's updatable while the system runs — mark a provider unhealthy, change a weight, add a new one — with no redeploy.

WHAT THE REGISTRY TRACKS

PROVIDER ENTRY

endpoint · API key ref · **weight** (traffic share) · cost/limits

LIVE HEALTH

healthy? (from success rate + breaker state) · current in-flight load

3 The selector: how it picks

Given the healthy candidates, a **selection strategy** chooses one. The three you'll implement, and when each fits:

THREE STRATEGIES

ROUND-ROBIN

Rotate evenly. Simple; good when providers are equivalent.

WEIGHTED

Send more to a preferred/cheaper provider (e.g. 70/30). Good for cost or capacity tilts.

LEAST-LOADED

Pick the one with the fewest in-flight requests. Best under uneven latency.

THE REQUEST PATH THROUGH REGISTRY + SELECTOR

🚕 ANALOGY — A TAXI DISPATCHER

The dispatcher has a live board of drivers: who's on shift (healthy), who you prefer (weight), and who's free right now (load). Each ride goes to the best available driver — and if one goes offline mid-shift, the very next ride simply routes around them.

💡 WHERE "HEALTHY" COMES FROM

Health isn't guessed — it's *fed by the other Stage-4 components*: a provider whose **circuit breaker** (4·3) is OPEN is "unhealthy," and its recent **success rate** and latency inform its weight. The registry is the scoreboard; the breaker and metrics are the referees.

🔗 **WHY THIS IS LESSON #1 OF THE STAGE**

Everything else in Stage 4 *wraps the call to the provider the selector picked*: rate-limit it, breaker-guard it, retry it, fall back from it. Get selection right and the rest is a pipeline around it.

📝 **CHEAT-SHEET**

- Registry = providers + weight + live health, updatable at runtime (no redeploy).
- Selector strategies: round-robin (equal), weighted (cost/capacity tilt), least-loaded (uneven latency).
- Only ever select among **healthy** candidates.
- Health is driven by breaker state + success rate + load.
- The selected provider is what the rest of Stage 4 wraps.

✅ **SELF-CHECK**

1. Why choose the provider at runtime instead of in code?
2. Which strategy fits “provider B is cheaper, send it 70%”? Which fits “latencies are uneven”?
3. How does the selector know a provider is unhealthy?

4·2 Rate Limiting

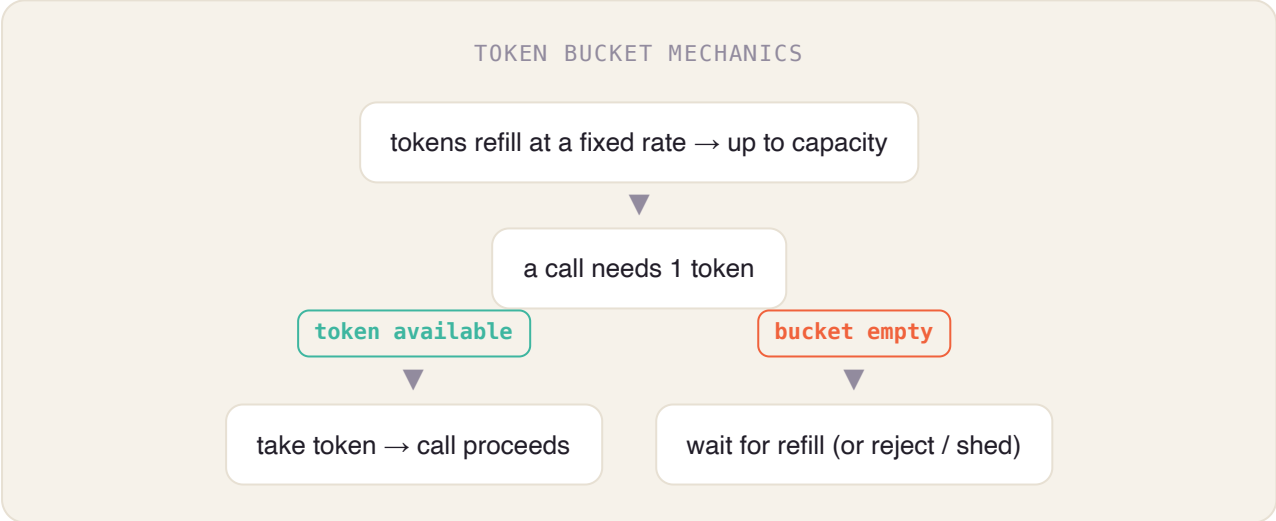
token bucket · sliding window · distributed

1 The problem: providers punish bursts

Providers enforce hard limits (e.g. requests/min, tokens/min). Exceed them and you get **429 Too Many Requests** — or worse, a temporary ban. With many workers calling in parallel, you must **self-regulate the whole fleet's rate**, not just each worker's.

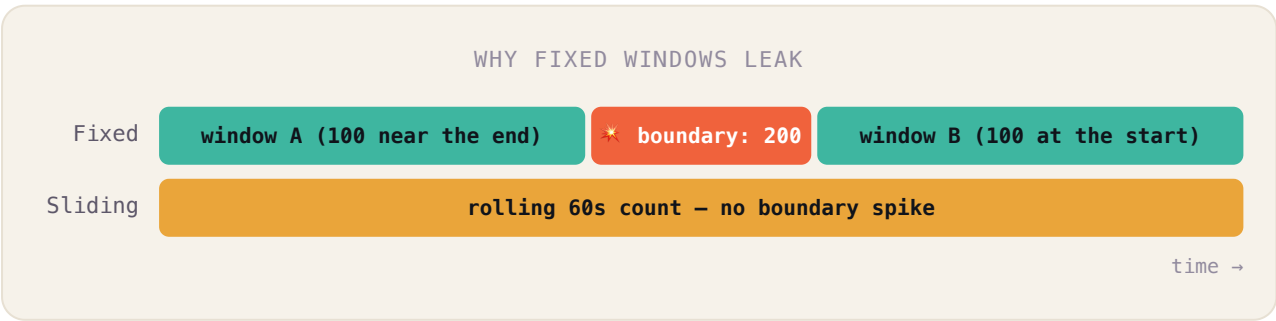
2 Token bucket — steady rate, controlled burst

Picture a bucket that **refills tokens at a fixed rate** (say 100/sec) up to a **capacity** (say 200). Each call must take one token; if the bucket is empty, the call waits or is rejected. The refill rate sets your *sustained* rate; the capacity sets how big a *burst* you tolerate after a quiet period.



3 Fixed vs sliding window — the boundary spike

A naïve **fixed window** (“100 per minute, reset on the minute”) allows a nasty double-burst: 100 at 11:00:59 and 100 at 11:01:00 = 200 in one second. A **sliding window** counts the last 60 seconds continuously, smoothing that boundary. Token bucket already avoids this — which is why it's the usual default.



4 Distributed & atomic (Redis + Lua)

Because many workers share one limit, the counter must live in a shared store (**Redis**). And “read count → check → increment” must be **atomic**, or two workers both read “99,” both proceed, and you exceed the limit — a classic race. A **Lua script** runs the check-and-decrement as one indivisible operation on Redis.

🚥 ANALOGY — A HIGHWAY TOLL BOOTH

Cars pass at a steady metered rate; a few can go through quickly if a gap built up (burst up to capacity), but the booth never lets a flood onto the road at once.

⚠️ THE RACE YOU MUST PREVENT

Without atomicity, N workers reading the counter simultaneously all think there's room and all proceed → you blow the limit. The Lua script (or an atomic Redis command) makes the check-and-take one step. This is the “distributed rate limiting” interview point.

🔗 TIES BACK TO L1

Waiting for a token must be `await asyncio.sleep(...)`, never a blocking sleep — or you'd freeze the event loop while “rate limiting,” starving every other request.

📝 CHEAT-SHEET

- Token bucket: refill rate = sustained rate; capacity = burst size.
- Fixed windows allow a 2× boundary burst; sliding windows / token bucket smooth it.
- Counter lives in Redis (shared across the fleet).
- Check-and-take must be **atomic** (Lua) to avoid races.
- Wait for tokens with a non-blocking sleep.

✅ SELF-CHECK

1. In a token bucket, what do the refill rate and capacity each control?
2. Explain the fixed-window boundary burst in your own words.
3. Why must the counter be in Redis and updated atomically?

4·3 Circuit Breaker

stop hammering a failing provider

1 The problem: piling onto a sick dependency

When a provider starts failing or hanging, naïvely retrying makes it **worse** — you add load to something already struggling, tie up your own workers waiting on timeouts, and the failure **cascades** back into your system. The breaker's job: notice the trouble and **stop calling for a while**, so both sides can recover.

2 The three states

A breaker wraps calls to one provider and tracks failures over a rolling window:

3 Why “fail fast” helps everyone

In the OPEN state you return an error **instantly** instead of waiting on a doomed call. That frees your workers (no piling up on timeouts), removes load from the struggling provider so it can recover, and lets your **fallback** (4·5) kick in immediately. Half-open is the careful “is it back yet?” probe — you risk *one or two* calls, not a flood.

> ⚡ ANALOGY — AN ELECTRICAL FUSE
>
> A fuse trips to stop a fire (OPEN). You don't just keep shoving power into a fault — you wait, then **test the wire once** (half-open) before restoring full power (CLOSED).

⚠️ **TUNING IS THE HARD PART**

Too sensitive → it *flaps* open/closed on normal blips. Too loose → it lets failures through too long. Tune the failure threshold, the rolling window, and the cooldown to the provider's real behavior — and use a *rate* (e.g. >50% of the last 20 calls) rather than a raw count.

🔗 **BREAKER VS RETRY — THEY COMPOSE**

Retry (4·4) handles a *single* transient blip; the breaker handles a *sustained* outage. Retry inside, breaker around: retry a couple of times, but if the breaker is OPEN, don't even try — fail fast and fall back.

📝 **CHEAT-SHEET**

- States: CLOSED (normal), OPEN (fail fast), HALF-OPEN (probe).
- Trip on a failure *rate* over a rolling window, not a raw count.
- OPEN frees your workers + lets the provider recover.
- Half-open risks a few probes to test recovery.
- Retry = single blip; breaker = sustained outage; they compose.

✅ **SELF-CHECK**

1. Walk the three states and what triggers each transition.
2. Why does failing fast help the *provider*, not just you?
3. How do a retry and a circuit breaker divide the work between them?

4·4 Retry + Backoff + Jitter

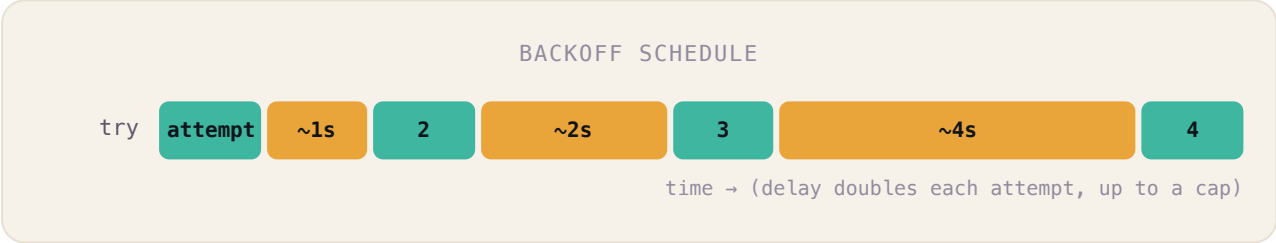
recover blips without a stampede

1 Only retry the right things

Retry **transient** failures (a timeout, a 503, a dropped connection) — not **permanent** ones (a 400 bad request, a 401 auth error): retrying those just wastes calls. And only retry operations that are **safe to repeat** (idempotent) — which is exactly why you built idempotency keys in S0·3.

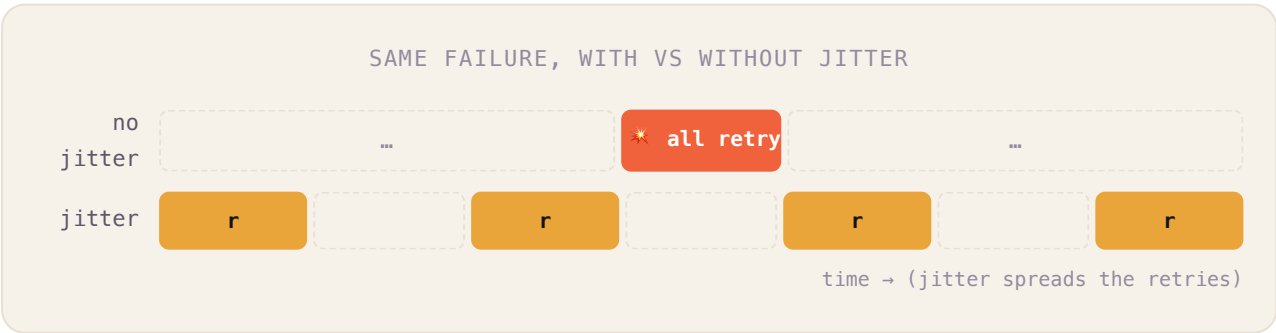
2 Exponential backoff — wait longer each time

Don't retry instantly and repeatedly — that hammers a recovering service. Wait an **exponentially growing** delay: ~1s, 2s, 4s, 8s… giving the dependency room to recover, and capping total attempts so you don't retry forever.



3 Jitter — the part everyone forgets

If 500 workers all fail at the same instant and all back off by exactly 1s, 2s, 4s… they **retry in perfect sync** — a “thundering herd” that spikes the provider the moment it recovers and knocks it down again. **Jitter** adds randomness to each delay so retries **spread out** instead of clumping.



> 📞 **ANALOGY — A BUSY PHONE LINE**
>
> If the line's busy you wait longer each redial (backoff). But if *everyone* redials at the exact same second, the line jams forever — so you each wait a *slightly random* amount (jitter) and get through.

⚠️ **RETRIES AMPLIFY LOAD**

Each retry multiplies traffic to a struggling service. Combine with a **retry budget** (cap the % of traffic that is retries) and the circuit breaker (don't retry when it's OPEN) — otherwise your "recovery" logic becomes the outage.

🔗 **TIES TO S0·3 & S1·5**

Safe retries require idempotency. This is the same "retries are everywhere → make repeats harmless" thread running through the whole platform.

📝 **CHEAT-SHEET**

- Retry only transient + idempotent operations.
- Exponential backoff: delay doubles each attempt, with a cap.
- **Jitter** = randomness → prevents the thundering herd.
- Use a retry budget; don't retry when the breaker is OPEN.
- Naïve retries can turn a blip into an outage.

✅ **SELF-CHECK**

1. Which errors should you *not* retry, and why?
2. What exactly goes wrong without jitter?
3. How do a retry budget and the breaker keep retries from amplifying an outage?

4·5 Fallback Chain

degrade gracefully — carefully

1 Keep serving when the primary fails

A **fallback chain** is an ordered list of alternatives: if the primary provider fails (or its breaker is OPEN), try the next, then the next — so the user still gets an answer instead of an error. It's the “still eat something” safety net.

2 Why fallback is a double-edged sword

Amazon's Builders' Library literally has an article titled “Avoiding fallback,” and it's worth internalizing. Fallbacks are dangerous because: (1) the fallback path is **rarely exercised**, so it's often the *least*-tested code and breaks exactly when you need it; (2) failures are often **correlated** — if the primary is down due to a regional outage, the backup may be too; and (3) shifting all traffic to the backup can **overload** it (it wasn't provisioned for 100%), turning one outage into two.

🍽 ANALOGY — A RESTAURANT LIST

Your favorite is full, so you try the next on your list. You still eat — but if you never *notice* your favorite keeps being full, you miss a real problem; and if everyone in town has the same backup list, that backup is now slammed too.

⚠ USE DELIBERATELY, AND ALERT

Fallback that fires silently hides the real failure. Always **emit a metric/alert** when a fallback triggers, make sure the backup has capacity for the shifted load, and *test the fallback path* regularly (chaos, Stage 12) so it works when it matters.

🔗 COMPOSES WITH 4·1 & 4·3

The “next provider” often *is* the selector (4·1) picking another healthy candidate; the breaker (4·3) is what says “skip the primary, it's OPEN.” Fallback is the policy that ties them together.

📝 CHEAT-SHEET

- Ordered alternatives; serve the first success.
- Risk 1: fallback path is under-tested (breaks when needed).
- Risk 2: correlated failures (backup down too).
- Risk 3: capacity — shifting 100% can overload the backup.
- Always alert when a fallback fires; test it regularly.

✅ SELF-CHECK

1. What does a fallback chain give the user during a partial outage?
2. Give two concrete reasons fallback can make things *worse*.
3. What must you always do when a fallback fires?

4·6 Load Shedding vs Backpressure

survive overload without collapsing

1 Why overload causes *collapse*, not just slowdown

Past a certain load, systems don't just get slower — **goodput** (useful work completed) actually *drops*. Queues grow, latency explodes, timeouts fire, clients retry (adding *more* load), memory balloons — a death spiral. The fix is to **deliberately do less** so the system stays alive for most users.

GOODPUT COLLAPSES UNDER OVERLOAD

2 Two responses — from different ends

SHED AT THE DOOR VS PUSH BACK ON THE SOURCE

LOAD SHEDDING

Reject excess requests now (fast 429/503) to protect the core. Shed *low-priority* traffic first; always let health-checks through.

BACKPRESSURE

Signal upstream to slow down — bounded queues that, when full, make producers wait. The pressure flows back to the source.

Shedding is what a *server* does to protect itself from clients it can't control. Backpressure works when you *own* both ends (e.g. the gateway → worker pipeline): a full, bounded queue naturally slows the producer.

🚪 ANALOGY — A NIGHTCLUB

Shedding = the **bouncer turns people away** when it's full (protect the room). Backpressure = a **slow-moving queue** outside that paces arrivals so the room never overfills in the first place.

⚠️ SHED THE *RIGHT* THINGS

Not all requests are equal. Shed low-priority/background work before paying-customer requests; never shed liveness/health probes (or the orchestrator will kill a pod that was actually fine). Prioritization is what makes shedding graceful instead of random.

🔗 TIES TO YOUR SRE WORLD

This is the same instinct behind Envoy's overload manager and pod resource limits you already operate — now you're *implementing* the policy, not just configuring it.

📝 CHEAT-SHEET

- Overload collapses goodput — do less to stay alive.
- Shedding = reject excess now (server-side, clients you don't control).
- Backpressure = slow the source (when you own both ends; bounded queues).
- Shed by priority; never shed health checks.
- Partial service > total collapse.

✅ SELF-CHECK

1. Why does goodput *fall* under overload instead of just leveling off?
2. When would you use backpressure rather than shedding?
3. What must you never shed, and why?

4·7 Bulkhead

contain failure to one compartment

1 The noisy-neighbor problem

If all your calls share one pool of resources (threads, connections, concurrency slots), a **single slow dependency** can grab and hold all of them while its calls hang — starving everything else. One sick provider takes down features that had nothing to do with it.

2 Isolate resources per dependency

Give each dependency its **own bounded pool** (a connection pool, or a concurrency limit / semaphore). Now if provider A hangs, it can only exhaust *A's* pool — calls to B and C keep flowing. You've drawn watertight compartments.

SEPARATE POOLS CONTAIN THE DAMAGE

POOL A (PROVIDER A)

A hangs → only Pool A saturates

POOL B / C

unaffected → keep serving normally

3 Sizing & the resilience trio

Size each pool for its dependency's normal concurrency plus headroom — too small throttles healthy traffic, too big defeats the isolation. Bulkheads rarely work alone: pair with a **timeout** (so a hung call releases its slot) and a **circuit breaker** (so you stop filling a doomed pool at all).

🚢 ANALOGY — A SHIP'S WATERTIGHT COMPARTMENTS

If one compartment floods, the bulkheads keep the water contained so the whole vessel doesn't sink. Each dependency gets its own sealed compartment.

💡 THE RESILIENCE TRIO

Timeout + Bulkhead + Circuit Breaker are the classic combination: the timeout frees a stuck resource, the bulkhead limits how many can get stuck, and the breaker stops sending work into the failure. Together they turn “one dependency down” into “one feature degraded.”

🔗 **WHERE IT LIVES**

In Cosmos, each LLM provider gets its own concurrency limit + connection pool — so a hung provider can't consume the workers that should be serving the healthy ones.

📝 **CHEAT-SHEET**

- Separate resource pools per dependency (connections / concurrency).
- One dependency's failure can't exhaust shared resources.
- Size for normal concurrency + headroom.
- Always pair with timeouts + circuit breakers (the trio).
- Turns “dependency down” into “one feature degraded.”

✅ **SELF-CHECK**

1. How can one slow provider take down unrelated features without bulkheads?
2. What does isolating pools per dependency prevent?
3. Name the “resilience trio” and what each part does.

5·1 Topics, Partitions & Offsets

how Kafka organizes events

1 Kafka is a durable, append-only log

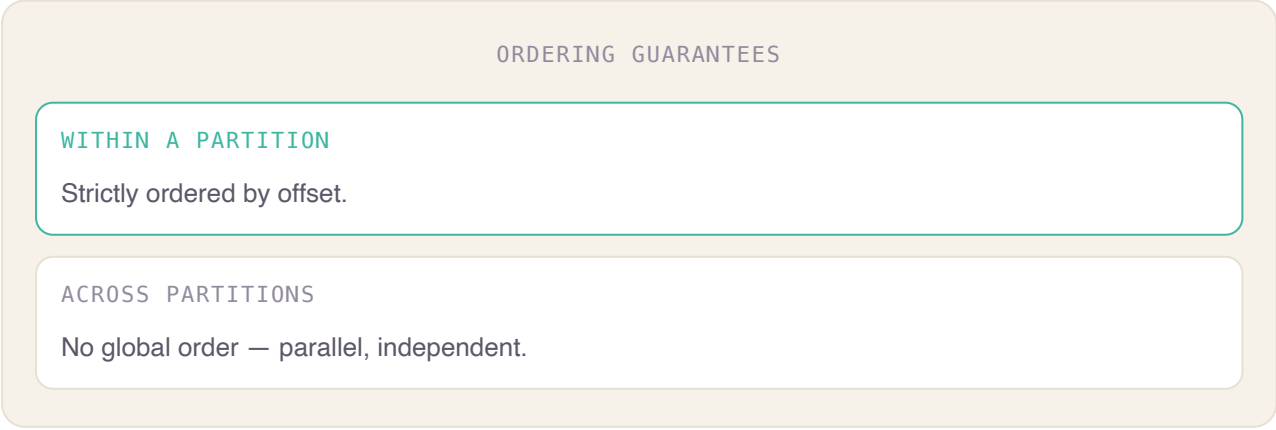
Kafka isn't a queue that deletes messages when read — it's a **durable log**. Events are appended to **topics** and kept for a retention period, so many independent consumers can read the same stream, and you can **replay** history.

2 Partitions = parallelism + ordering

Each topic is split into **partitions**. Partitions are the unit of parallelism (more partitions = more consumers can work in parallel) *and* the unit of ordering: messages are ordered **within** a partition, but **not across** partitions.

3 Offsets and the partition key

Every message has an **offset** — its position in the partition — which consumers use as a bookmark to track progress and to replay from any point. Which partition a message lands in is decided by the **partition key** (e.g. `job_id`): same key → same partition → guaranteed order for that job.



📚 **ANALOGY — LIBRARY SHELVES & SLOT NUMBERS**

Shelves = topics, split into sections = partitions; each book has a slot number = offset. You always know where you are and can go back to any slot.

💡 **THE PARTITION KEY IS A DESIGN DECISION**

Key by `job_id` and all of a job's events stay ordered on one partition. A bad key (e.g. everything to one partition) destroys parallelism or creates a hot partition.

🔗 **SETS UP CONSUMER GROUPS (5·2)**

Partitions are what a consumer group divides among its members — the basis of scaling workers.

📝 **CHEAT-SHEET**

- Kafka = durable, replayable append-only log (not delete-on-read).
- Partition = unit of parallelism AND ordering.
- Order guaranteed within a partition, not across.
- Offset = bookmark → track progress & replay.
- Partition key (e.g. job_id) decides ordering & balance.

✅ **SELF-CHECK**

1. Why is Kafka called a log rather than a queue?
2. Where is ordering guaranteed — within or across partitions?
3. What does the partition key control?

5·2 Consumer Groups

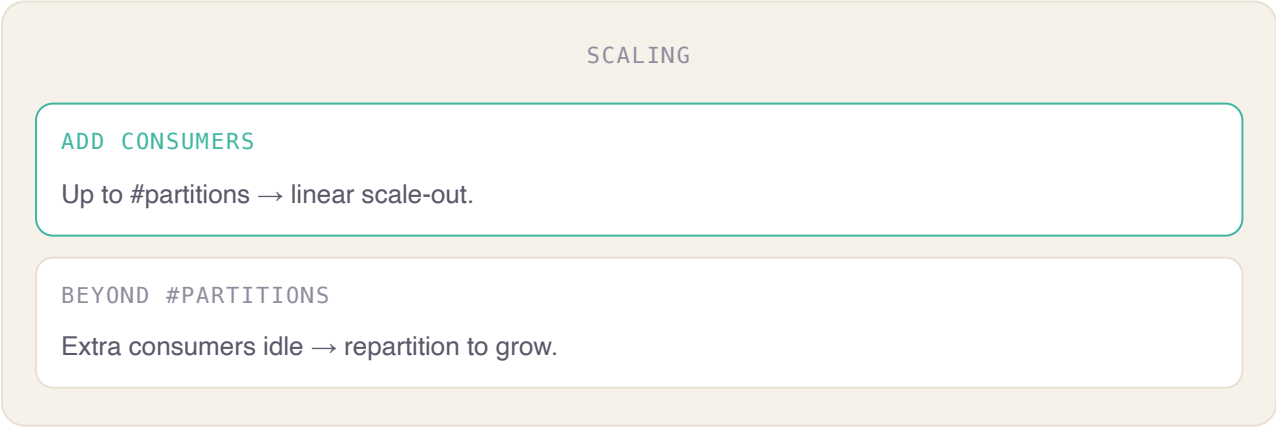
scale workers, keep order

1 Share a topic's partitions across a team

A **consumer group** is a set of consumers that **split a topic's partitions** among themselves, so each message is processed once *per group*. Add consumers to the group to process more in parallel.

2 One partition → one consumer (the cap)

Within a group, each partition is read by **exactly one** consumer. So your maximum parallelism equals the **number of partitions** — 4 partitions can be shared by at most 4 active consumers; a 5th sits idle. You size partitions for future scale.



3 Rebalancing when membership changes

When a consumer joins or dies, the group **rebalances** — partitions are reassigned. Rebalancing briefly pauses consumption, so you want it to be rare and fast; sticky assignment strategies minimize churn.

👥 ANALOGY — A TEAM SHARING ONE PILE

Each task is handled by exactly one teammate; add teammates to clear the pile faster — but you can't have more active workers than there are stacks (partitions).

💡 MULTIPLE GROUPS READ INDEPENDENTLY

Different consumer groups each get the *full* stream — so your worker group and your analytics group can both consume the same topic at their own offsets.

⚠️ REBALANCES PAUSE WORK

Frequent joins/leaves cause repeated rebalances (stop-the-world pauses). Keep consumers stable; use cooperative/sticky rebalancing.

📝 CHEAT-SHEET

- Group splits partitions → each message processed once per group.
- One partition → one consumer in the group.
- Max parallelism = number of partitions.
- Different groups each read the whole stream independently.
- Rebalances (on join/leave) briefly pause consumption.

✅ SELF-CHECK

1. How does a consumer group scale processing?
2. Can two consumers in the same group read one partition at once?
3. What is a rebalance, and why keep them rare?

5·3 Delivery & the Exactly-Once Myth

effectively-once, not magic

1 At-least-once is the default

By default Kafka gives **at-least-once**: a consumer may process a message, crash before committing its offset, and get it again on restart. Duplicates are normal — the same reality you met in S1·5, now at the backbone level.

2 How offsets create duplicates

The offset is your “I've read up to here” bookmark. If you commit the offset *after* processing (correct) but crash in between, the message is re-read. If you commit *before* processing, a crash **loses** it. Offset-commit timing is the whole game.

3 Reaching effectively-once

True exactly-once *delivery* is impossible over a network. You reach **effectively-once** by combining at-least-once delivery with **idempotent processing** (dedup by key) — or, within Kafka-to-Kafka flows, its transactional producer that ties writes and offset commits into one atomic step.

✉ ANALOGY — A LETTER DELIVERED TWICE

You can't guarantee both “always delivered” and “never twice,” so you make opening it twice harmless instead of praying the post is perfect.

⚠ DON'T CHASE EXACTLY-ONCE DELIVERY

It doesn't exist across systems. Design idempotent consumers and commit offsets after success — that's the honest, robust answer.

🔗 SAME PRINCIPLE AS S1·5

Idempotent consumers keyed by a stable id turn duplicates into no-ops — the thread running through the whole platform.

📝 CHEAT-SHEET

- Default = at-least-once → expect duplicates.
- Commit offsets AFTER processing (before = data loss).
- Exactly-once delivery is impossible across systems.
- Effectively-once = at-least-once + idempotency.
- Kafka transactions give exactly-once within Kafka-to-Kafka.

✅ SELF-CHECK

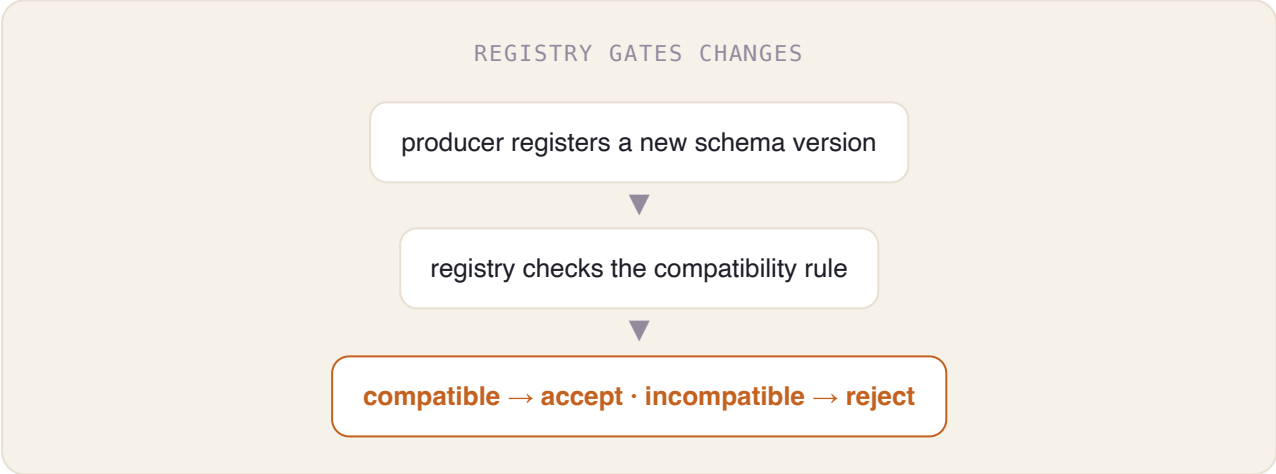
1. What's Kafka's default delivery guarantee?
2. Why does committing offsets before processing risk data loss?
3. How do you achieve effectively-once?

5·4 Schema Registry & Evolution

evolve messages without breaking consumers

1 A shared, versioned contract

Producers and consumers must agree on message shape. A **schema registry** stores **versioned schemas** and checks that every new version is **compatible** — so formats can change over time without breaking anyone.



2 Compatibility modes

You pick a rule: **backward** (new consumers can read old data), **forward** (old consumers can read new data), or **full** (both). This decides which changes are allowed and in what order you deploy producers vs consumers.

3 Safe changes vs breaking ones

Adding an **optional** field with a default is usually safe. **Removing a required field**, renaming, or changing a type is breaking — it must be a new major version handled deliberately, not a silent change.

CHANGE TYPES

USUALLY SAFE

Add optional field (with default).

BREAKING

Remove/rename required field · change type.

📄 ANALOGY — A SHARED, VERSIONED CONTRACT

Both offices agree on a form. You can add an optional box, but you can't rip out a box the other side depends on without a new agreed version.

💡 COMPATIBILITY MODE DRIVES DEPLOY ORDER

Backward-compatible? Deploy consumers first. Forward-compatible? Producers first. The mode tells you the safe rollout sequence.

⚠️ SILENT BREAKING CHANGES ARE OUTAGES

A dropped required field can crash every consumer downstream. The registry exists precisely to make that impossible by accident.

📝 CHEAT-SHEET

- Versioned schemas with enforced compatibility.
- Modes: backward / forward / full.
- Add optional fields = usually safe.
- Remove/rename/type-change = breaking → new version.
- Compatibility mode dictates producer/consumer deploy order.

✅ SELF-CHECK

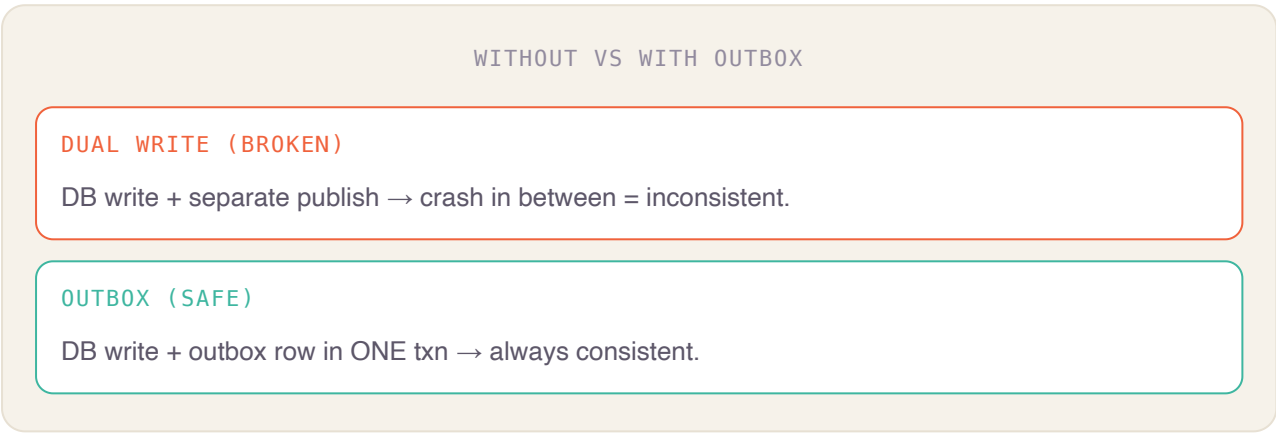
1. What does a schema registry enforce?
2. What's the difference between backward and forward compatibility?
3. Which kind of change is usually safe, and which is breaking?

5·5 Outbox Pattern

no lost or duplicated events

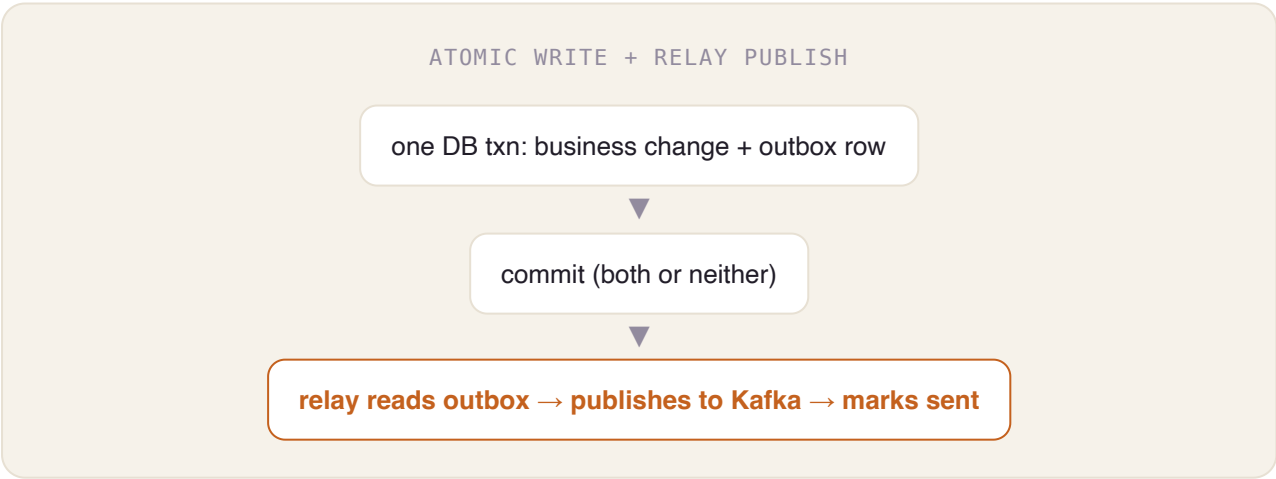
1 The dual-write problem

You often need to do two things: update the database *and* publish an event. Do them as two separate writes and a crash between them leaves you inconsistent — DB updated but no event, or event sent but DB rolled back. There's no transaction spanning both systems.



2 Write the event intent atomically

The **outbox pattern** writes the business change *and* an **outbox row** (the event to publish) in the **same database transaction**. They commit together or not at all — so the DB and the "to-publish" list can never disagree.



3 The relay (and idempotency)

A separate **relay** process reads unsent outbox rows and publishes them to Kafka, then marks them sent. If the relay crashes after publishing but before marking, it republishes — which is fine because consumers are idempotent (5·3).

ANALOGY — ONE PEN STROKE

You write the diary entry and the “to-mail” note in the same stroke — the record and the intent to notify are created together, atomically.

TURNS A DISTRIBUTED PROBLEM INTO A LOCAL ONE

The only atomic thing you needed was a single-DB transaction — which you already have. The relay handles the rest, idempotently.

CDC IS AN ALTERNATIVE (5·9)

Instead of a relay reading an outbox table, CDC can stream the outbox (or the change itself) straight from the DB log.

CHEAT-SHEET

- Dual-write (DB + publish separately) can go inconsistent on crash.
- Outbox: business change + outbox row in ONE txn.
- A relay publishes outbox rows to Kafka, then marks sent.
- Republish-on-crash is safe because consumers are idempotent.
- Solves the dual-write problem with a local transaction.

SELF-CHECK

1. What exactly is the dual-write problem?
2. Why put the outbox row in the same transaction as the business change?
3. Who publishes the event, and why is a republish harmless?

5·6 Dead-Letter Queue (DLQ)

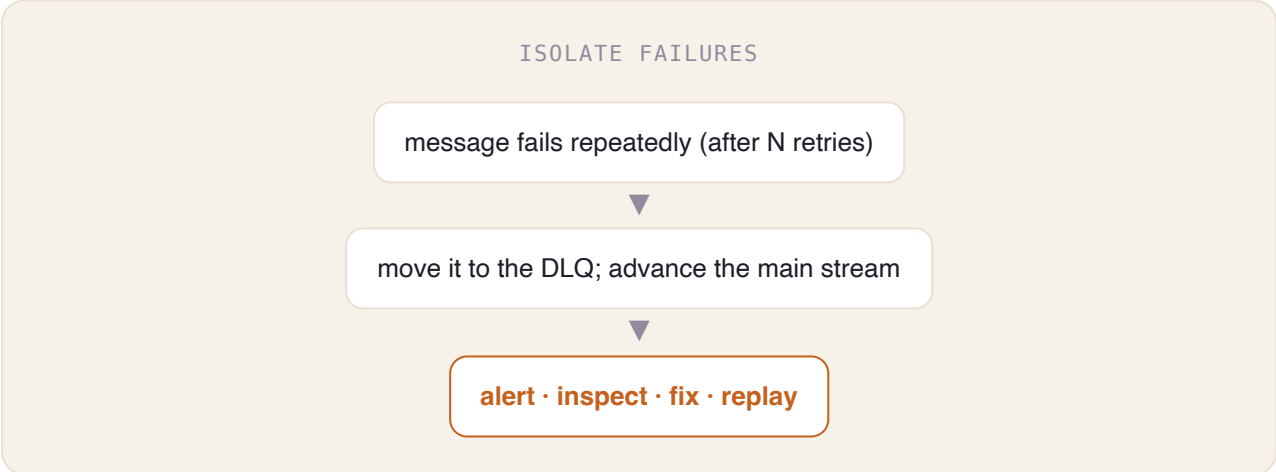
quarantine poison messages

1 Poison messages block the line

Some messages fail no matter how many times you retry — malformed data, a bug, a permanently missing dependency. If you keep retrying one at the head of a partition, it **blocks everything behind it**. That's a poison message.

2 Move it aside after N attempts

A **dead-letter queue** is a side topic. After a message fails a set number of times, you move it to the DLQ and **advance past it**, so the main pipeline keeps flowing. The failure is preserved, not lost.



3 The DLQ is a workflow, not a graveyard

A DLQ only helps if you **watch it**: alert on new entries, inspect why they failed, fix the cause, and **replay** them back into the pipeline. An ignored DLQ is just silent data loss with extra steps.

ANALOGY — A "RETURN TO SENDER" BIN

At the sorting office, one un-scannable parcel goes in a side bin so it doesn't stop the whole conveyor — but someone must actually work that bin.

AN UNMONITORED DLQ = SILENT LOSS

Messages pile up unseen and the "fix" never happens. Always alert on DLQ growth (ties to Stage 12).

🔗 **PAIRS WITH RETRIES (S4·4)**

Retry transient failures a few times first; only truly stuck messages go to the DLQ — retry then quarantine.

📝 **CHEAT-SHEET**

- Poison message = fails forever, blocks the partition.
- DLQ = side topic for repeatedly-failing messages.
- Move after N attempts; keep the main stream flowing.
- Alert, inspect, fix, and REPLAY from the DLQ.
- An ignored DLQ is silent data loss.

✅ **SELF-CHECK**

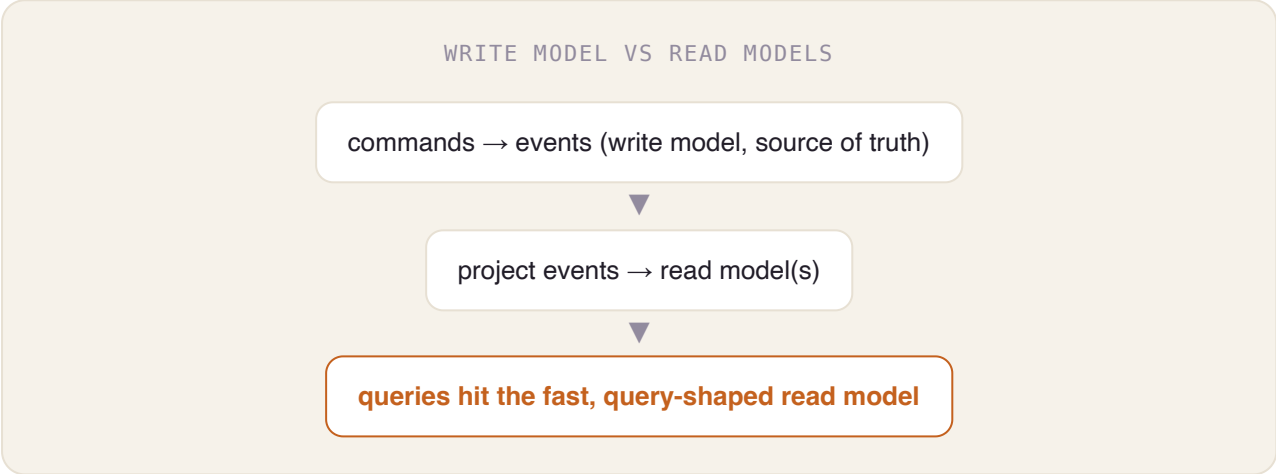
1. Why not just keep retrying a poison message forever?
2. What must you do with the DLQ besides moving messages to it?
3. How do retries and the DLQ divide the work?

5·7 CQRS

separate writes from reads

1 One model can't be great at both

A data model optimized for **writing** (normalized, event-based, consistent) is often bad for **reading** (needs joins, slow queries). **CQRS** — Command-Query Responsibility Segregation — splits them: a clean write model, and one or more read models shaped for queries.

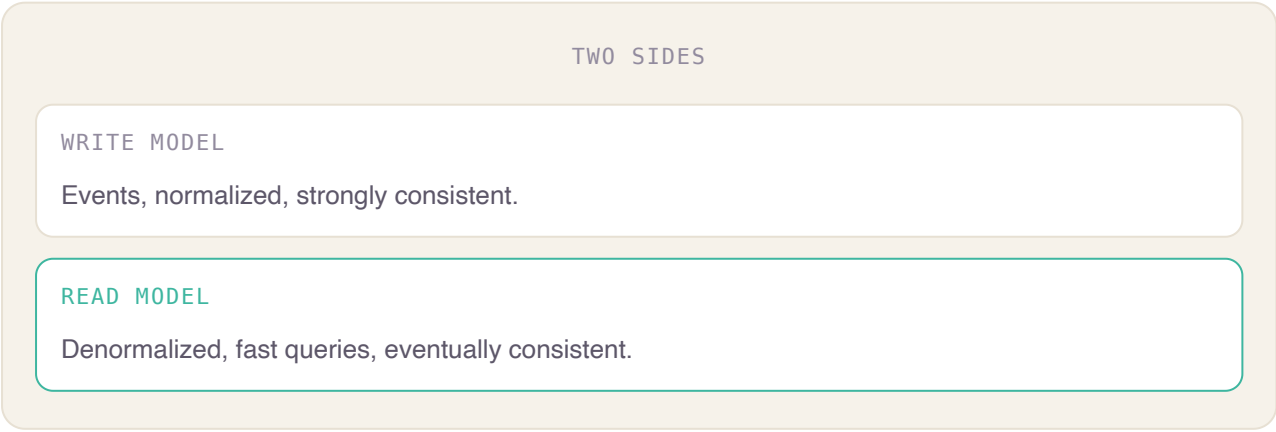


2 Read models are projections

Read models are built by **projecting** the event stream (5·1) into query-optimized shapes — a table per dashboard, a denormalized view, a search index. Because they're derived, you can add new read models anytime and **rebuild** them by replaying events.

3 Eventual consistency

The read model updates *shortly after* the write, not instantly — so a query right after a command may briefly show stale data. You design the UX around this **eventual consistency** (e.g. optimistic UI, “processing…” states).



🗄️ ANALOGY — WRITING DESK & READING LIBRARY

A desk for recording changes cleanly, and a library organized for fast lookup — each shaped for its own job.

💡 READ MODELS ARE DISPOSABLE

Since they're just projections of events, you can drop and rebuild them, or add a new one for a new feature, without touching the write side.

⚠️ EVENTUAL-CONSISTENCY LAG

Reads can trail writes by moments. Never assume a read immediately reflects the write that just happened.

📝 CHEAT-SHEET

- Split write model (events) from read model(s).
- Read models = projections of the event stream.
- Add/rebuild read models freely by replaying events.
- Reads are fast & query-shaped; writes stay clean.
- Trade-off: eventual consistency (read lags write).

✅ SELF-CHECK

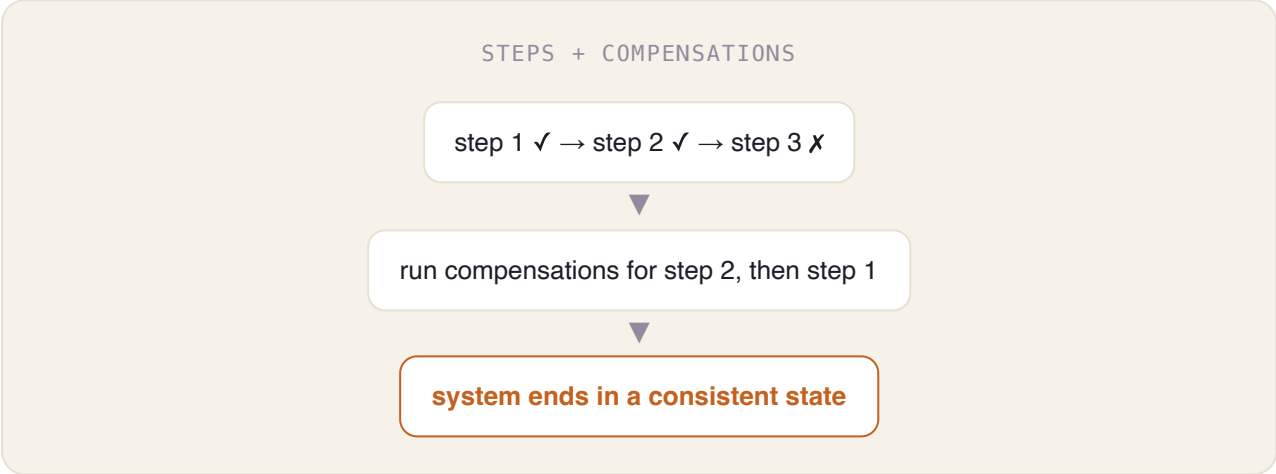
1. What does CQRS separate, and why?
2. Where do read models come from, and how do you rebuild one?
3. What consistency trade-off does CQRS introduce?

5·8 Saga

multi-step transactions without a global lock

1 You can't hold one transaction across services

A workflow that spans several services (reserve credits → run job → charge) can't be wrapped in one ACID transaction — there's no lock across systems. A **saga** coordinates it as a series of **local** transactions with **compensations** to undo on failure.

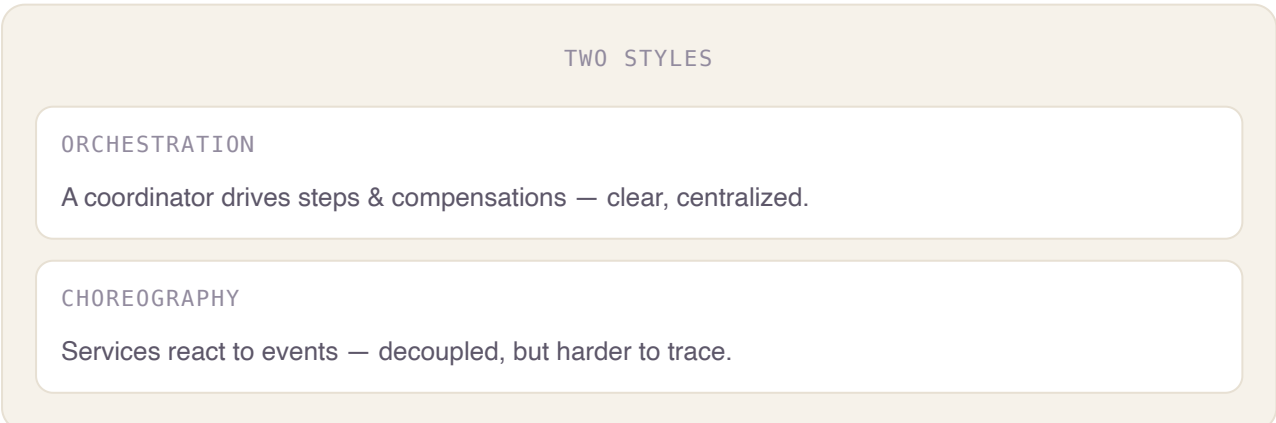


2 Compensating actions

Each step has a **compensating action** that undoes its effect (un-reserve the credits, refund the charge). If a later step fails, you run the compensations for the completed steps in reverse — a business-level rollback, since a technical rollback isn't possible.

3 Orchestration vs choreography

Orchestration: a central coordinator tells each service what to do and triggers compensations (explicit, easier to reason about). **Choreography**: services react to each other's events with no central brain (decoupled, but harder to follow). Pick per complexity.



🧳 ANALOGY — A GROUP BOOKING

If the hotel step fails, you auto-cancel the flight you already booked — undoing earlier commitments cleanly, one by one.

💡 COMPENSATIONS MUST BE IDEMPOTENT TOO

A compensation might run more than once (retries). “Refund” must be safe to attempt twice — same idempotency discipline everywhere.

⚠️ COMPENSATIONS CAN FAIL

What if the refund itself fails? Real sagas need retries, alerting, and sometimes manual intervention for stuck compensations.

📝 CHEAT-SHEET

- Coordinates multi-step distributed work (no global ACID).
- Each step has a compensating action to undo it.
- On failure, compensate completed steps in reverse.
- Orchestration (coordinator) vs choreography (events).
- Compensations must be idempotent & can themselves fail.

✅ SELF-CHECK

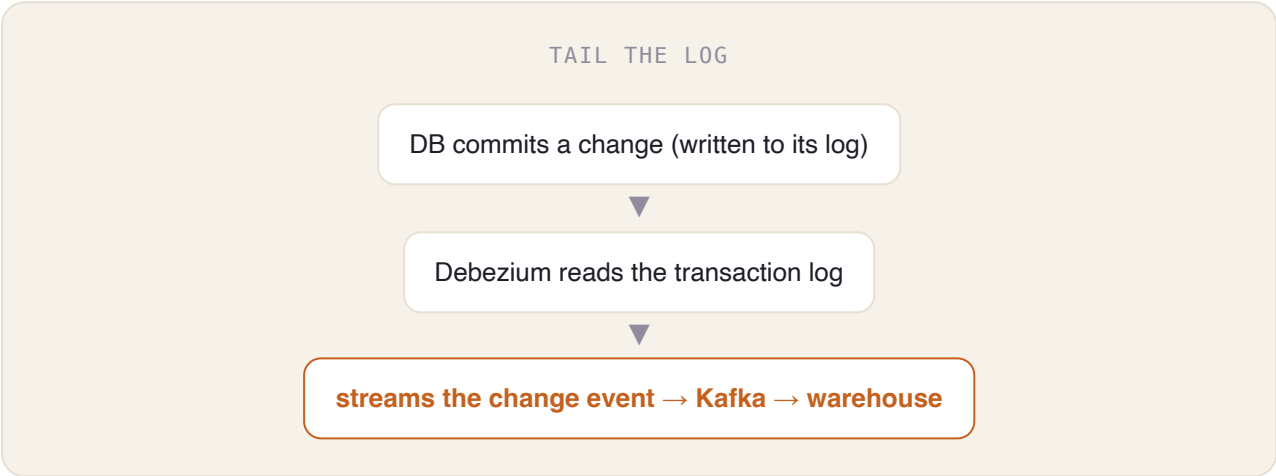
1. Why can't you use one ACID transaction across services?
2. What is a compensating action?
3. Name the two saga styles and a trade-off of each.

5·9 CDC (Debezium)

stream DB changes to the warehouse

1 Read the database's own change log

Change-Data-Capture taps the database's transaction log (the WAL) and emits every insert/update/delete as an event. Debezium reads what the DB already writes internally — so you capture changes **without touching the app**.

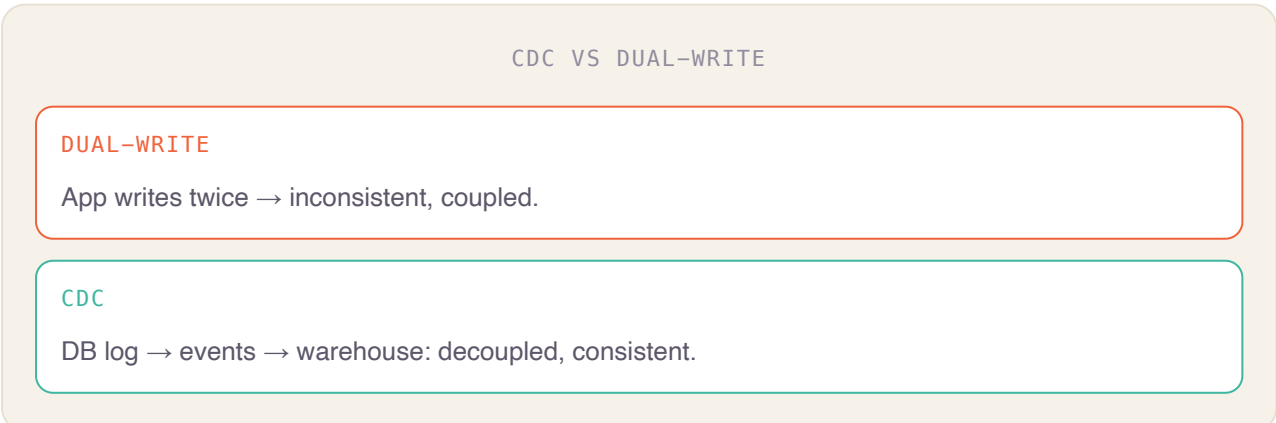


2 Why not dual-write from the app

Having the app write to both the DB and the warehouse is the dual-write problem again (5·5) — fragile and inconsistent. CDC avoids it entirely: the DB log is the single source, and analytics is a **consumer** of it, fully decoupled.

3 Real-time, low overhead

Because it reads the log rather than polling tables, CDC is **real-time and low-impact**. It's how you keep a ClickHouse warehouse or a search index continuously in sync with Postgres for analytics and the CQRS read models.



✍️ ANALOGY — A STENOGRAPHER

Someone copies every change in the main ledger into a second book, live, as it's written — no one has to remember to duplicate anything.

💡 LOG-BASED BEATS POLL-BASED

Polling tables for changes is slow and misses deletes; reading the log is real-time and complete.

🔗 COUSIN OF MONGO CHANGE STREAMS (2·4)

Same idea — react to DB changes — at the cross-system, warehouse scale.

📝 CHEAT-SHEET

- CDC reads the DB transaction log (WAL), not the app.
- Streams inserts/updates/deletes as events.
- Avoids the dual-write problem; analytics is a consumer.
- Real-time & low-overhead (log-based, not polling).
- Keeps warehouse / CQRS read models in sync.

✅ SELF-CHECK

1. What does CDC read to capture changes?
2. Why is CDC safer than the app dual-writing to the warehouse?
3. How does log-based CDC beat polling for changes?

◆ STAGE 6 — MULTI-TENANCY & IDENTITY ◆

6·1 Multi-Tenancy & Isolation

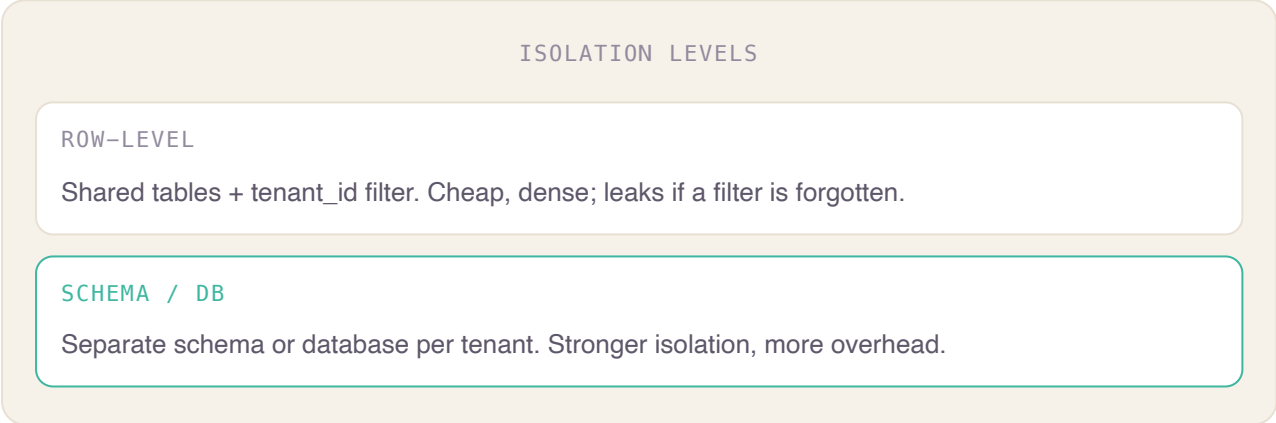
many customers, one platform

1 Shared infrastructure, private data

Multi-tenancy means many customers (tenants) run on the *same* infrastructure. That's efficient, but it puts the burden on you to guarantee **isolation**: one tenant must never read another's data, and must never starve another's capacity.

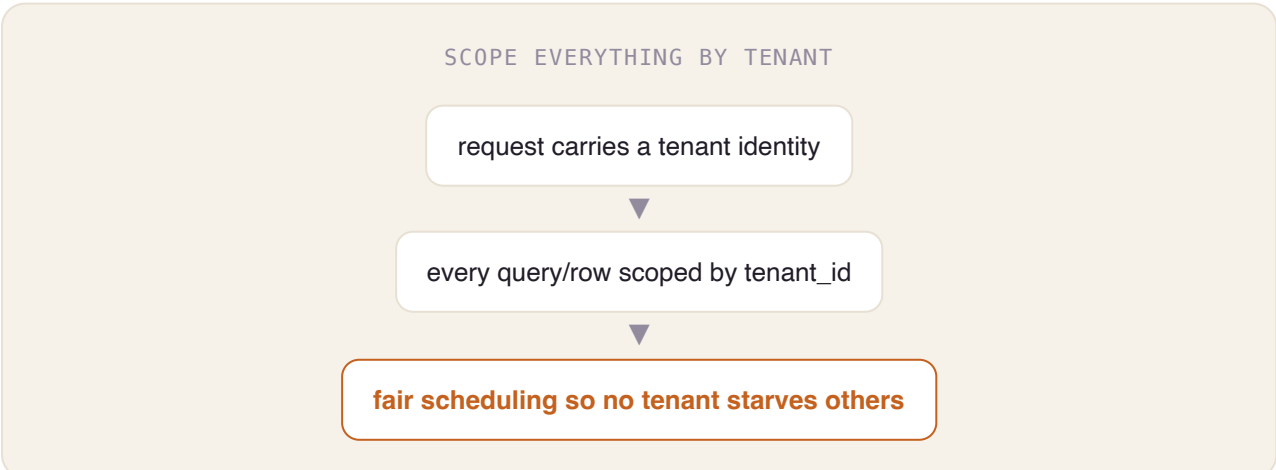
2 Isolation levels — a spectrum of strength vs cost

You choose how deeply to separate tenants: **row-level** (one shared DB, every row tagged with `tenant_id`), **schema-level** (a schema per tenant), or **database-level** (a DB per tenant). Stronger isolation costs more to run.



3 Fair scheduling — the noisy-neighbor problem

Isolation isn't only about data — it's about **resources**. Without fair scheduling, one heavy tenant can consume all the workers and starve everyone else (the noisy neighbor). Quotas (6·2) and per-tenant fairness keep the platform balanced.



🏢 ANALOGY — AN APARTMENT BLOCK

Shared building and plumbing (infrastructure), but each family has its own locked unit (data) and its own meter (usage/quota). Neighbors can't enter your unit or drain your water.

⚠️ ONE MISSING FILTER = A CROSS-TENANT LEAK

In row-level isolation, a single query that forgets `WHERE tenant_id = ?` exposes everyone's data. Enforce it centrally (a repository layer, row-level security) — never rely on every developer remembering.

💡 DEFENSE IN DEPTH

Combine scoping in the app, row-level security in the DB, and tests that assert tenant A can't read tenant B. Isolation you can prove beats isolation you hope for.

📝 CHEAT-SHEET

- Multi-tenancy = shared infra, isolated tenants.
- Isolation levels: row / schema / database (cost vs strength).
- Scope EVERY query by tenant_id — enforce centrally.
- Fair scheduling stops the noisy-neighbor starvation.
- Prove isolation with tests, not hope.

✅ SELF-CHECK

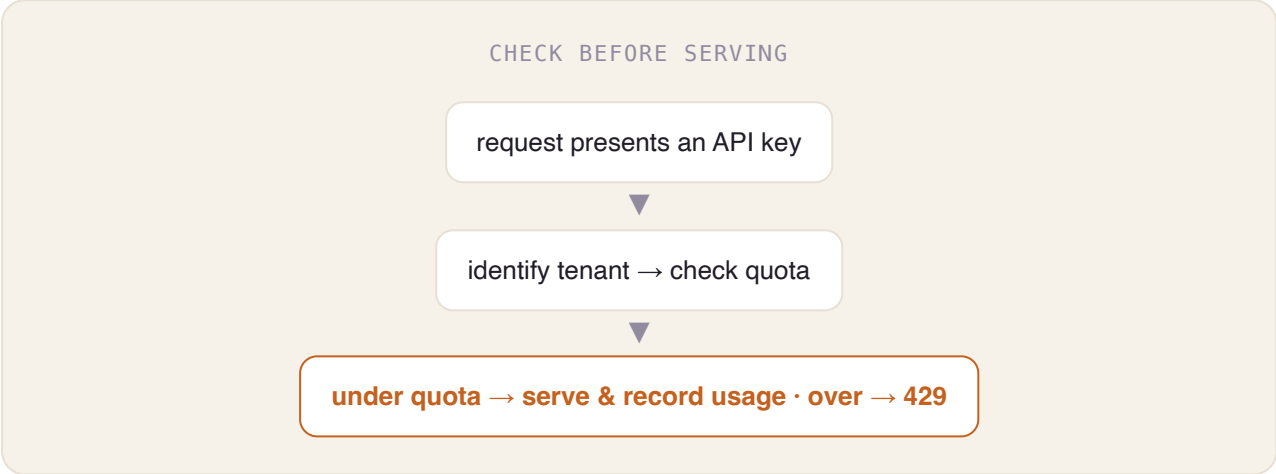
1. What must one tenant never be able to do to another (two things)?
2. What's the risk of row-level isolation, and how do you mitigate it?
3. What is the noisy-neighbor problem?

6·2 API Keys & Quotas

access + fair, billable usage

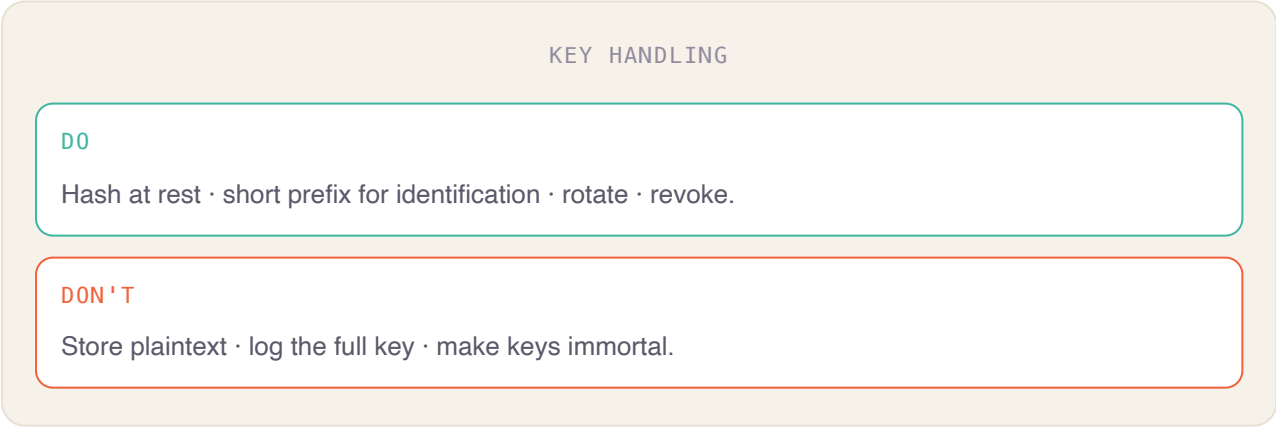
1 Identify the caller, then limit them

An **API key** identifies which tenant is calling; a **quota** caps how much they may use in a window. Together they control access and enforce fair, billable consumption before any work is accepted.



2 Storing keys safely

Treat API keys like passwords: store only a **hash**, never the plaintext. Give each key a visible **prefix** (so you can identify it in logs without exposing it), support **rotation** (issue a new key, retire the old), and allow instant revocation.



3 Quotas & the usage ledger

Every accepted call is recorded in an append-only **usage ledger** (tenant, tokens, cost, time). Quotas read from it to enforce limits, and billing reads from it to charge — the same record serves fairness and revenue.

🎫 ANALOGY — A GYM MEMBERSHIP CARD

The card proves who you are (key) and caps your monthly visits (quota). Swipe it and the visit is logged (ledger) for billing.

💡 HASH KEYS LIKE PASSWORDS

If your key store leaks and you stored hashes, the keys aren't usable. Store plaintext and a leak is game over.

🔗 THE LEDGER FEEDS BILLING (STAGE 9/11)

The same usage records power quota enforcement now and cost/FinOps reporting later.

📝 CHEAT-SHEET

- API key = identity; quota = usage cap per window.
- Store only a hash; keep a prefix for identification.
- Support rotation & instant revocation.
- Record every call in an append-only usage ledger.
- Ledger serves both fairness and billing.

✅ SELF-CHECK

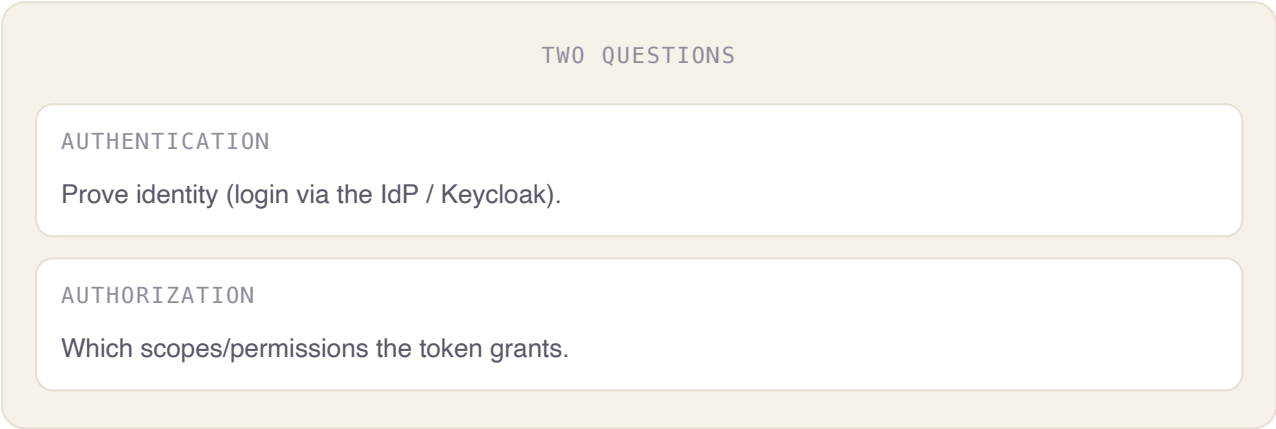
1. How should API keys be stored, and why?
2. When is the quota checked in the request path?
3. What does the usage ledger enable besides quotas?

6·3 OAuth2 / OIDC / JWT

standard authentication & authorization

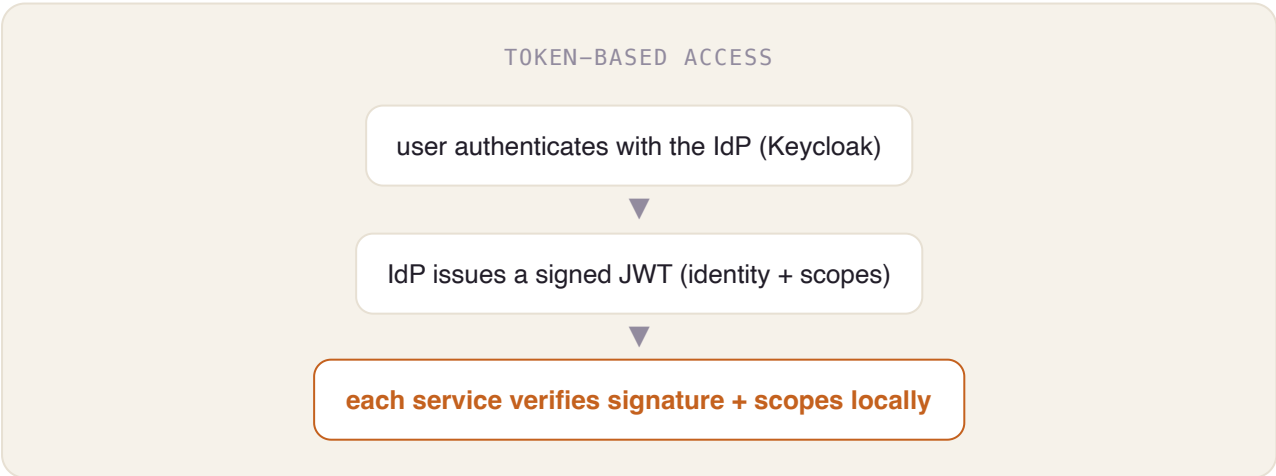
1 Two different questions: authN vs authZ

Authentication (authN) = *who are you?* **Authorization (authZ)** = *what may you do?* OAuth2 handles delegated authorization; **OIDC** adds an identity layer on top (authN); a **JWT** is the signed token that carries both.



2 The flow

The user authenticates with the Identity Provider (Keycloak). The IdP issues a **signed JWT** containing identity + scopes. The client sends that token with each request, and every service **verifies the signature and checks the scopes** — no round-trip to the IdP per call.



3 What a JWT is (and its trade-off)

A JWT is `header.payload.signature`. Any service can verify the signature with the IdP's public key and trust the claims — **stateless**, no DB lookup. The trade-off: a JWT is

valid until it **expires**, so revocation is hard. You use **short lifetimes + refresh tokens** to limit exposure.

> 🎪 **ANALOGY — A FESTIVAL WRISTBAND**
>
> Verified once at the entrance (login), then just flashed at each stage (services verify the signature) — no re-checking your ID everywhere.

> ⚠️ **A JWT IS SIGNED, NOT ENCRYPTED**
>
> Its payload is readable by anyone who has the token (base64, not secret). Never put passwords or secrets in it — only identity + claims.

> 💡 **STATELESS VERIFICATION VS REVOCATION**
>
> No DB hit to verify = fast & scalable, but you can't instantly kill a token. Keep lifetimes short (minutes) and rotate via refresh tokens.

> 📝 **CHEAT-SHEET**
>
> - authN = who you are; authZ = what you may do.
> - OIDC = identity on top of OAuth2; JWT carries both.
> - JWT = header.payload.signature; verify signature locally.
> - Stateless (no DB lookup) — but revocation is hard.
> - Short lifetimes + refresh tokens; never put secrets in a JWT.

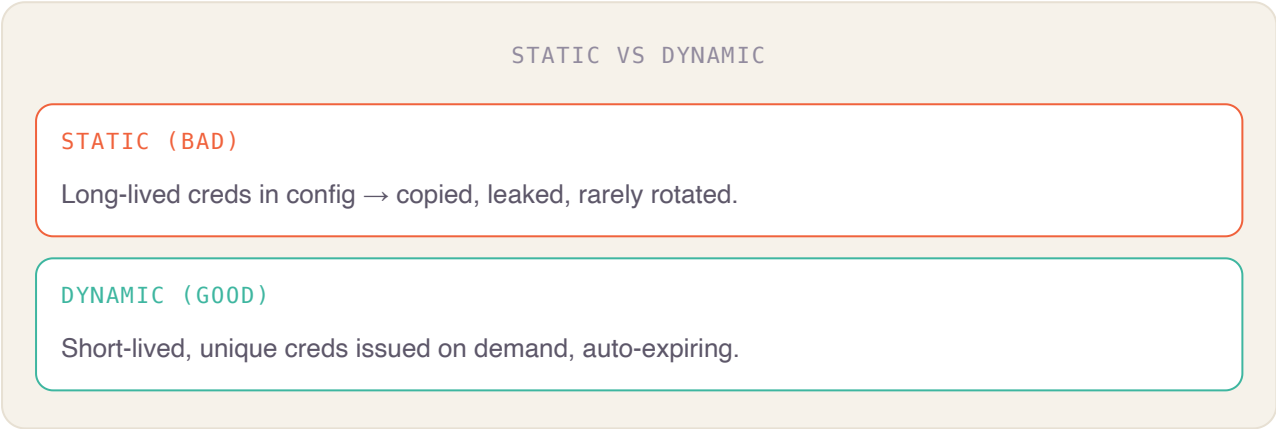
> ✅ **SELF-CHECK**
>
> 1. What's the difference between authentication and authorization?
> 2. Why can any service trust a JWT without calling the IdP?
> 3. Why must you keep JWT lifetimes short?

6·4 Vault — Dynamic Secrets & PKI

short-lived, audited secrets

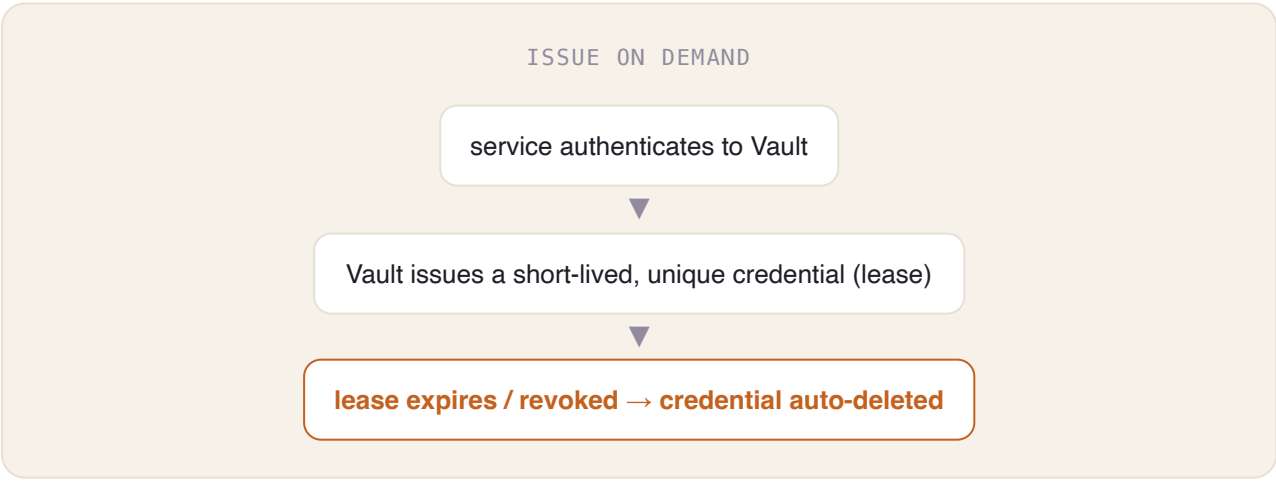
1 Static secrets in config are the problem

Long-lived passwords and keys sitting in env files or config get copied, leaked, and never rotated. If one leaks, you may not even know. The fix: don't hand out standing secrets at all.



2 Dynamic secrets on demand

Vault issues **short-lived, unique** credentials when a service asks — e.g. a DB username/password valid for an hour, tied to a **lease**. When the lease ends (or you revoke it), Vault deletes the credential at the source. Every issuance is audited.



3 PKI — certificates as a service

Vault's **PKI** engine issues and rotates TLS certificates on demand — the machinery behind mTLS (7·4) between your services. Instead of manually minting and distributing certs, services request short-lived ones that auto-renew.

🔑 ANALOGY — A VENDING MACHINE OF SINGLE-USE KEYS

Instead of copying the master key for everyone, a machine hands out short-lived, single-purpose keys that stop working on their own.

💡 LEASES + AUDIT CHANGE THE GAME

Every secret has an owner, an expiry, and an audit trail. A leaked credential is useless in an hour, and you can see exactly who accessed what.

🔗 POWERS MTLS (7·4) & ENCRYPTION KEYS (6·6)

Vault PKI issues the certs for the service mesh; its transit/KMS features manage encryption keys.

📝 CHEAT-SHEET

- Static secrets in config = copied, leaked, un-rotated.
- Vault issues short-lived, unique creds on demand (leases).
- Auto-expire / revoke deletes the credential at the source.
- Every access is audited.
- PKI engine issues & rotates TLS certs (for mTLS).

✅ SELF-CHECK

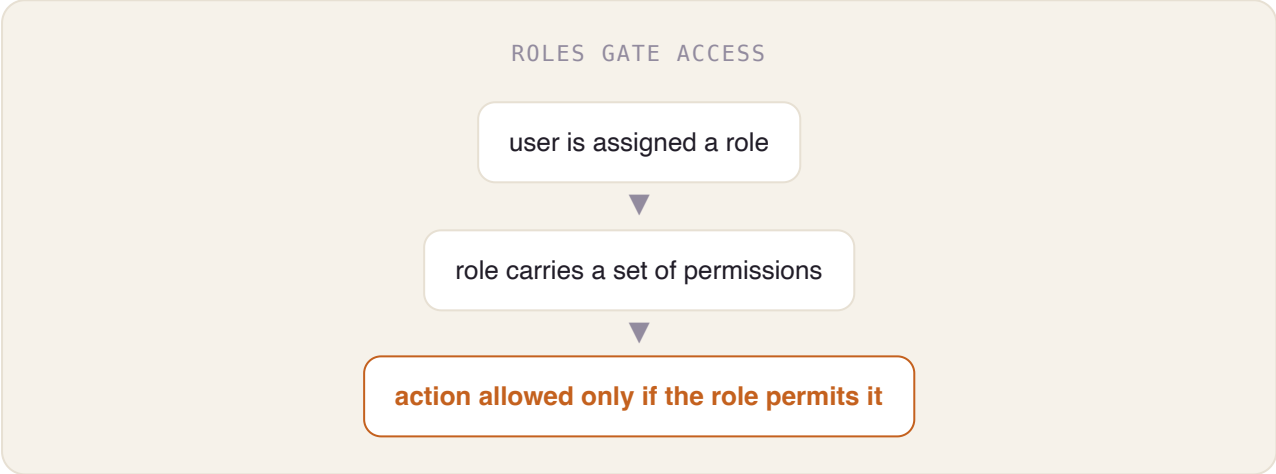
1. Why are dynamic secrets safer than static passwords?
2. What is a lease, and what happens when it ends?
3. What does Vault's PKI engine provide?

6·5 RBAC

permissions via roles

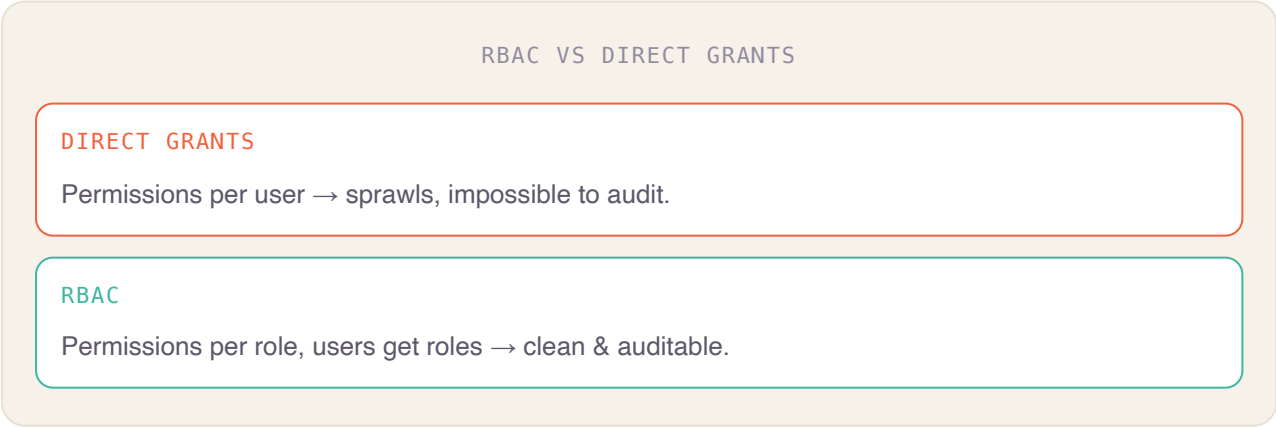
1 Grant permissions through roles, not to people

Role-Based Access Control assigns permissions to **roles**, and roles to users — instead of granting permissions to individuals directly. Change what a role can do once, and everyone in it updates.



2 Why roles scale

Directly granting permissions to hundreds of users is unmanageable and unauditable. Roles (admin, editor, viewer) collapse that into a handful of well-understood buckets you can reason about and audit at a glance.



3 Least privilege (and when you need ABAC)

Roles should follow **least privilege** — the minimum needed. When access depends on *attributes* beyond role (e.g. “only your own tenant's data,” “only during business hours”), you layer **ABAC** (attribute-based) on top — which is exactly your tenant-scoping from 6·1.

🪪 ANALOGY — JOB TITLES & DOORS

Your job title decides which doors your badge opens. Promote someone to “editor” and they get the editor doors — no per-door paperwork.

💡 YOU ALREADY OPERATE THIS

Kubernetes RBAC and AWS IAM roles are the same model. Here you're applying it to your *application's* users and tenants.

🔗 COMBINES WITH TENANT SCOPING (6·1)

RBAC says “editors can edit jobs”; ABAC/tenant-scoping adds “…but only their own tenant's jobs.”

📝 CHEAT-SHEET

- Assign permissions to roles; assign roles to users.
- Roles scale & stay auditable vs per-user grants.
- Follow least privilege by default.
- Layer ABAC for attribute-based rules (tenant, time).
- Same model as K8s/AWS RBAC — applied to your app.

✅ SELF-CHECK

1. What do you assign users, instead of individual permissions?
2. Why does RBAC scale better than direct grants?
3. When do you need ABAC on top of RBAC?

6·6 Encryption in Transit & at Rest

protect data moving and stored

1 Two threats, two defenses

Data faces two risks: being **intercepted** as it travels the network, and being read from a **stolen disk or backup**. You defend both: encrypt **in transit** (TLS) and **at rest** (KMS-managed keys).

TWO PLACES TO ENCRYPT

IN TRANSIT

TLS between clients & services; mTLS internally (7·4).

AT REST

KMS-managed keys on disk, backups, and object storage.

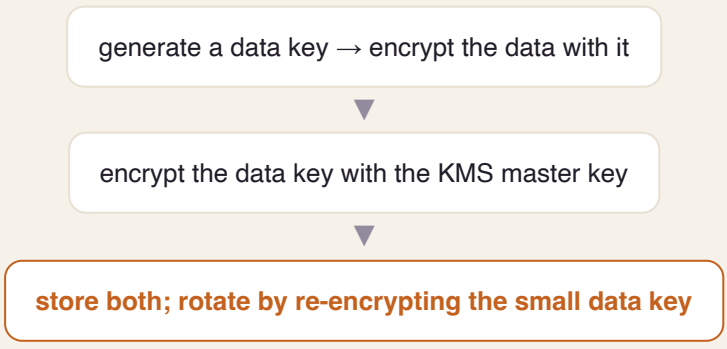
2 In transit: TLS everywhere

Every hop uses TLS so no one on the network can read or tamper with the data. Internally, the service mesh adds **mTLS** (both ends authenticated) for zero-trust — traffic is encrypted even inside your own network.

3 At rest: envelope encryption

Encrypting big data directly with a master key is slow and risky. **Envelope encryption** generates a per-object **data key** to encrypt the data, then encrypts that data key with a **master key** in KMS. To rotate, you re-encrypt the small data keys, not the terabytes.

ENVELOPE ENCRYPTION



🚚 ANALOGY — ARMORED TRUCK + A SAFE

An armored truck for money on the move (in transit) and a locked safe for money stored in the vault (at rest). You need both — a strong safe doesn't help while the cash is being driven across town.

💡 WHY ENVELOPE ENCRYPTION

Rotating or revoking access means re-wrapping tiny data keys, not re-encrypting all your data. It also lets KMS never touch the bulk data itself.

⚠️ KEY MANAGEMENT IS THE HARD PART

Encryption is easy; safely storing, rotating, and controlling access to keys is the real work — which is why you use a KMS/Vault, not homegrown key handling.

📝 CHEAT-SHEET

- Two threats: interception (transit) & stolen disk (rest).
- In transit: TLS everywhere; mTLS internally.
- At rest: KMS-managed keys on disk/backups/object store.
- Envelope encryption: data key encrypts data; master key encrypts data key.
- The hard part is key management — use KMS/Vault.

✅ SELF-CHECK

1. What two threats do in-transit vs at-rest encryption each address?
2. How does envelope encryption make key rotation cheap?
3. What's the genuinely hard part of encryption?

◆ STAGE 7 — INTERNAL COMMS & GO ◆

7·1 gRPC & Protobuf

fast, typed service-to-service calls

1 REST/JSON is costly for internal, high-volume calls

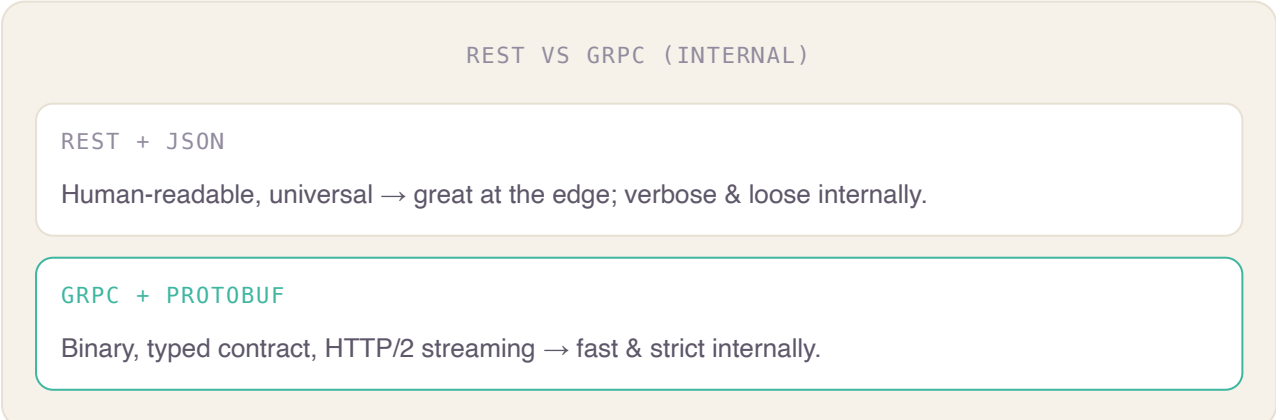
REST + JSON is perfect at the public edge — human-readable, universal. But *between* your services, called millions of times, JSON is verbose to send and slow to parse, and there's no enforced contract, so a field rename silently breaks the caller.

2 Contract-first with Protobuf

You define the service and its messages in a `.proto` file — the single source of truth. A code generator produces a typed **client and server** in each language, so both sides are always in sync and mismatches are caught at compile time.

3 Why gRPC is fast

gRPC sends compact **binary** over **HTTP/2** — smaller payloads, multiplexed connections, and built-in **streaming** (client/server/bidirectional). For internal service calls that adds up to real latency and CPU savings versus REST/JSON.



📋 **ANALOGY — A STRICT PRE-AGREED FORM**

Two offices exchanging a fixed, pre-agreed form: no ambiguity, quick to fill and process — unlike mailing free-form polite letters back and forth.

💡 **CODE-GEN KEEPS BOTH SIDES HONEST**

Because client and server are generated from the same .proto, a breaking change won't even compile — the contract is enforced, not hoped for.

⚠️ **NOT HUMAN-READABLE**

You can't just curl a gRPC endpoint and read it. You need tooling (grpcurl, reflection) to debug — the price of binary speed.

📝 **CHEAT-SHEET**

- REST/JSON at the edge; gRPC/Protobuf internally.
- Contract-first: .proto → generated typed client + server.
- Binary over HTTP/2 → compact, multiplexed, streaming.
- Breaking changes caught at compile time.
- Trade-off: not human-readable; needs tooling.

✅ **SELF-CHECK**

1. Why is JSON a poor fit for high-volume internal calls?
2. What is the single source of truth in gRPC, and what does it generate?
3. Name one downside of gRPC vs REST.

7·2 Go Concurrency

cheap parallelism for the worker

1 Why rewrite the worker in Go

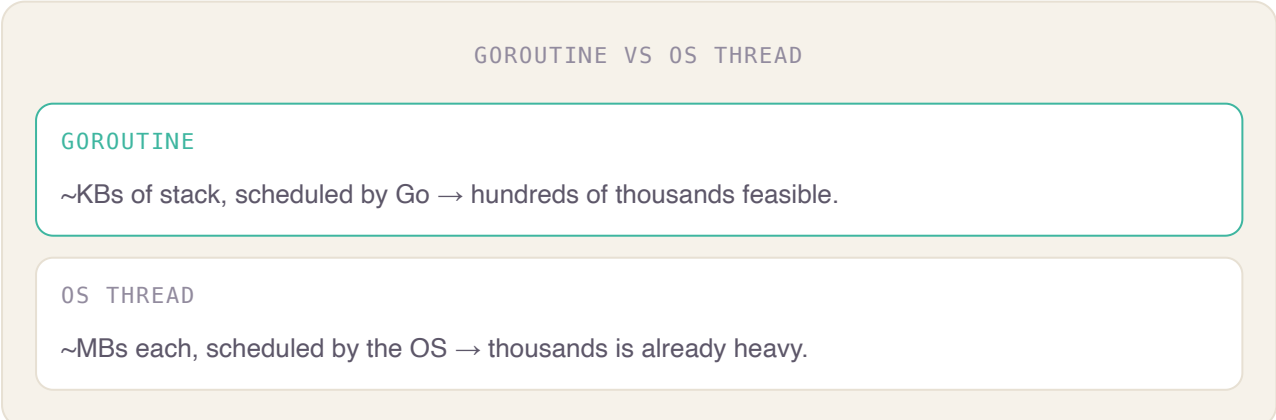
The worker fleet runs thousands of concurrent agent jobs — mostly waiting on I/O. Go makes that kind of concurrency **cheap and simple**: tiny memory per task, fast startup, and small static binaries that are ideal in containers.

2 Goroutines & channels

A **goroutine** is a function running concurrently — so lightweight you can have hundreds of thousands. **Channels** pass data between goroutines safely, so you coordinate by *communicating* rather than by sharing memory and locking.

3 context — cancellation & deadlines

`context` carries a **cancel signal and deadline** down through calls. Pass it into every operation; check it at boundaries; when it's cancelled, everything downstream stops cleanly. This is the cooperative cancellation from S1·6, made idiomatic.



👨‍🍳 ANALOGY — A CHEF JUGGLING POTS

One chef effortlessly tends many simmering pots with cheap helpers — adding another pot costs almost nothing, because the work is mostly waiting for things to cook.

💡 "SHARE MEMORY BY COMMUNICATING"

Go's motto: instead of many goroutines mutating shared state behind locks, pass ownership of data through channels. Fewer races, clearer code.

⚠️ GOROUTINE LEAKS

A goroutine blocked forever on a channel never exits and leaks memory. Always give goroutines a way out — a closed channel or a cancelled context.

📝 CHEAT-SHEET

- Go: cheap concurrency, fast, tiny container images.
- Goroutines = ultra-light concurrent functions.
- Channels pass data safely (communicate, don't lock).
- context carries cancel + deadline (like S1·6).
- Beware goroutine leaks — always provide an exit.

✅ SELF-CHECK

1. Why is Go a good fit for a high-concurrency worker fleet?
2. What do channels let you avoid?
3. What is context used for?

7·3 Service Mesh & Sidecar

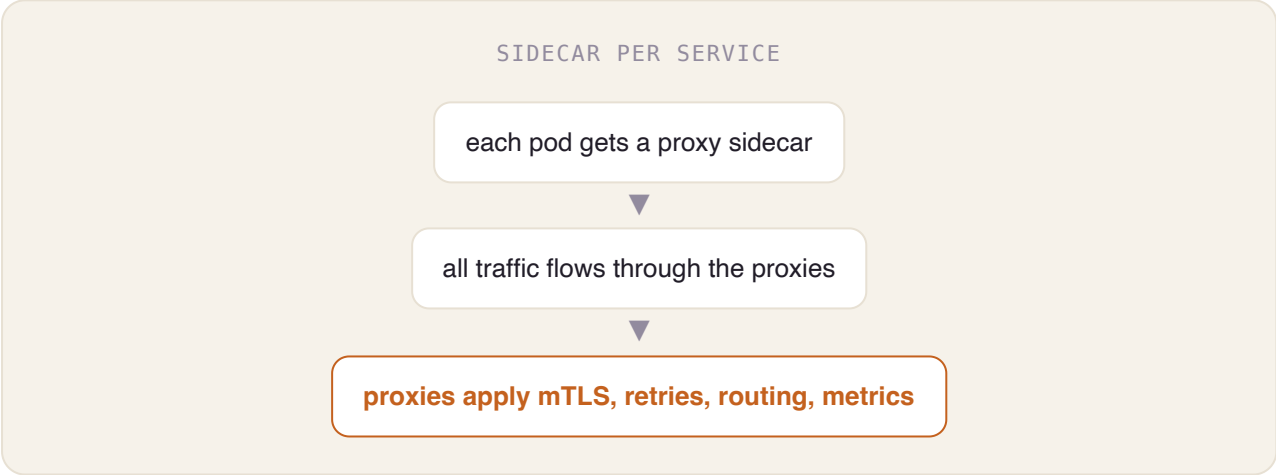
security & traffic control without app code

1 Cross-cutting network concerns keep repeating

Every service needs the same things between calls: encryption, retries, timeouts, traffic routing, metrics. Building all that into every service (in every language) is duplicated, inconsistent effort. A **service mesh** lifts it out of the app.

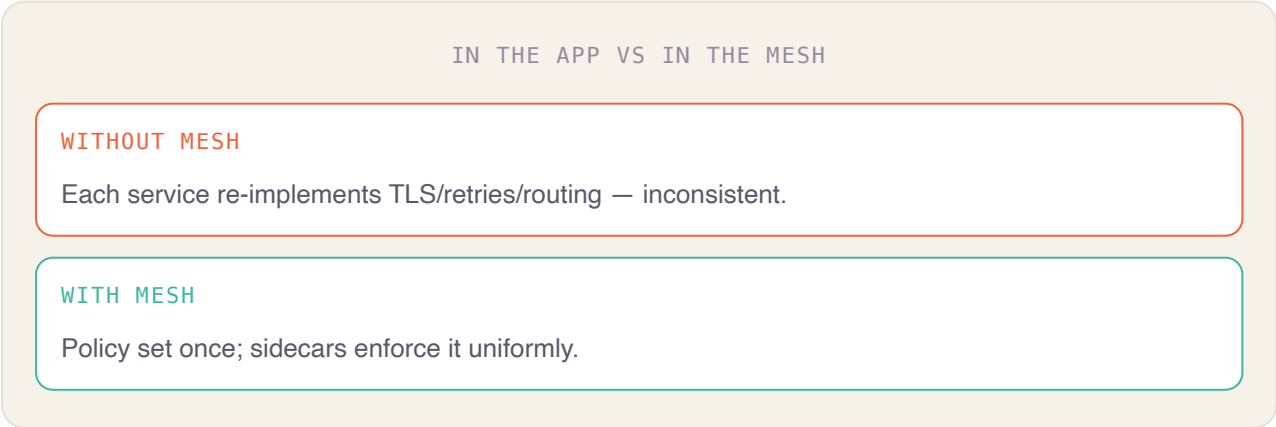
2 The sidecar proxy

The mesh runs a **proxy sidecar** next to every service pod. All traffic in/out flows through the proxy, which handles mTLS, retries, timeouts, and routing — **transparently**, so the application code does none of it.



3 Control plane vs data plane

The **data plane** is the fleet of sidecars carrying traffic; the **control plane** (Istio) is where *you* set policy — “retry these 3×,” “canary 5% to v2,” “mTLS everywhere.” You configure intent centrally; the sidecars enforce it.



🕴️ **ANALOGY — A TRANSLATOR/BODYGUARD PER EMPLOYEE**

Each employee gets a personal translator/bodyguard (sidecar) who handles secure comms and routing, so the employee just focuses on their actual job.

💡 **POLICY, NOT CODE**

Traffic rules become *configuration* you change centrally — no redeploying services to add a retry or shift a canary.

⚠️ **REAL COST: LATENCY & COMPLEXITY**

Every hop now passes through two proxies, and the mesh is another complex system to run. Adopt it when you have enough services to justify it.

📝 **CHEAT-SHEET**

- Mesh moves cross-cutting network concerns out of app code.
- Sidecar proxy beside each service handles traffic.
- Control plane = policy you set; data plane = sidecars enforcing it.
- mTLS, retries, timeouts, routing, metrics — as config.
- Cost: extra latency + operational complexity.

✅ **SELF-CHECK**

1. What repeated concerns does a mesh remove from app code?
2. What runs beside each service, and what does it do?
3. What's the difference between the control plane and data plane?

7·4 mTLS

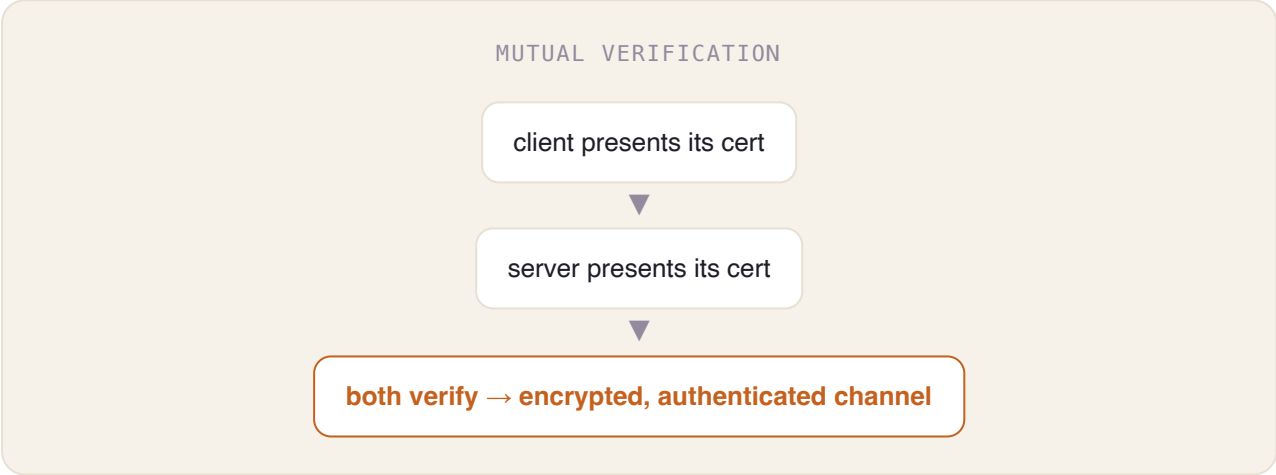
both ends prove identity

1 Zero-trust: the network isn't trusted

Being “inside the network” is not proof of identity — an attacker who breaches one pod could otherwise call everything. **Zero-trust** says every connection must be authenticated. **mTLS** (mutual TLS) makes both client *and* server prove who they are.

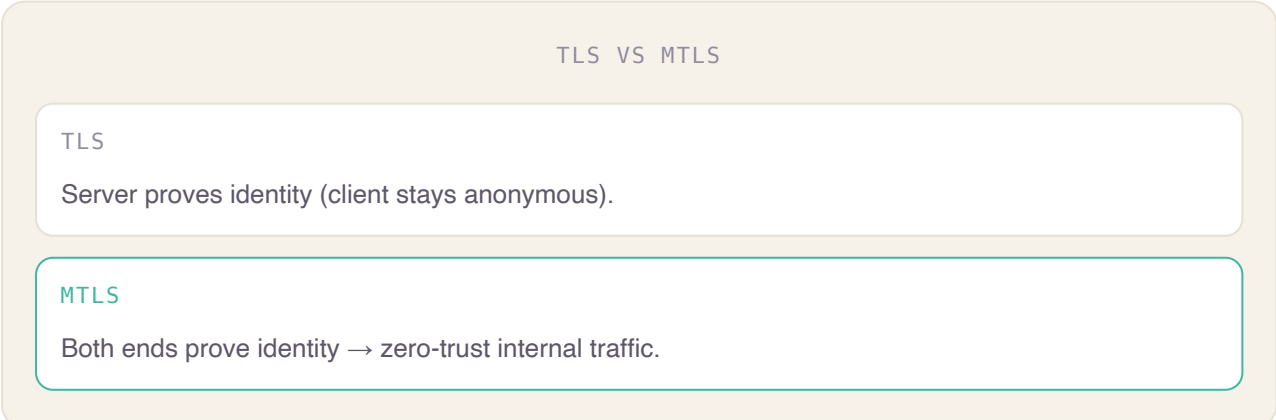
2 Two-way certificate verification

Regular TLS authenticates only the server (like a website). **mTLS** adds the reverse: the client also presents a certificate, so each side verifies the other before any data flows. The channel is both encrypted and mutually authenticated.



3 The mesh automates the certs

Managing certs by hand is painful; the mesh (with Vault PKI from 6·4) **issues, distributes, and rotates** short-lived certs automatically. Identity is the **certificate** — not the IP address — so a service is trusted for *who it is*, not where it sits.



🪪 ANALOGY — BOTH SHOW ID

Not one person flashing a badge while the other is assumed fine — **both** show ID to each other before talking. No one is trusted just for being in the building.

💡 IDENTITY = CERT, NOT IP

In a world of ephemeral pods and changing IPs, the certificate is the stable identity. Authorize on identity, not network location.

🔗 POWERED BY VAULT PKI (6·4)

Those short-lived, auto-rotated certs come from the PKI engine you set up in Stage 6.

📝 CHEAT-SHEET

- Zero-trust: network location ≠ identity.
- mTLS = both client & server present certs.
- Encrypted AND mutually authenticated channel.
- Mesh + Vault PKI auto-issue & rotate short-lived certs.
- Authorize on certificate identity, not IP.

✅ SELF-CHECK

1. How does mTLS differ from regular TLS?
2. What does “zero-trust” mean for internal traffic?
3. Where do the certificates come from, and why short-lived?

7·5 Traffic Shaping & Canary

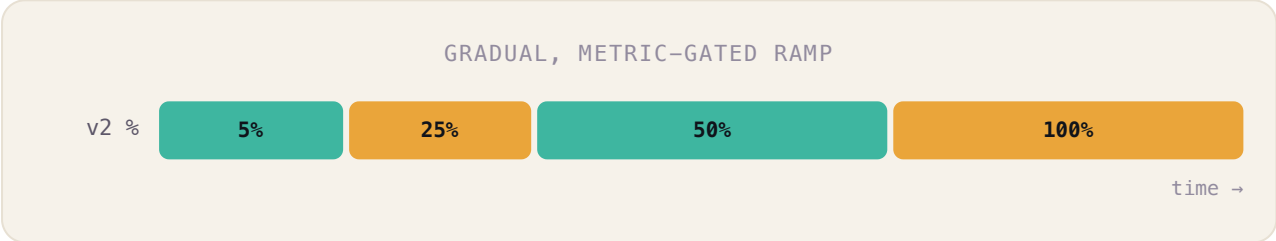
safe, gradual releases

1 Big-bang deploys are a gamble

Flipping 100% of traffic to a new version at once means a bad release hits every user instantly. A **canary** release de-risks this: send a *small* slice of traffic to the new version first and watch how it behaves.

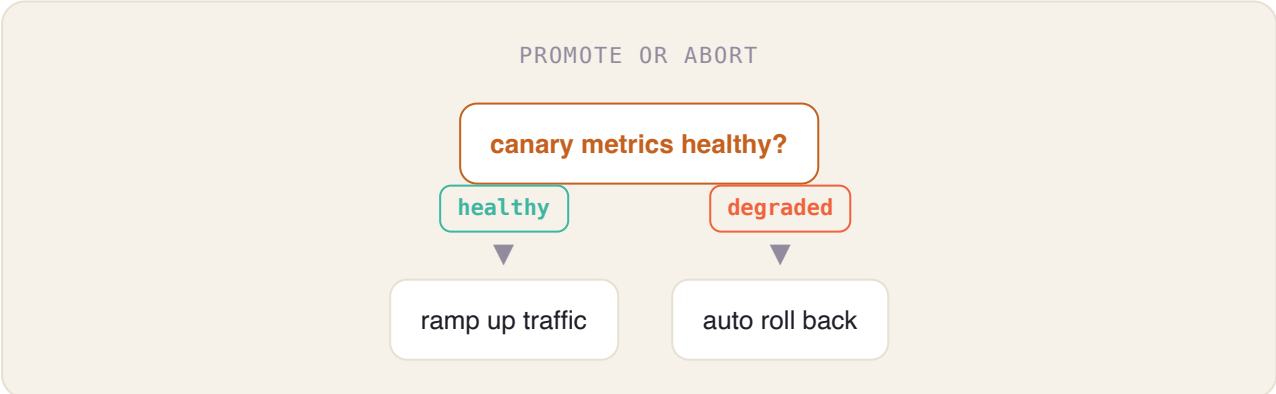
2 Route a percentage, then decide on data

The mesh (or Argo Rollouts) routes, say, **5%** to v2 while 95% stays on v1. You watch v2's **error rate and latency**; if healthy, you **ramp up** (5→25→50→100); if not, you **roll back** instantly — most users never saw v2.



3 The mesh also shapes traffic generally

Beyond canaries, the mesh applies **retries, timeouts, and mirroring** at the network layer — e.g. mirror real traffic to a new version without affecting users, or fail over on timeout. Traffic shaping is release safety plus runtime resilience.



ANALOGY — TESTING A BRIDGE WITH 1% OF CARS

Before opening all lanes, you let a few cars across and watch it hold. If it does, you allow more; if it groans, you close it — with almost no one at risk.

💡 METRIC-DRIVEN PROMOTION

Good rollouts promote based on *metrics*, not a stopwatch — the SLOs from Stage 12 become the gate that decides ramp-up vs rollback.

🔗 ARGO ROLLOUTS (STAGE 8/9)

The GitOps tooling automates this canary + analysis + auto-rollback as part of the deploy pipeline.

📝 CHEAT-SHEET

- Canary = route a small % to the new version first.
- Watch error rate + latency on the canary slice.
- Healthy → ramp up; degraded → instant rollback.
- Mesh also does retries, timeouts, traffic mirroring.
- Promote on metrics (SLOs), not on time.

✅ SELF-CHECK

1. Why is a big-bang deploy risky?
2. What do you watch on the canary before ramping up?
3. What decides promotion vs rollback in a good rollout?

8·1 K8s Core Objects

the declarative building blocks

1 You declare desired state; K8s makes it real

Kubernetes is **declarative**: you describe the end state you want (“3 replicas of this image”), and K8s continuously works to make reality match — starting, healing, and rescheduling containers as needed. You don't run commands step-by-step; you state intent.

2 The objects you compose

Pods run containers (the smallest unit). **Deployments** manage stateless replicas (roll updates, self-heal). **StatefulSets** give stable identity + storage for stateful things (databases, Kafka). **Services** give a stable network address in front of changing pods.

WHAT RUNS WHAT

POD / DEPLOYMENT

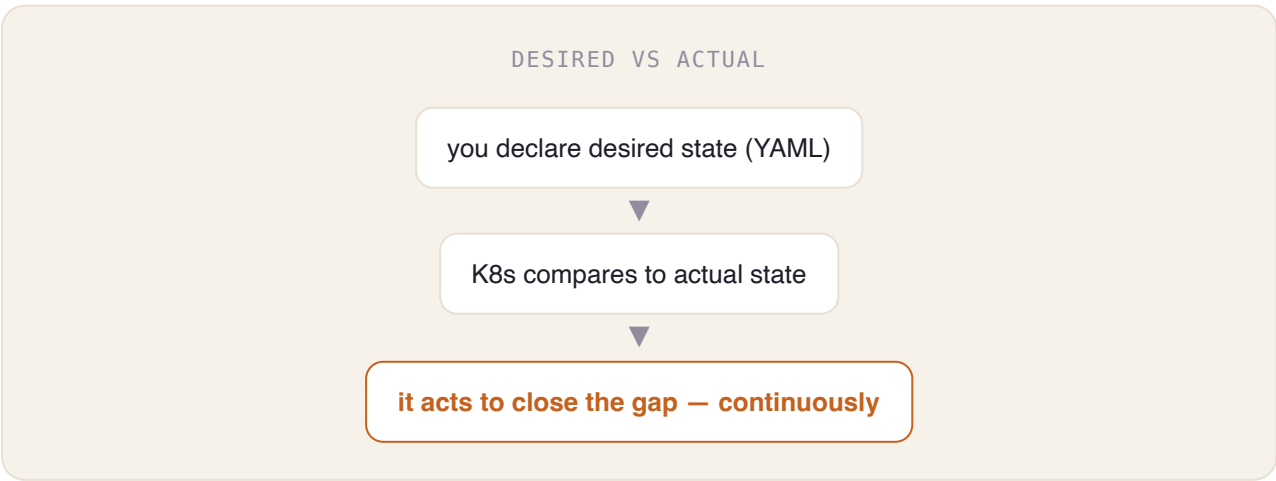
Pod = smallest unit; Deployment = self-healing stateless replicas.

STATEFULSET / SERVICE

StatefulSet = stable identity for DBs/queues; Service = stable address.

3 Self-healing & scaling for free

Because it's a control loop, K8s restarts crashed pods, reschedules them off dead nodes, and scales replicas up/down — all to keep actual state matching desired state. This continuous reconciliation is the same idea you'll *extend* with your own Operator (8·3).



🏗️ ANALOGY — AN AUTOMATED SHIPPING PORT

You post the manifest (“these containers, here”), and robotic cranes load, move, and replace containers to match it — you never drive a crane yourself.

💡 DECLARATIVE BEATS IMPERATIVE

You don't script “start container, then…”; you declare the goal and let the control loop get there and keep it there — including after failures.

🔗 THE OPERATOR EXTENDS THIS LOOP

Your custom Operator (8·3) is just another controller reconciling desired vs actual — for *your own* object types.

📝 CHEAT-SHEET

- K8s is declarative: you state desired end-state.
- Pod = smallest unit; Deployment = stateless replicas.
- StatefulSet = stable identity (DBs); Service = stable address.
- A control loop self-heals & scales to match desired state.
- This same loop is what an Operator extends.

✅ SELF-CHECK

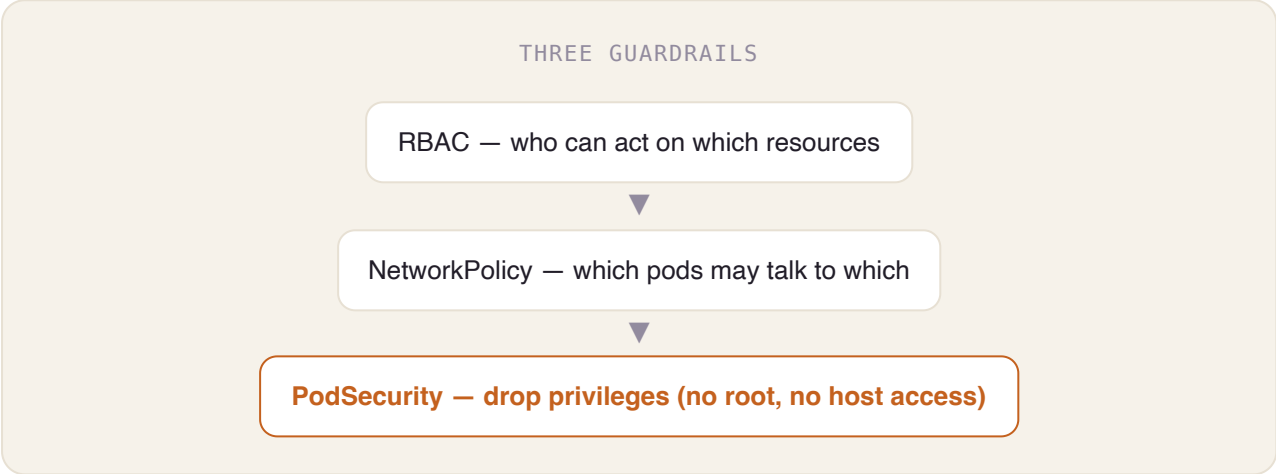
1. What does “declarative” mean in Kubernetes?
2. Which object runs a stateless app with replicas, and which for a database?
3. What keeps actual state matching desired state?

8·2 RBAC / NetworkPolicies / PodSecurity

lock the cluster down

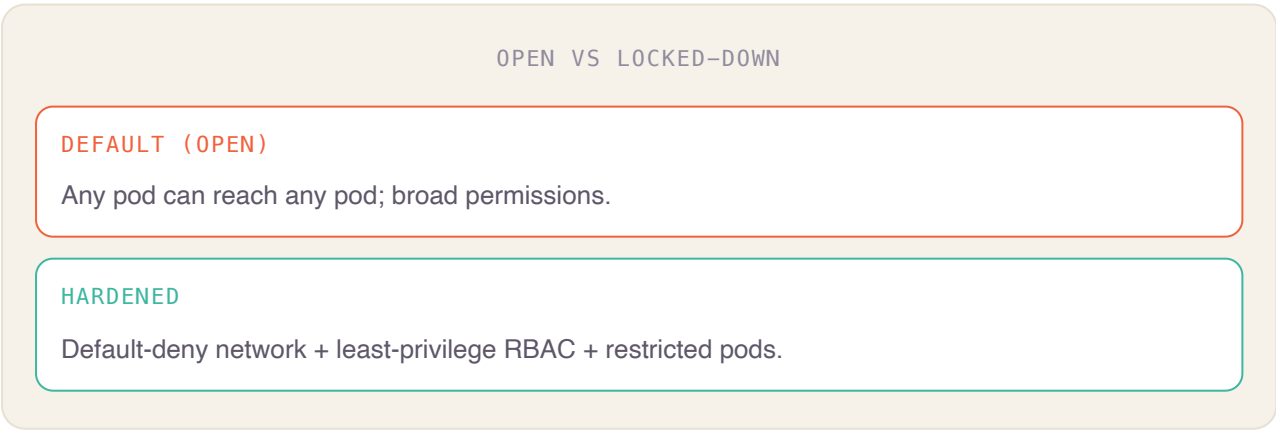
1 Least privilege & blast radius

A breached pod or a fat-fingered permission shouldn't be able to touch everything. You minimize the **blast radius** with three controls: who can act (RBAC), which pods can talk (NetworkPolicy), and how safely pods run (PodSecurity).



2 Default-deny is the posture

By default, Kubernetes pods can talk to *any* other pod — a flat, open network. You flip that to **default-deny** with NetworkPolicies and only allow the specific flows you need (gateway→worker, worker→DB). RBAC likewise grants the minimum verbs on the minimum resources.



3 PodSecurity: restrict what a pod can do

PodSecurity standards stop pods from running as **root**, mounting the host filesystem, or escalating privileges. Even if an attacker lands inside a pod, these limits keep them boxed in — they can't break out to the node.

🛂 ANALOGY — BADGES, INTERNAL FIREWALLS, SAFETY RULES

Badge access (RBAC), firewalls between departments (NetworkPolicy), and safety rules every worker follows (PodSecurity). Together they contain any single breach.

⚠️ THE DEFAULT NETWORK IS WIDE OPEN

Until you add NetworkPolicies, one compromised pod can scan and reach everything. Default-deny, then allow explicitly.

💡 DEFENSE IN DEPTH

No single control is enough — RBAC + NetworkPolicy + PodSecurity layer so that getting past one doesn't grant the keys to the cluster.

📝 CHEAT-SHEET

- Minimize blast radius: least privilege everywhere.
- RBAC = who can act on what.
- NetworkPolicy = allowed pod-to-pod traffic (default-deny!).
- PodSecurity = no root, no host access, no priv-escalation.
- Layer all three — defense in depth.

✅ SELF-CHECK

1. What does a NetworkPolicy control, and what's the safe default?
2. What does PodSecurity restrict, and why does it matter after a breach?
3. Why layer all three controls?

8·3 CRD + Controller + Reconcile Loop

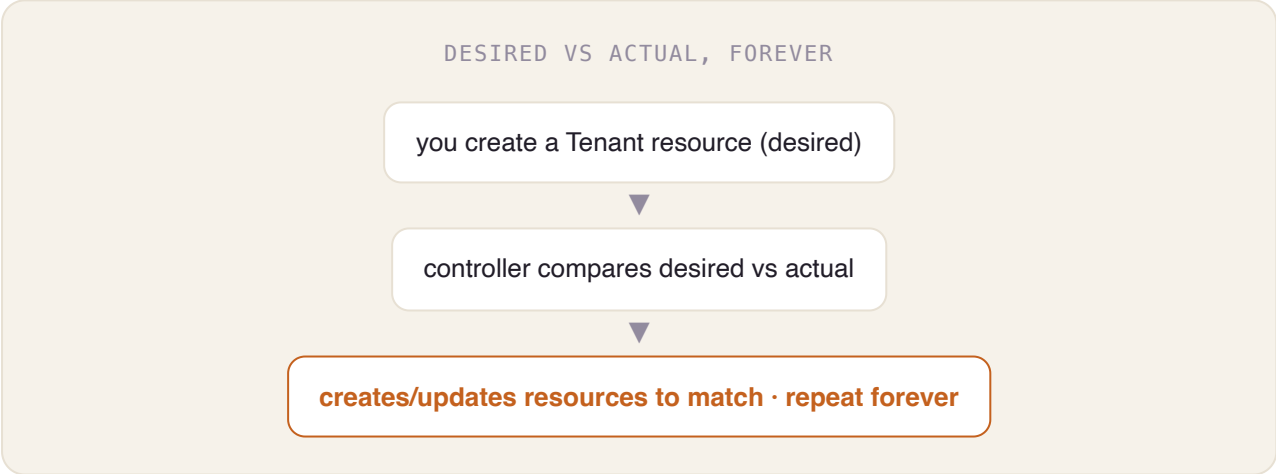
extend K8s with your own automation

1 Teach Kubernetes a new noun

A **Custom Resource Definition (CRD)** adds your own object type to the cluster — e.g. a `Tenant` or `PreviewEnv`. Now `kubectl get tenants` works, and your tenant becomes a first-class, declarative Kubernetes object like Pods and Services.

2 The controller runs a reconcile loop

A **controller** watches your custom resources and runs a **reconcile loop**: read desired state (the resource), read actual state (what exists), and **act to close the gap** — create the tenant's namespace, DB, secrets, quotas. It runs continuously, forever.



3 Level-triggered = self-healing

Crucially the loop is **level-triggered** (act on the current *state*), not edge-triggered (act on events). So if someone deletes a resource the Operator manages, the next reconcile simply **recreates** it — the system self-heals to the declared state. That's the magic of the pattern.

IMPERATIVE SCRIPT VS RECONCILE LOOP

ONE-SHOT SCRIPT

Runs once; drift & failures aren't fixed.

RECONCILE LOOP

Continuously converges to desired state; self-heals.

🌡️ ANALOGY — A THERMOSTAT

You set “desired 22°C.” The thermostat constantly compares to the room and nudges the heater — forever, without you. Open a window and it just works harder to converge again.

💡 LEVEL-TRIGGERED, NOT EVENT-TRIGGERED

Reconcile on the current state, not on “an event happened.” Missed events don't matter — the next loop still converges. This is why Operators are robust.

🔗 POWERS PREVIEW ENVIRONMENTS (STAGE 10)

Your Operator is what provisions and tears down per-tenant preview stacks — and “I wrote a Kubernetes Operator in Go” is your ⭐ interview differentiator.

📝 CHEAT-SHEET

- CRD = your own Kubernetes object kind.
- Controller reconciles desired vs actual, continuously.
- Level-triggered → self-heals (recreates deleted things).
- Encodes your ops knowledge as native automation.
- Top-tier skill to show (your ⭐ anchor #2).

✅ SELF-CHECK

1. What does a CRD add to Kubernetes?
2. What does the reconcile loop do on every pass?
3. Why does level-triggered reconciliation make an Operator self-heal?

8·4 KEDA — Event-Driven Autoscaling

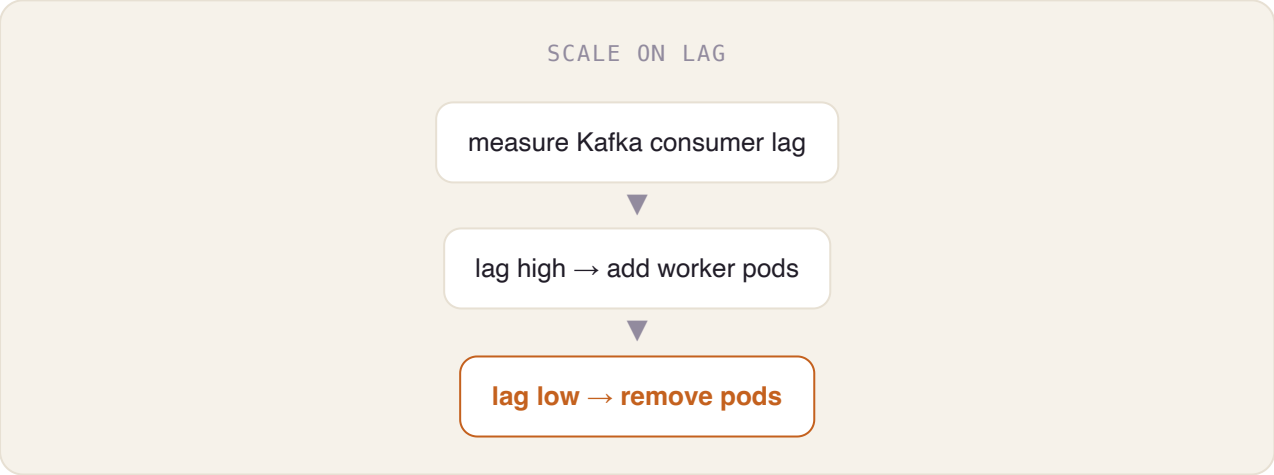
scale on real backlog

1 CPU scaling misses the real signal

The default autoscaler (HPA) scales on CPU/memory. But your workers are mostly *waiting* on LLM calls — low CPU even when a huge backlog of jobs is piling up in Kafka. Scaling on CPU would leave you under-provisioned exactly when you're busiest.

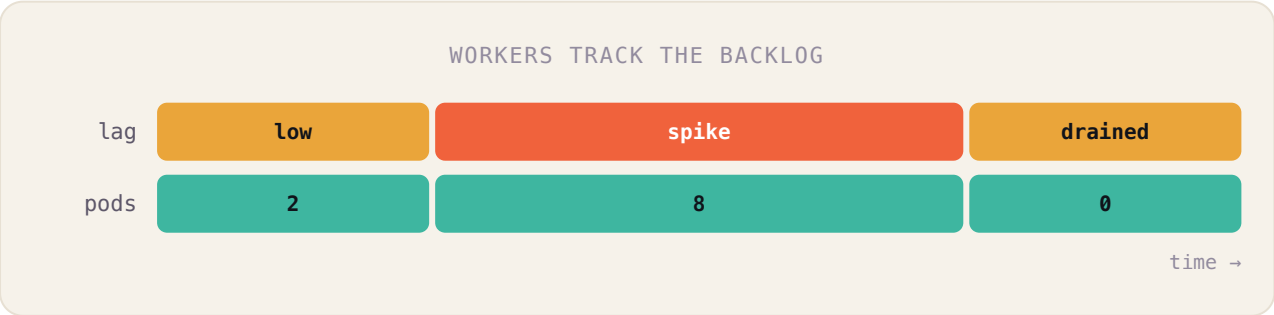
2 Scale on Kafka consumer lag

KEDA scales on *external* signals — here, **Kafka consumer lag** (how many jobs are waiting). Backlog grows → add worker pods; backlog drains → remove them. You scale on the metric that actually reflects demand.



3 Scale to zero

When there's *no* work, KEDA can scale the workers to **zero** — you pay for nothing while idle — and spin the first pod back up when a message arrives. Great for cost, with a small cold-start trade-off.



👥 ANALOGY — CALL IN STAFF WHEN THE QUEUE IS LONG

You don't hire based on how hard the current staff are sweating (CPU) — you hire based on how many orders are waiting (lag), and send people home when it's quiet.

💡 SCALE ON THE SIGNAL THAT MEANS "DEMAND"

For queue workers, backlog depth is the true demand signal — not CPU. Match your scaler to what actually predicts load.

🔗 COMPLEMENTS HPA

KEDA can drive the HPA under the hood — it just feeds it external metrics like Kafka lag instead of CPU.

📝 CHEAT-SHEET

- CPU scaling misses I/O-bound backlog.
- KEDA scales on external signals (Kafka lag).
- Backlog up → add pods; drained → remove.
- Can scale to zero when idle (cost win, cold-start cost).
- Scale on the metric that reflects real demand.

✅ SELF-CHECK

1. Why is CPU a poor scaling signal for these workers?
2. What signal does KEDA scale on here, and why is it better?
3. What does scale-to-zero trade off?

8·5 Admission Control (Kyverno / OPA)

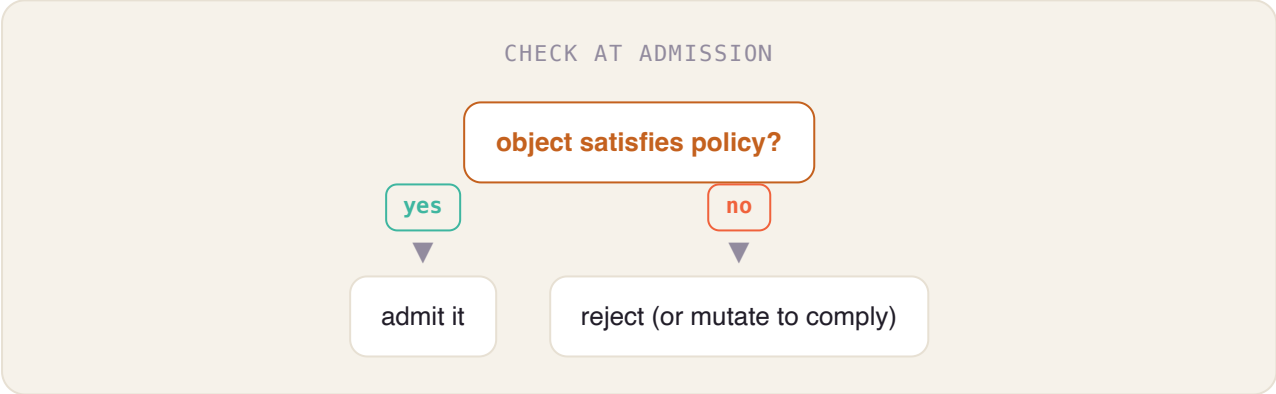
policy-as-code guardrails

1 Enforce standards at the moment of creation

You want cluster-wide rules — “no privileged pods,” “every workload must have a team label,” “images only from our registry.” **Admission control** checks every object as it's created or updated and **rejects or fixes** the ones that break the rules.

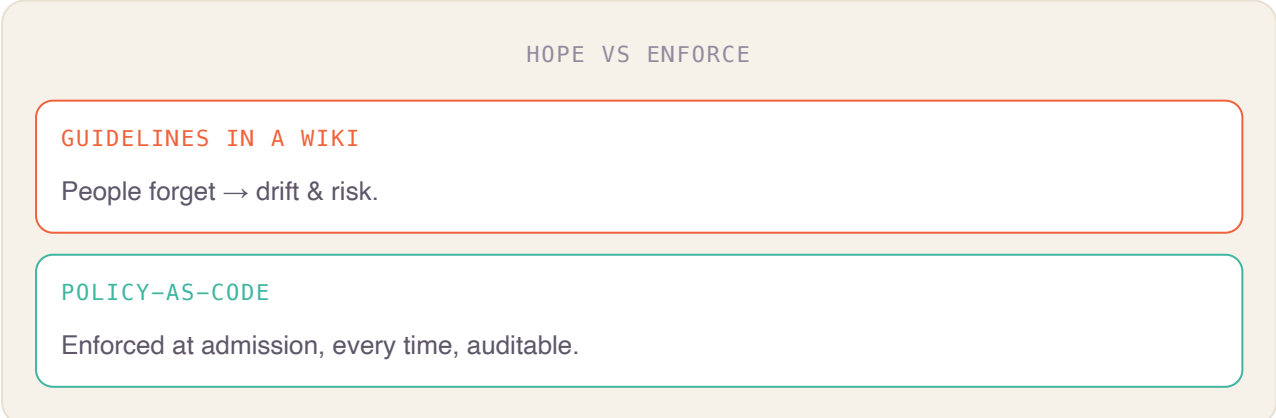
2 Validate and mutate webhooks

Two kinds: **validating** (reject anything non-compliant) and **mutating** (auto-fix it, e.g. inject a default label or a sidecar). Tools like **Kyverno** or **OPA Gatekeeper** let you write these as declarative policies.



3 Policy-as-code

Policies live in Git as code — versioned, reviewed, and applied uniformly across the cluster. Instead of hoping everyone follows the rules, the cluster **enforces** them automatically at the door, every time.



🚪 ANALOGY — A DOORMAN WITH A DRESS CODE

Anyone not meeting the code is turned away at the door (validate) — or handed the required jacket on the way in (mutate). Consistent, automatic, no exceptions.

💡 SHIFT GOVERNANCE LEFT & AUTOMATIC

Catch violations at create-time, not in a quarterly audit. The guardrail is code, so it's consistent and reviewable.

🔗 EXTENDS TO SUPPLY CHAIN (STAGE 12)

The same enforcement gate can require signed images (cosign/SLSA) before a pod is admitted.

📝 CHEAT-SHEET

- Admission control checks objects at create/update.
- Validating = reject; mutating = auto-fix.
- Kyverno / OPA Gatekeeper = policies as code.
- Enforced uniformly & automatically, not by memory.
- Can require signed images (supply-chain gate).

✅ SELF-CHECK

1. When are admission policies evaluated?
2. What's the difference between a validating and a mutating policy?
3. Why is policy-as-code better than written guidelines?

8·6 Helm & Kustomize

package & customize manifests

1 The same YAML, slightly different per environment

Dev, staging, and prod need almost-identical manifests with small differences (replica counts, image tags, resource limits). Copy-pasting YAML per environment is a drift-and-error factory. Two tools solve it differently.

TWO APPROACHES

HELM

Templated, versioned charts + a values file per environment.

KUSTOMIZE

A base set of manifests + overlays that patch it per environment.

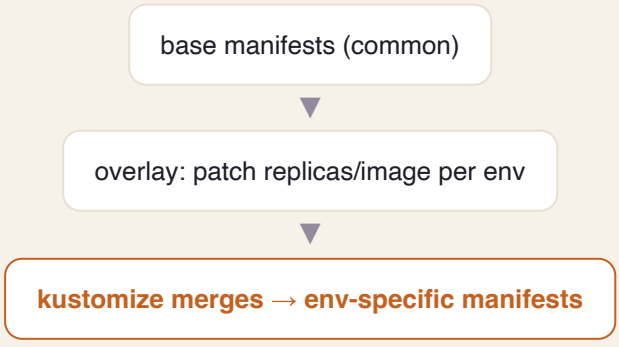
2 Helm — templated, versioned charts

Helm packages manifests as a **chart** with placeholders filled from a `values.yaml`. It's versioned and shareable (install/upgrade/rollback a whole app as a unit) — great for distributing reusable packages.

3 Kustomize — base + overlays

Kustomize takes a plain **base** and applies **overlays** that patch specific fields per environment — no templating language, just declarative patches. Great when you want to keep manifests readable and diff-friendly.

BASE + OVERLAY



📝 ANALOGY — A FILL-IN TEMPLATE VS A BASE RECIPE + TWEAKS

Helm = a fill-in-the-blanks template you supply values to. Kustomize = a base recipe plus small written tweaks per occasion. Both beat rewriting the whole dish each time.

💡 OFTEN USED TOGETHER

A common pattern: Helm to install third-party charts, Kustomize to layer your environment-specific patches on top.

⚠️ OVER-TEMPLATING HURTS

Helm charts full of nested conditionals become unreadable. Keep templates simple; push complexity into values or use Kustomize overlays.

📝 CHEAT-SHEET

- Avoid copy-pasting YAML per environment.
- Helm = templated, versioned, shareable charts (values.yaml).
- Kustomize = base + overlays (declarative patches, no templating).
- Often combined (Helm for deps, Kustomize for env patches).
- Keep templates simple; avoid over-templating.

✅ SELF-CHECK

1. What problem do Helm/Kustomize both solve?
2. How does Helm's approach differ from Kustomize's?
3. When might you use both together?

◆ STAGE 9 — FRONTEND & EXPERIMENTATION ◆

9·1 React / Next.js / TypeScript

the dashboard stack

1 Components: build UIs from reusable pieces

React builds a UI out of **components** — small, self-contained pieces (a JobCard, a StatusBadge) that manage their own state and re-render when data changes. You compose complex screens from simple, reusable parts.

2 Next.js: the framework around React

Next.js adds what a real app needs: routing, server-side rendering (fast first paint, SEO), API routes, and an optimized build. It turns React from a library into a full application framework.

THE THREE PIECES

REACT

Reusable components that re-render on state change.

NEXT.JS

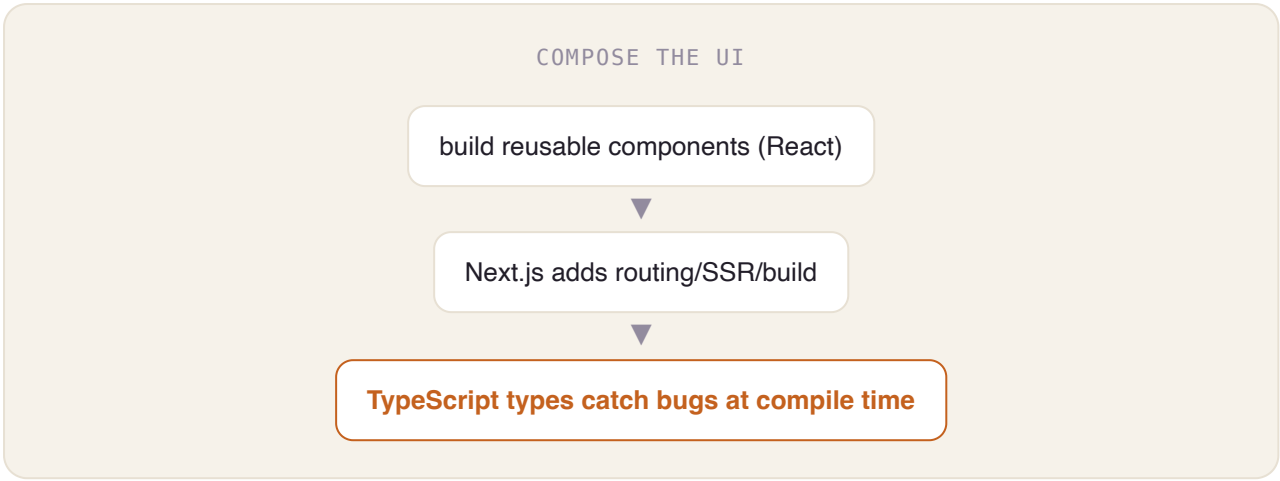
Routing, SSR, build tooling around React.

TYPESCRIPT

Static types across the whole app.

3 TypeScript: catch bugs before users do

TypeScript adds static types to JavaScript — the same discipline as Python's type hints (L2). It catches “undefined is not a function” style bugs at compile time and makes refactoring across a large dashboard safe.



🧱 ANALOGY — LEGO FOR UIS

React gives you labeled bricks (components); Next.js is the builder kit with instructions; TypeScript makes the bricks only fit where they should.

💡 SAME TYPING DISCIPLINE AS THE BACKEND

TypeScript on the frontend mirrors mypy/Pydantic on the backend — types are your safety net on both sides of the wire.

⚠️ FRONTEND CHURN IS REAL

The JS ecosystem moves fast. Pin versions, keep dependencies lean, and don't chase every new tool mid-project.

📝 CHEAT-SHEET

- React = reusable components, re-render on state change.
- Next.js = routing + SSR + build tooling.
- TypeScript = static types → fewer runtime bugs.
- Same typing discipline as the backend.
- Keep dependencies lean; pin versions.

✅ SELF-CHECK

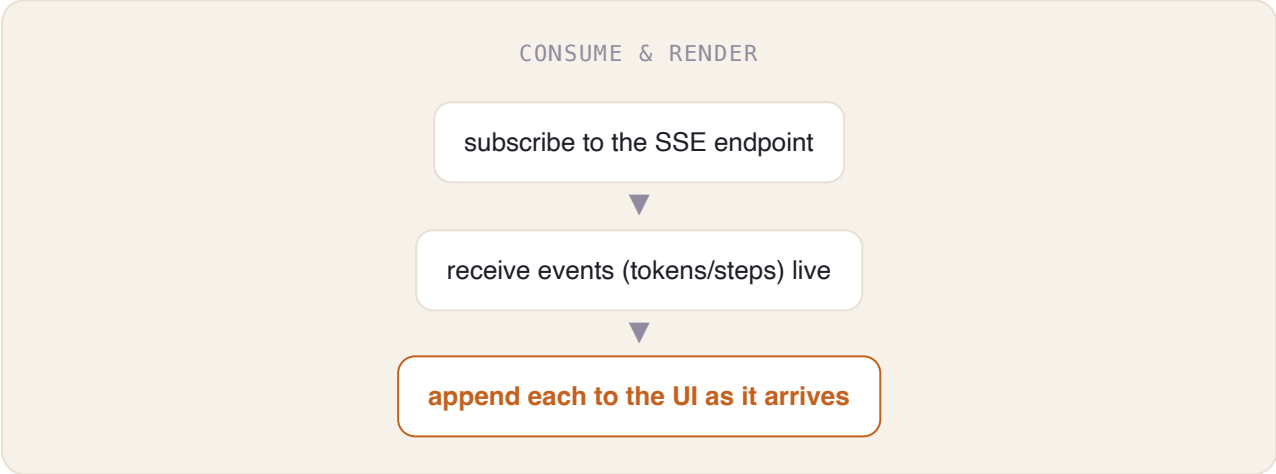
1. What is a React component?
2. What does Next.js add on top of React?
3. How does TypeScript relate to the backend's mypy/Pydantic?

9·2 SSE UI (live trajectory)

render the agent's stream

1 Consume the server's SSE stream

The dashboard subscribes to the SSE endpoint you built in Stage 3 (3·6) and receives the agent's steps as a live event stream — each token or step arriving as its own message over one long-lived HTTP connection.

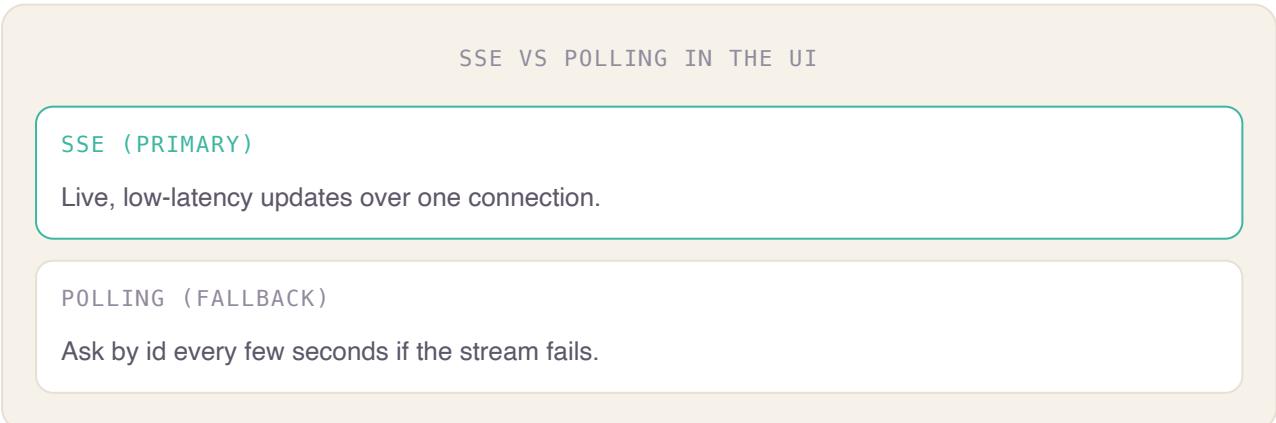


2 Append live, keep the UI responsive

As events arrive you append them to the view so the user watches the agent think in real time. You batch/throttle rendering so a fast token stream doesn't thrash the browser, and keep the UI scrolled and responsive.

3 Handle disconnects gracefully

Connections drop. SSE **auto-reconnects**, but you also design for it: the job keeps running server-side, so on reconnect you resume the stream, and if streaming fails entirely you **fall back to polling** `GET /jobs/{id}` by id.



🗺️ ANALOGY — A LIVE DELIVERY MAP

Instead of a static “out for delivery,” you watch the driver move in real time — engaging, and reassuring that progress is happening.

💡 OPTIMISTIC UI

Show “submitting…” and the job id immediately (from the 202), then stream results in — the interface feels instant even though work is async.

🔗 THE OTHER HALF OF 3·6

Stage 3 produced the stream; this lesson consumes it. Same feature, two ends of the wire.

📝 CHEAT-SHEET

- Subscribe to the Stage-3 SSE endpoint.
- Append events live; throttle rendering for fast streams.
- SSE auto-reconnects; the job runs server-side regardless.
- Fall back to polling by id if streaming fails.
- Optimistic UI: show id/“submitting” from the 202 instantly.

✅ SELF-CHECK

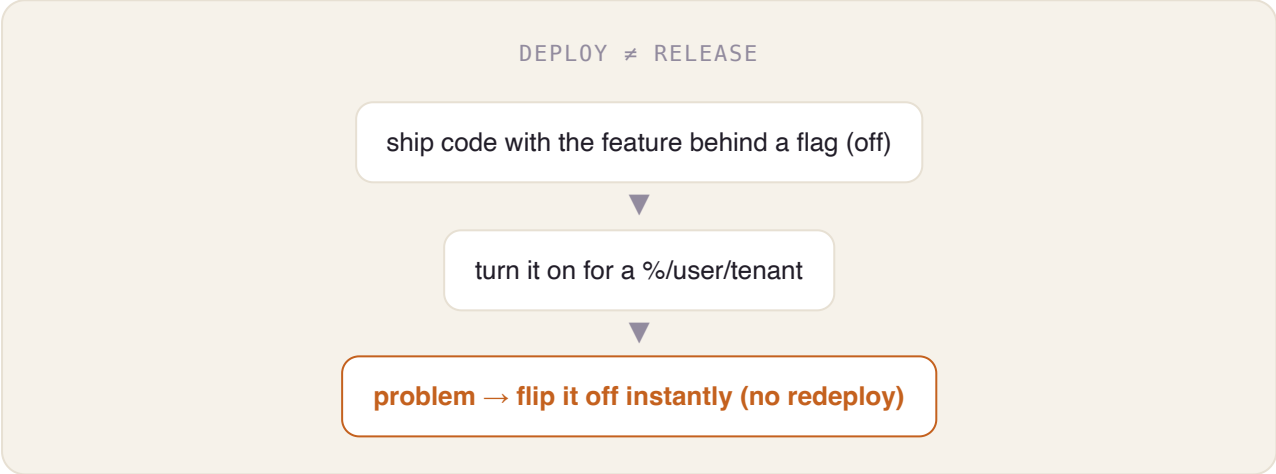
1. Where does the streamed data come from?
2. What happens to the job if the client disconnects mid-stream?
3. What's the fallback if streaming fails entirely?

9·3 Feature Flags

toggle features at runtime

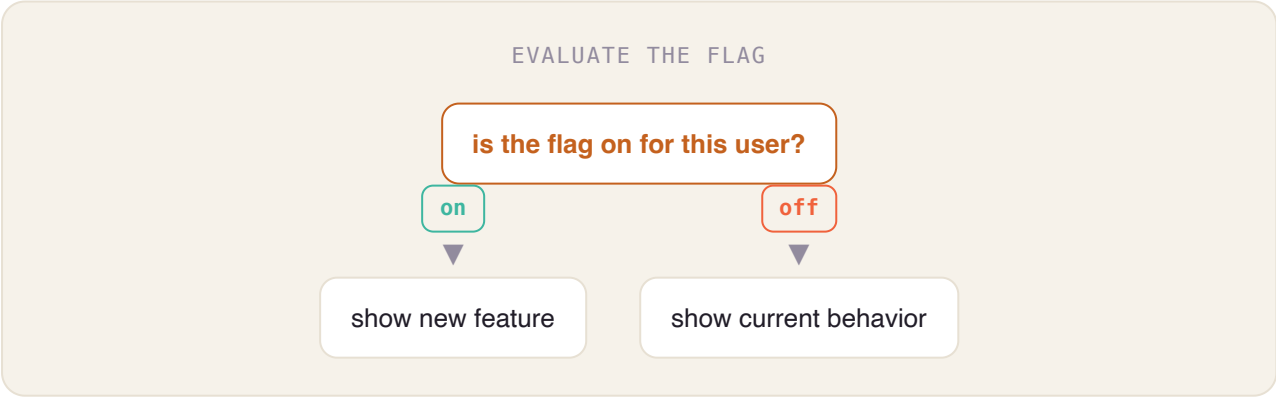
1 Decouple deploy from release

Normally shipping code = releasing a feature. **Feature flags** break that link: you deploy code with the feature **behind a switch** that's off, then turn it on later — for everyone, or a subset — *without a new deploy*.



2 Targeting: who sees it

A flag can be on for **10% of users**, for a **specific tenant**, for internal staff only, or globally. This powers gradual rollout, **dark launches** (ship code inert, enable later), and per-tenant features.



3 The kill switch — and flag debt

A flag is an instant **kill switch**: if the new feature misbehaves, flip it off in seconds — far faster than a rollback deploy. The cost: **flag debt** — stale flags accumulate and tangle the code, so you must retire them once a feature is fully shipped.

💡 **ANALOGY — A LIGHT SWITCH**

You can flip a feature on or off without rewiring the house (redeploying). And you can wire the switch to just one room (a tenant/%).

💡 **DARK LAUNCH**

Deploy risky code turned *off*, enable it for internal users first, then ramp — you separate the risk of *deploying* from the risk of *releasing*.

⚠️ **FLAG DEBT**

Every flag is a branch in your code. Stale flags multiply paths and confuse behavior — schedule their removal once the feature is permanent.

📝 **CHEAT-SHEET**

- Flags decouple deploy from release.
- Target by %/user/tenant; enables dark launches.
- Instant kill switch — faster than a rollback.
- Cost: flag debt — retire stale flags.
- Foundation for the A/B experiments (9·4).

✅ **SELF-CHECK**

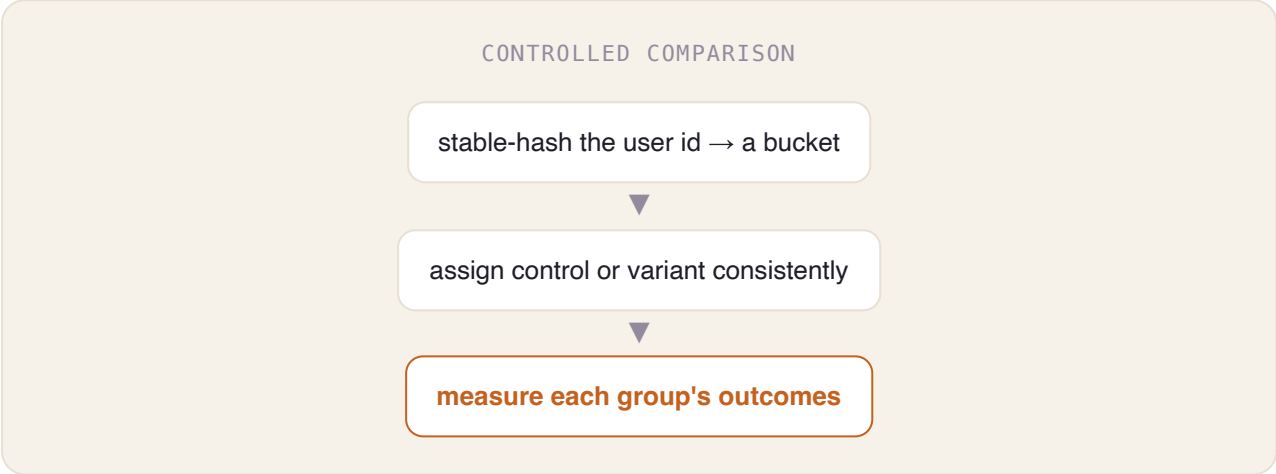
1. How do flags separate deploying from releasing?
2. What is a dark launch?
3. What's the maintenance cost of flags?

9·4 Traffic Splitting & Assignment

the mechanism behind A/B

1 A controlled comparison on real traffic

To know if a new agent config or model is actually better, you run both at once: a **control** (current) and a **variant** (new), each serving a slice of real users, and compare outcomes. Traffic splitting is the plumbing that assigns who gets which.

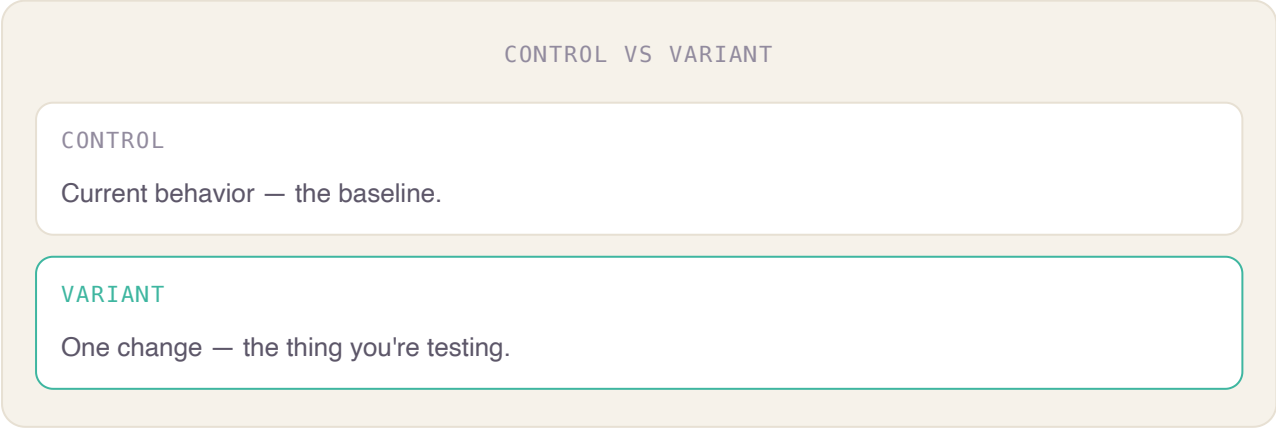


2 Stable assignment (sticky buckets)

Assignment must be **deterministic and sticky**: hash the user id so the *same* user always lands in the *same* group across requests. Otherwise a user flips between variants and your comparison is meaningless.

3 Isolate one change; validate the pipe

Change **one thing at a time** so any difference is attributable. And run an **A/A test** occasionally (both groups identical) — if it shows a "difference," your splitting/measurement is broken. Watch for **sample-ratio mismatch** (groups not the size you set) as a red flag.



🍽️ **ANALOGY — 10% GET THE NEW MENU**

Give a tenth of diners the new menu and compare against the old — same restaurant, same conditions, one difference: the menu.

💡 **STICKY ASSIGNMENT IS ESSENTIAL**

Consistent bucketing per user is what makes the comparison valid. Random-per-request assignment destroys the experiment.

⚠️ **SAMPLE-RATIO MISMATCH**

If you set 50/50 but see 55/45, something's wrong with assignment or logging — don't trust the results until it's fixed.

📝 **CHEAT-SHEET**

- Compare control vs variant on real traffic.
- Stable (sticky) assignment: hash the user id.
- Change one thing at a time → attributable results.
- A/A test validates the splitting pipeline.
- Watch for sample-ratio mismatch.

✅ **SELF-CHECK**

1. Why must assignment be stable per user?
2. What's the point of a control group?
3. What does an A/A test check?

9·5 Statistical Significance

is the result real or luck?

1 Real effect vs random noise

A variant looking 3% better might be a genuine improvement — or just noise from a small sample. **Statistical significance** is how you decide whether the observed difference is unlikely to be chance.

REAL VS NOISE

SIGNAL

Difference persists with enough data → likely real.

NOISE

Difference vanishes as data grows → was luck.

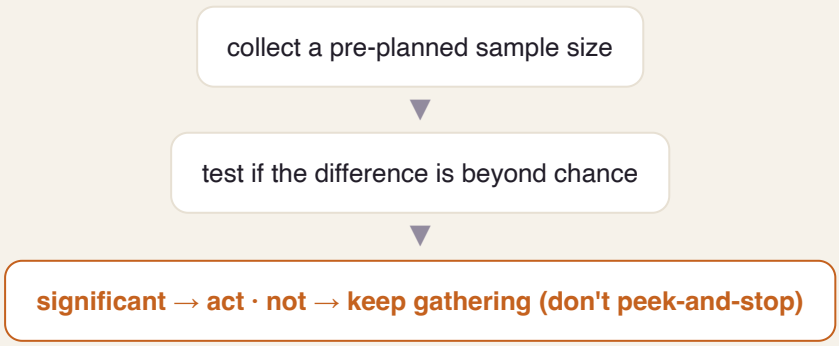
2 Sample size & confidence

You need **enough samples** before trusting a result — small samples swing wildly. A significance test (p-value / confidence interval) tells you the probability the difference is chance; below a threshold, you call it significant.

3 The peeking problem

If you keep checking and **stop the moment it looks good**, you'll fool yourself — repeatedly peeking inflates false positives. Fix the sample size (or use sequential/Bayesian methods designed for continuous monitoring) and don't stop early on a lucky reading.

DECIDE HONESTLY



🪙 ANALOGY — A LUCKY COIN

Three heads in a row doesn't mean the coin is rigged. You need many flips before betting the business that “heads” is really more likely.

⚠️ PEEKING & EARLY STOPPING

Watching the dashboard and stopping at the first “win” is the most common way teams ship noise. Commit to a sample size (or a proper sequential method).

💡 STATISTICAL ≠ PRACTICAL SIGNIFICANCE

A change can be statistically significant but too tiny to matter. Ask “is this difference big enough to care about?” too.

📝 CHEAT-SHEET

- Distinguish real effects from random noise.
- Need adequate sample size before trusting a result.
- Significance test = probability the difference is chance.
- Don't peek-and-stop → inflates false positives.
- Statistical significance ≠ practical significance.

✅ SELF-CHECK

1. What does statistical significance protect you from?
2. Why is sample size important?
3. What's the danger of stopping the moment it looks good?

9·6 Guardrails & Auto-Rollback

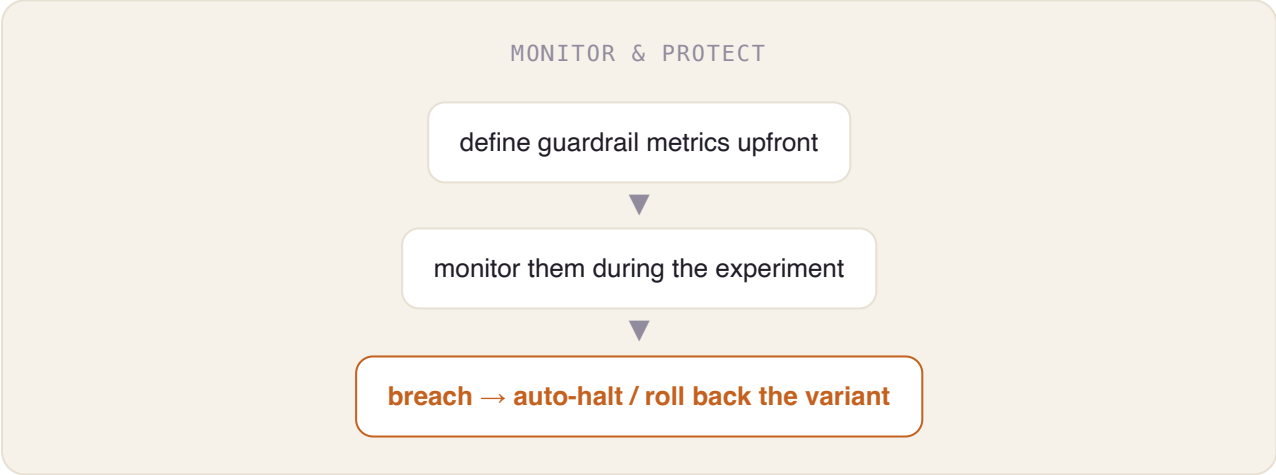
experiment boldly, fail safe

1 Bold experiments need a safety net

You want to try aggressive changes, but not at the cost of harming users. **Guardrail metrics** + **auto-rollback** let you push hard because a bad experiment is *caught and reverted automatically* before it does real damage.

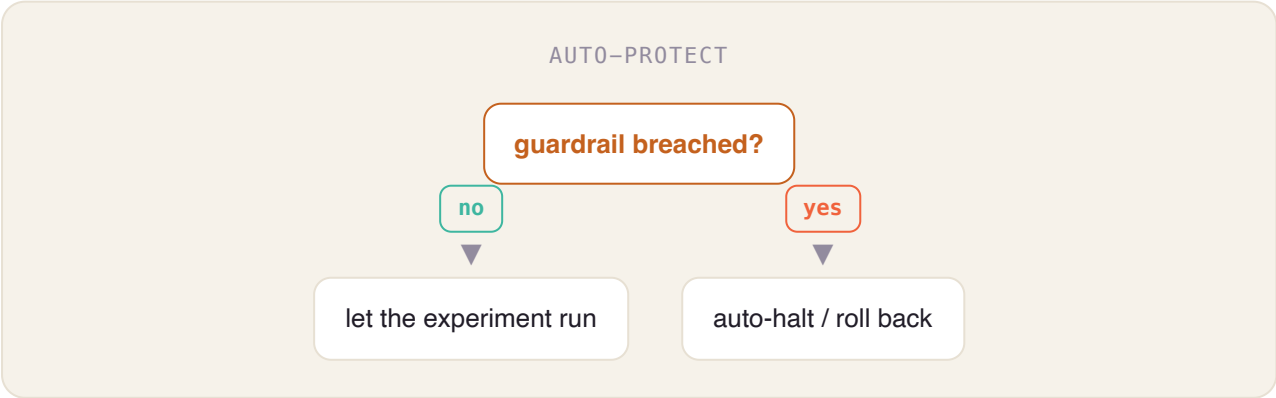
2 Guardrail metrics

Alongside the metric you're *optimizing* (say, task success), you monitor **guardrails** you must not harm — latency, error rate, cost, safety. These are the “don't make anything worse” tripwires, distinct from the goal metric.



3 Automatic halt

If a guardrail crosses its threshold, the system **automatically halts** the variant (or rolls it back to control) — no human in the loop at 3am. This is what makes fast experimentation safe: the blast radius is bounded and the reaction is instant.



🔌 ANALOGY — A SAFETY CUTOFF

Like a machine that cuts power the instant a limit is crossed — you can run it fast because the cutoff protects you and the equipment automatically.

💡 GUARDRAILS ≠ YOUR GOAL METRIC

Optimizing task success while a guardrail watches latency/cost/errors keeps you from “winning” on one metric by quietly wrecking another.

🔗 GUARDRAILS ARE SLO-SHAPED (STAGE 12)

The same reliability targets and burn-rate thinking from Stage 12 make natural guardrails for experiments.

📝 CHEAT-SHEET

- Guardrails + auto-rollback make bold experiments safe.
- Guardrail metrics = “don't harm” tripwires (latency/error/cost).
- Distinct from the goal metric you're optimizing.
- Breach → automatic halt/rollback, no human needed.
- Guardrails are SLO-shaped.

✅ SELF-CHECK

1. What triggers an auto-rollback?
2. How do guardrail metrics differ from the metric you're optimizing?
3. Why do guardrails let you experiment more boldly?

◆ STAGE 10 — PER-TENANT PREVIEW ENVIRONMENTS ◆

10·1 Ephemeral Environments

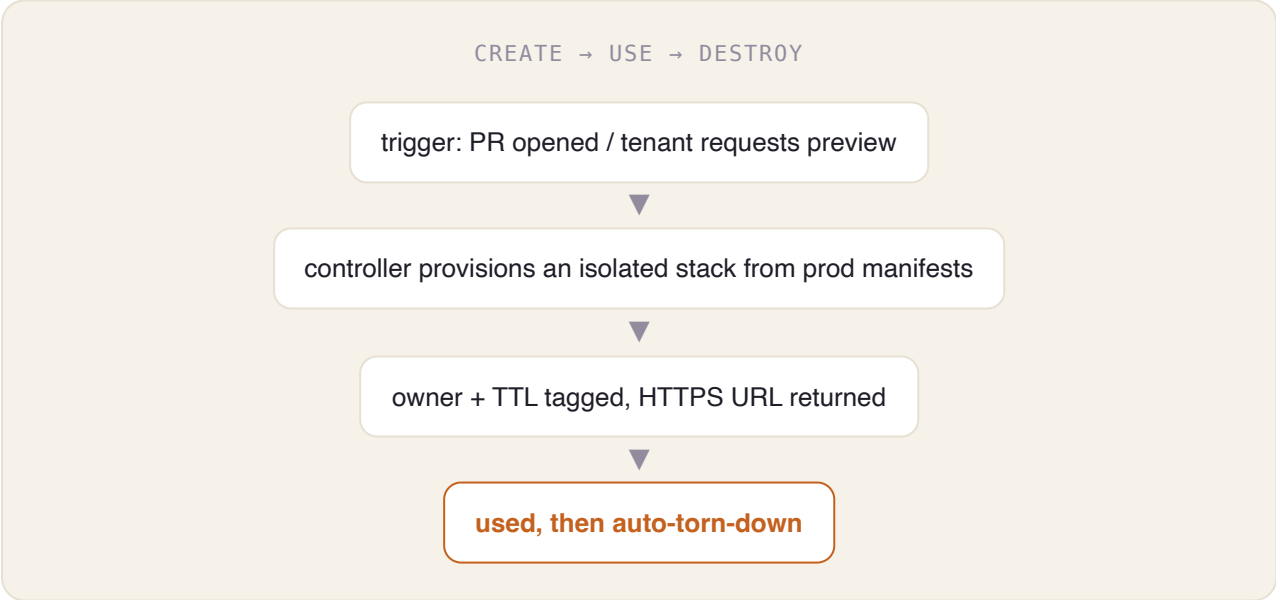
on-demand, disposable stacks

1 The problem: one shared staging is a bottleneck

A single long-lived staging environment is contended: many branches and tenants share it, so a broken change blocks *everyone*, state drifts as people poke at it, and after a few weeks “works on staging” stops meaning anything. You need an isolated, production-shaped stack for each change or tenant that exists only as long as it's needed.

2 The mechanism: provision a full stack on a trigger

On a trigger — a PR opened, a tenant requesting a preview — a controller provisions a **complete stack** (app + its datastores + routing) from the *same* manifests production uses, tags it with an **owner** and a **TTL**, and hands back a URL. Nothing is pre-allocated; the environment is created on demand and is meant to be thrown away.



3 The nuance: ephemeral is not stateless, and teardown is the hard part

Each env still needs **seeded or throwaway data** to be useful — ephemeral ≠ stateless. And the whole value collapses if environments **leak**: the genuinely hard engineering is disciplined teardown (10·5), not creation. They must also be **cheap and fast to create** — if spin-up takes ten minutes, people route around the platform and go back to sharing staging.

SHARED STAGING VS EPHEMERAL PREVIEWS

SHARED STAGING

One env, contended by all; a bad change blocks everyone; drifts over time.

EPHEMERAL PREVIEWS

Many isolated envs, one per change/tenant; disposable; no lasting footprint.

🎬 ANALOGY — A FILM SET

A set is built for one shoot, used, then struck — nobody keeps a hundred half-standing sets around. Each preview is a set built for one change and struck when the scene's done.

💡 THIS IS THE EMERGENT PREVIEW MODEL

A per-app/per-tenant preview that spins up on demand and sleeps or dies when idle is exactly the product pattern here — the same lifecycle you debug in support (wake, idle-sleep, teardown) is what this stage builds.

🔗 BUILT ON THE STAGE-8 OPERATOR

The controller that provisions and reaps these stacks is your custom operator from Stage 8; teardown discipline is 10·5.

📝 CHEAT-SHEET

- Created on demand, per tenant/branch — not pre-allocated.
- Full prod-shaped stack (app + data + routing), not a mock.
- Tagged with an owner + a TTL so it can be reaped.
- Ephemeral ≠ stateless — still needs seeded/throwaway data.
- Must be cheap+fast to create, or people won't use it.

✅ SELF-CHECK

1. What makes an environment “ephemeral”?
2. Why is a single shared staging a bottleneck?
3. What's the hardest part of a preview platform — creation or teardown?

10·2 vcluster vs Namespace Isolation

how strong a wall?

1 The problem: sharing one cluster leaks between tenants

Previews share one physical cluster to save money — but sharing leaks. A namespace boundary scopes *namespaced* objects, yet **cluster-scoped** resources (CRDs, ClusterRoles, PriorityClasses) are shared, and a noisy or misconfigured tenant can still affect neighbors. The question is: how strong must the wall between environments be?

2 The mechanism: two isolation levels

Namespace isolation draws a boundary inside a shared control plane — RBAC, `NetworkPolicy`, and `ResourceQuota` confine each env, but they all talk to the *same* API server and share cluster-scoped objects. **vcluster** runs a **virtual cluster**: its own API server and controller-manager as pods inside a host namespace, so each tenant sees what looks like a full cluster (own CRDs, own cluster-admin) while the actual workloads still schedule onto the shared host nodes.

A SPECTRUM OF ISOLATION, PRICED BY BLAST RADIUS

NAMESPACE

Light & dense; shared API server; cluster-scoped objects shared.

VCLUSTER

Own virtual API server + CRDs; workloads run on shared nodes.

SEPARATE CLUSTER

Hardest wall; own control plane + nodes; most cost.

3 The nuance: pick by threat model, not by default

Choose by what the tenant needs and who you're isolating *from*. A **namespace** is enough for trusted internal branches. Reach for a **vcluster** when a tenant must install their own operators/CRDs or wants cluster-admin without touching neighbors. Neither is a hard *security* boundary — both share the host kernel, so a kernel escape still crosses; only separate clusters/VMs or sandboxed runtimes give that. The cost of vcluster is a control plane per tenant (memory + a little API latency).

🏢 ANALOGY — FLOOR VS BUILDING

A namespace is a **locked floor** in a shared building; a vcluster is a **mini-building** on the same lot; a separate cluster is a building across town. More isolation, more rent.

💡 ISOLATION IS A SPECTRUM, NOT A SWITCH

“Isolated” isn't binary — it's a dial from namespace → vcluster → cluster → VM, each buying more blast-radius protection at more cost. Architects name the level, not just “isolate it.”

⚠️ NEITHER SURVIVES A KERNEL ESCAPE

vcluster and namespaces share the host kernel. For hostile multi-tenancy you need separate nodes/clusters or a sandbox runtime (gVisor/Kata) — don't oversell soft isolation as a security boundary.

📝 CHEAT-SHEET

- Namespace = light, shared control plane, cluster-scoped objects shared.
- vcluster = own virtual API server/CRDs; workloads on shared nodes.
- Separate cluster/VM = hardest wall, highest cost.
- Pick by threat model + whether tenants need cluster-scoped resources.
- Soft isolation shares the kernel — not a security hard-wall.

✅ SELF-CHECK

1. What does a namespace *not* isolate?
2. When is a vcluster worth its overhead?
3. Why isn't a vcluster a real security boundary against a hostile tenant?

10·3 external-dns & cert-manager

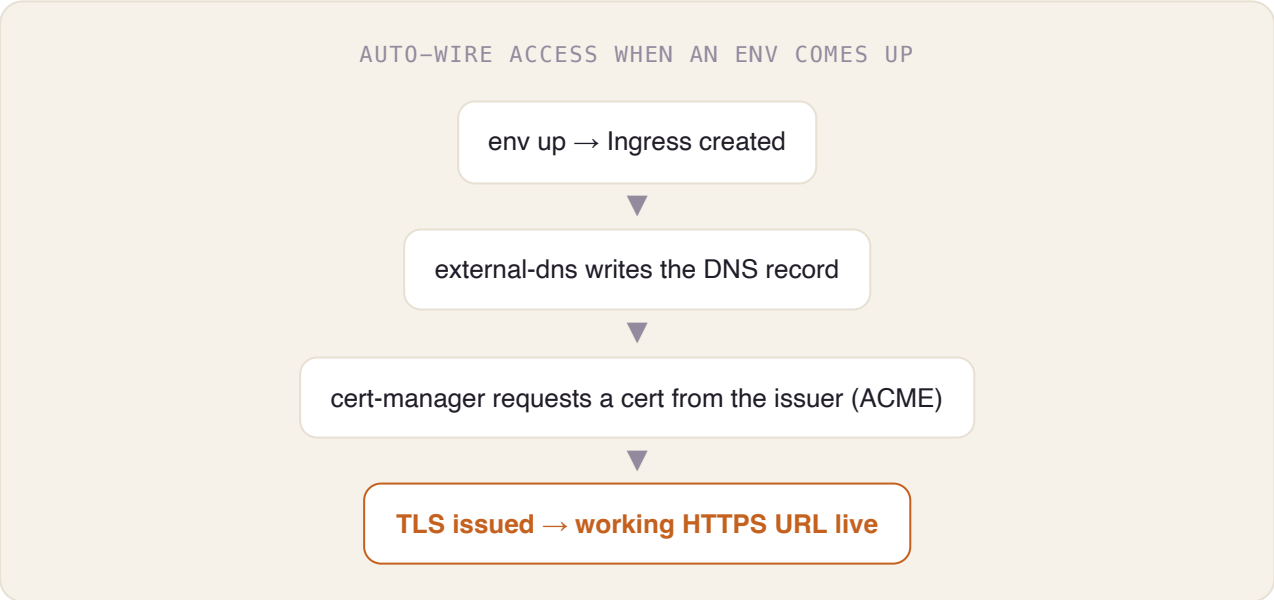
auto domains + TLS

1 The problem: every env needs a working HTTPS URL, hands-off

Each ephemeral environment needs its own unique, reachable **HTTPS URL**. Wiring DNS records and TLS certificates by hand doesn't scale to hundreds of short-lived envs — and a preview that throws a certificate warning is worthless to whoever you asked to test it.

2 The mechanism: two controllers reconciling desired → actual

external-dns watches `Ingress` / `Service` objects and writes the matching records into your DNS provider (Route 53, Cloudflare, …) automatically. **cert-manager** watches for `Certificate` resources / ingress annotations and obtains TLS certs from an **issuer** — Let's Encrypt over ACME, or an internal CA — then renews them before expiry. Both are just the reconciling-controller pattern again: declare the hostname, the records and cert appear.



3 The nuance: wildcard vs per-host, and the latency it adds

Hundreds of envs under `*.preview.example.com` are far better served by **one wildcard cert** (a DNS-01 ACME challenge) than by issuing a fresh per-host cert (HTTP-01) for every env — the latter hammers Let's Encrypt **rate limits** and fails. Remember that DNS propagation + ACME issuance add seconds-to-minutes to “env ready,” so a wildcard (or pre-provisioned) cert keeps spin-up fast (ties back to 10·1's “cheap to create”).

PER-HOST VS WILDCARD

HTTP-01 PER-HOST

A cert per env; simple, but burns ACME rate limits at scale.

DNS-01 WILDCARD

One *.preview cert covers every subdomain; fast, rate-limit-safe.

🪧 ANALOGY — SIGN-WRITER + LOCKSMITH

external-dns is an automatic sign-writer that posts each pop-up's address; cert-manager is the locksmith that fits its locks — both show up the moment the shop opens, no call needed.

💡 SAME OPERATOR PATTERN, AGAIN

Neither tool is special magic — both are controllers reconciling a declared state (a hostname) into a real-world side effect (a DNS record, a cert). Once you see it, every k8s add-on looks the same.

⚠️ RATE LIMITS & PROPAGATION ARE REAL

Let's Encrypt caps issuance per domain per week; a cert-per-env pattern hits it fast. Prefer wildcard/DNS-01 and pre-warm, and budget DNS-propagation time into “env ready.”

📝 CHEAT-SHEET

- external-dns = auto DNS records from Ingress/Service objects.
- cert-manager = auto TLS issuance + renewal from an issuer (ACME/CA).
- Every preview gets a real HTTPS URL, zero manual wiring.
- Prefer a DNS-01 wildcard cert at scale — avoids ACME rate limits.
- Issuance/propagation add latency — count it in spin-up time.

✅ SELF-CHECK

1. What object does external-dns react to, and what does it produce?
2. Why is a wildcard/DNS-01 cert better than per-host at scale?
3. What two latencies do these add to “env ready”?

10·4 Argo CD ApplicationSets

stamp out many instances

1 The problem: one template, N tenants, all kept in sync

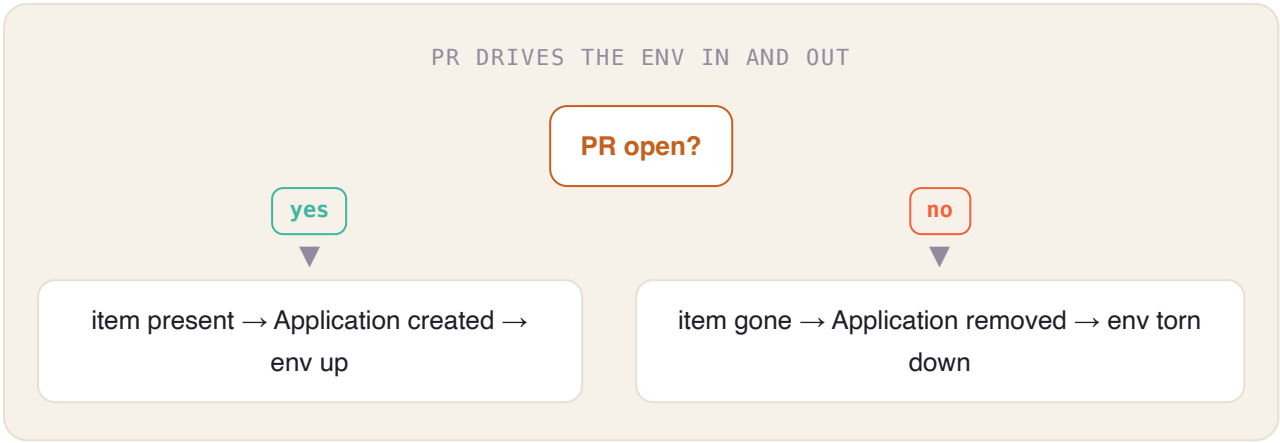
You have *one* preview-app template but **N** tenants/branches, each needing its own Argo `Application`, each kept in sync, each created and deleted as tenants come and go. Hand-writing and maintaining an `Application` per env doesn't scale and drifts the moment someone edits one by hand.

2 The mechanism: generator + template = for-each

An **ApplicationSet** is a controller with a **template** and one or more **generators**. A generator produces a *list of items* — one per PR (the PR generator), one per Git directory, one per registered cluster, or a static list — and the controller renders the `Application` template once per item, creating/updating/deleting Applications automatically as that list changes. It's a declarative `for-each` over your environments.

3 The nuance: this is what makes previews self-service

Wire the **PR generator** and the platform becomes self-serve: open a PR → an item appears → an `Application` is created → Argo syncs the env; close the PR → the item disappears → the `Application` (and, via a teardown hook, the whole env) is removed. Git/PR state is the single source of truth and drift self-heals. Two cautions: a bad edit to the *template* fans out to every env at once, and whether deletion truly tears down workloads depends on `preserveResourcesOnDeletion` and finalizers (10·5).



🍪 **ANALOGY — A COOKIE-CUTTER FROM GIT**

One master shape (the template) presses out an identical Application for each item the generator lists — all driven from one design in Git, not hand-cut one by one.

💡 **GENERATORS ARE THE "FOR-EACH" OF GITOPS**

Plain Argo `Application` = one app. ApplicationSet adds iteration: "for each PR / cluster / directory, make one." That single idea is what turns GitOps into a fleet manager.

🔗 **THE KNOT THAT TIES THE STAGE TOGETHER**

The PR generator is 10·1's on-demand *trigger*; its item disappearing is 10·5's teardown *signal*. ApplicationSet is the mechanism connecting create and destroy.

📝 **CHEAT-SHEET**

- ApplicationSet = controller + template + generator(s).
- Generator lists items (per PR / dir / cluster / static).
- Template rendered once per item → many Applications.
- PR generator makes previews self-service (open=up, close=down).
- Template edits fan out to all envs — change carefully.

✅ **SELF-CHECK**

1. What are the two moving parts of an ApplicationSet?
2. How does the PR generator make previews self-service?
3. What's the risk of editing the shared template?

10·5 Lifecycle & Teardown

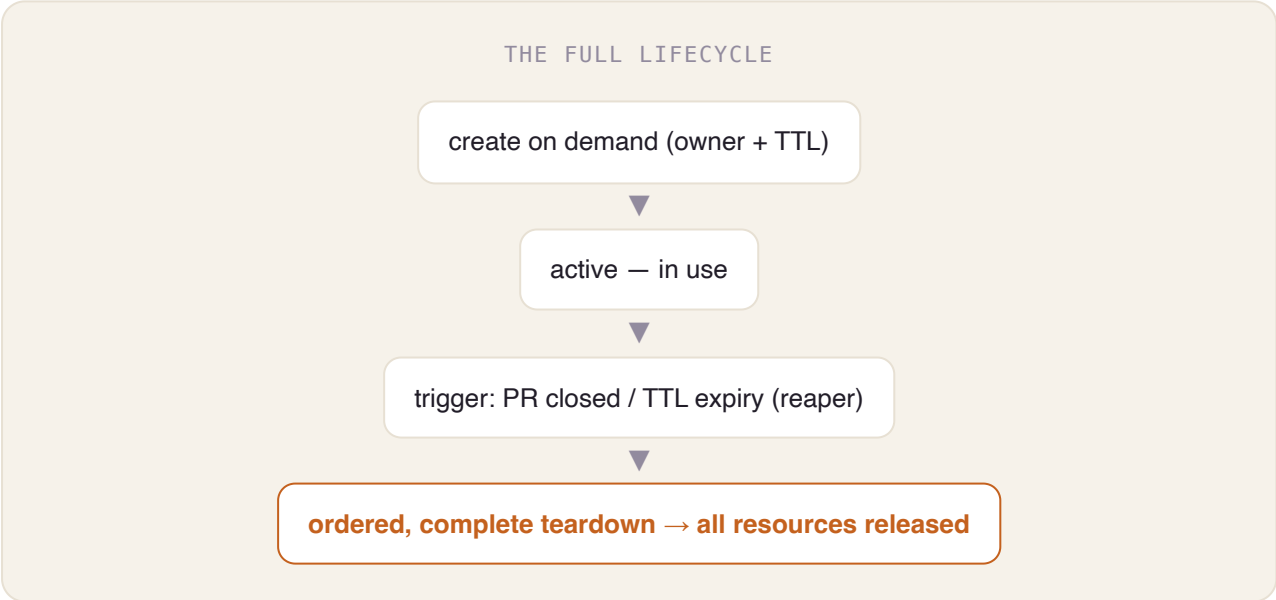
clean up automatically

1 The problem: the envs that never die

Creating environments is easy; the failure mode of *every* preview platform is envs that **never die** — orphaned namespaces, dangling PVCs, stale DNS records, forgotten cloud load balancers, spend accruing on stacks nobody's used in weeks. Beyond cost, each zombie env is a security liability: old secrets and old code still reachable.

2 The mechanism: bind every env to a trigger and a TTL

Give each env an explicit lifecycle tied to a **trigger** (PR merged/closed, tenant done) *and* a **TTL**, with a **reaper** controller that sweeps envs past expiry. Teardown must be **complete** — not just the Deployment, but PVCs, DNS records, certs, external load balancers, and secrets. **Finalizers** hold the namespace open until dependent cloud resources are actually released, so nothing is orphaned behind a deleted namespace.



3 The nuance: order it, make it idempotent, and choose sleep vs destroy

Teardown has an **order**: drain traffic → remove ingress/DNS → delete workloads → release volumes — and it must be **idempotent** so a half-torn-down env fully cleans on retry. Decide a **data policy** (snapshot before destroy if anything's worth keeping). And weigh **destroy vs idle-sleep**: fully destroying costs nothing but rebuilds cold; scaling to zero and waking on demand is fast but leaves a small residual footprint — exactly the idle-sleep/wake trade-off behind Emergent's preview pods.

DESTROY VS IDLE-SLEEP

DESTROY ON EXPIRY

Zero residual cost; full cold rebuild on next use.

IDLE-SLEEP / SCALE-TO-ZERO

Fast wake; small residual cost (storage/DNS kept warm).

🏨 ANALOGY — HOTEL CHECKOUT

Checkout cleans the room for the next guest — nothing lingers, nothing is billed forever. Idle-sleep is more like leaving the key active but the lights off between stays.

💡 A TTL + REAPER IS THE ONLY RELIABLE TEARDOWN

Never rely on a human remembering to delete an env. A default TTL and a reaper that enforces it is what keeps the platform from silently rotting into a cost and security problem.

⚠️ INCOMPLETE TEARDOWN IS THE #1 LEAK

Deleting the Deployment but leaking its PVC, DNS record, or cloud load balancer is the most common (and most expensive) mistake. Use finalizers to release cloud resources *before* the namespace disappears.

📝 CHEAT-SHEET

- Every env gets a trigger + a TTL; a reaper enforces expiry.
- Teardown must be complete: workloads, PVCs, DNS, certs, LBs, secrets.
- Finalizers release cloud resources before namespace deletion.
- Order teardown (drain → ingress/DNS → workloads → volumes); make it idempotent.
- Destroy vs idle-sleep = cost-to-zero vs fast-wake trade-off.

✅ SELF-CHECK

1. Why is a TTL + reaper essential rather than manual deletion?
2. Name three resources beyond the Deployment that must be torn down.
3. What's the trade-off between destroy and idle-sleep?

◆ STAGE 11 — CLOUD, IAC & MULTI-REGION ◆

11·1 VPC & Networking (SG vs NACL)

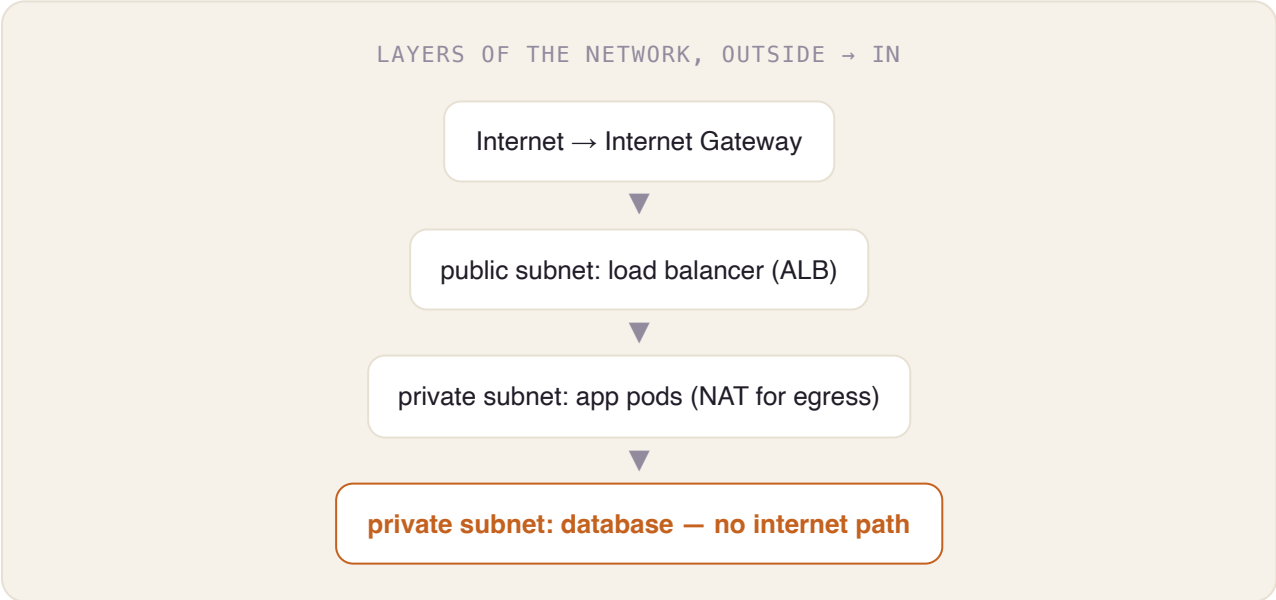
the private cloud network

1 The problem: a flat network means one breach reaches everything

Everything you run in the cloud sits on *some* network. If that network is flat and public, a single compromised host can reach your database directly. You need a private, segmented network where reachability is **deliberate** — nothing talks to anything unless you said so.

2 The mechanism: a VPC, subnets, and two firewall layers

A **VPC** is your own isolated IP space, carved into **subnets** across Availability Zones: *public* subnets (routed to an internet gateway) for load balancers, *private* subnets (egress via NAT, no inbound from the internet) for app and data. On top sit two firewalls: **Security Groups** — attached to a resource, **stateful** (return traffic auto-allowed), allow-rules only — and **NACLs** — attached to a subnet, **stateless** (each direction evaluated), and able to *deny*.



3 The nuance: defense in depth, and where each layer earns its keep

Security Groups are your primary control and their superpower is referencing *other SGs* — "the app SG may reach the db SG on 5432" — an identity rule that survives autoscaling where IPs churn. **NACLs are a coarse backstop**: block a CIDR, quarantine a subnet. Because NACLs are stateless you must remember to open **ephemeral return ports**, or connections silently hang. Spanning subnets across AZs (with matching route tables) is what makes the network survive an AZ loss.

SECURITY GROUP VS NACL

SECURITY GROUP

Per-resource, stateful, allow-only; can reference other SGs (identity-based).

NACL

Per-subnet, stateless, allow + deny by CIDR; a coarse subnet-wide backstop.

🏘️ ANALOGY — A GATED COMMUNITY

NACLs are the neighborhood's street-level rules (who may enter this block at all); SGs are the per-house door locks (who may enter *this* house). Two independent layers deciding who reaches what.

💡 SG-TO-SG REFERENCES BEAT IP RULES

Referencing an SG instead of a CIDR means the rule keeps working as pods scale in/out and IPs change — you're allowing an *identity*, not an address. This is the cloud-native way to write network policy.

⚠️ STATELESS NACLS BITE ON RETURN TRAFFIC

Forgetting to allow the ephemeral high-port range for return packets is the classic NACL mistake — the request goes out, the reply is dropped, and it looks like a mysterious timeout. Also: cross-AZ traffic is billable.

📝 CHEAT-SHEET

- VPC = your isolated cloud network; public vs private subnets across AZs.
- Private subnets hold app + data; only LBs live public.
- SG = stateful, per-resource, allow-only, references other SGs.
- NACL = stateless, per-subnet, supports explicit deny by CIDR.
- Span subnets across AZs for HA; watch cross-AZ egress cost.

✅ SELF-CHECK

1. Why put databases in a private subnet?
2. Which layer is stateful, and why does that matter?
3. What's the advantage of referencing an SG instead of a CIDR?

11·2 IAM & IRSA

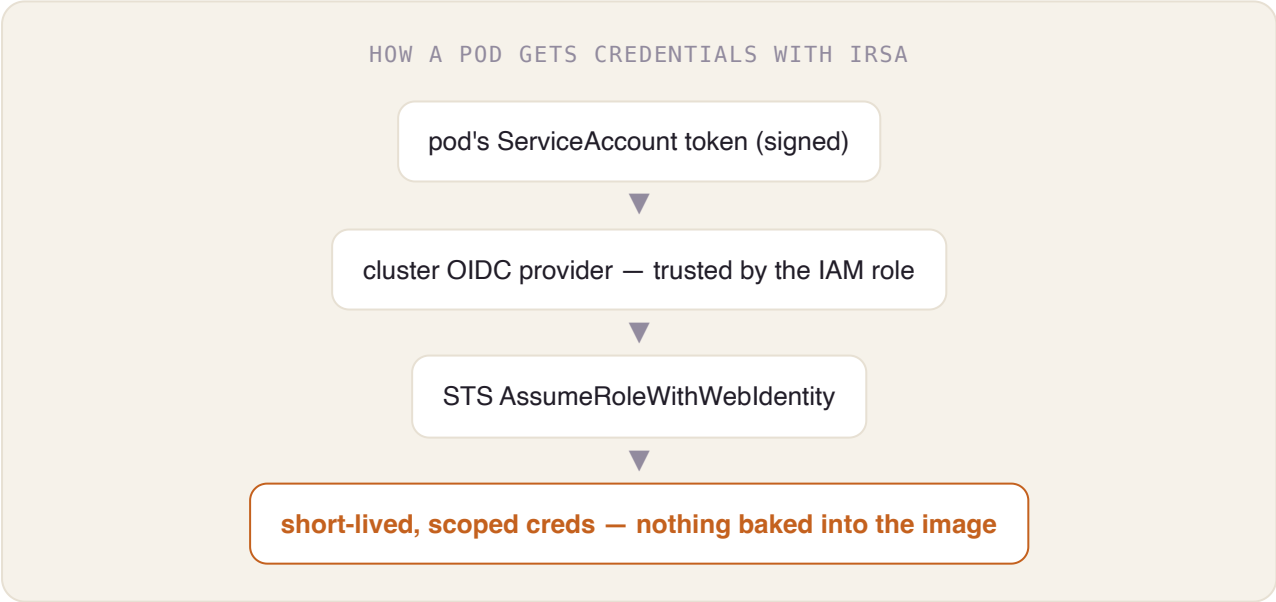
least-privilege cloud identity

1 The problem: static keys in workloads are the #1 breach vector

Your app needs to call cloud APIs — read an S3 bucket, publish to a queue. The lazy answer is to bake an access key into the image or an env var. That key is **long-lived, over-broad, and un-rotatable**; it leaks into logs, Git history, and crash dumps. Leaked static credentials are the single most common cause of cloud breaches.

2 The mechanism: IAM roles + IRSA give each pod its own short-lived identity

IAM models access as *principals* (users, roles) plus *policies* (JSON allow/deny over actions, resources, conditions). Prefer **roles** (assumed → temporary creds) over users (static keys). **IRSA** ties a Kubernetes `ServiceAccount` to an IAM role through an OIDC trust: the pod presents a signed SA token, STS exchanges it (`AssumeRoleWithWebIdentity`) for **short-lived, scoped** AWS credentials. No secret is ever stored.



3 The nuance: least privilege is a practice, and trust is the real perimeter

Least privilege means scoping **actions AND resources AND conditions** — start from deny-all and add only what breaks. **Explicit deny always wins**, and *permission boundaries* cap the maximum a role can ever hold. The same pattern exists everywhere — GCP Workload Identity, Azure Workload Identity — all OIDC federation. The classic IRSA hole isn't the permissions but the **trust policy**: too broad a `sub` claim lets *any* pod assume the role.

STATIC KEY VS IRSA ROLE

STATIC ACCESS KEY

Long-lived, often over-broad, leaks easily, manual rotation.

IRSA ROLE

Short-lived, tightly scoped, auto-rotated, fully auditable.

🪪 ANALOGY — JOB-SPECIFIC KEYCARDS

Instead of copying the building master key into every worker's pocket (static key), each worker gets their own card that opens only their doors and expires each shift (IRSA).

💡 FEDERATION REPLACES STORED SECRETS

IRSA's real idea: trust a token *issuer* rather than store a credential. Once you see identity as "a claim someone else vouches for," OIDC/SAML/workload-identity all become the same move.

⚠️ THE TRUST POLICY IS THE HOLE, NOT THE PERMISSIONS

A wildcard or overly broad OIDC `sub` condition lets any ServiceAccount assume the role — scope the *trust* as tightly as the permissions, or least-privilege on the policy is moot.

📝 CHEAT-SHEET

- IAM = principals + policies (actions, resources, conditions).
- Prefer roles (temporary creds) over users (static keys).
- IRSA = per-pod AWS identity via OIDC → STS short-lived creds.
- Least privilege: deny-all, add minimally; explicit deny wins.
- Scope the OIDC trust policy tightly, not just the permissions.

✅ SELF-CHECK

1. What security problem does IRSA remove entirely?
2. What does a permission boundary do that a policy doesn't?
3. Where's the classic misconfiguration in an IRSA setup?

11·3 Managed Services (EKS/RDS/…)

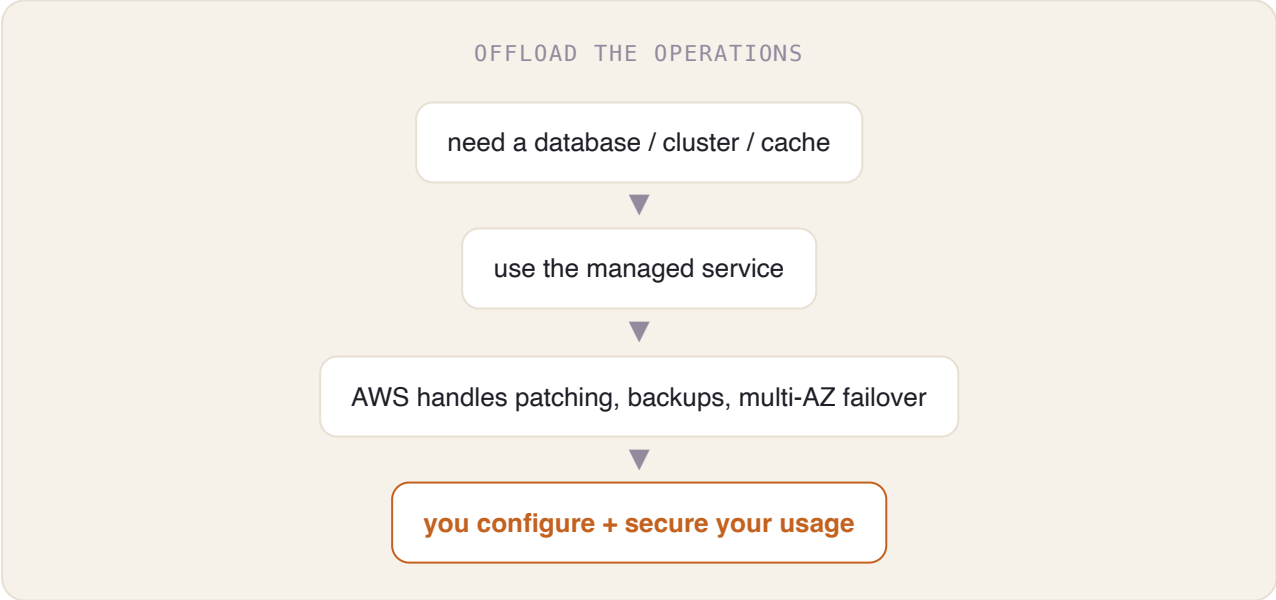
rent, don't self-host

1 The problem: self-hosting stateful infra is undifferentiated heavy lifting

Running your own database, Kafka, or cache means *you* own patching, backups, failover, version upgrades, and the 3am page when a disk fills. None of that makes your product better than a competitor's — it's **undifferentiated heavy lifting**, and it quietly consumes the team.

2 The mechanism: rent AWS-run building blocks

Instead you rent managed primitives — **RDS/Aurora** (databases), **ElastiCache** (cache), **MSK** (Kafka), **S3** (objects), **EKS** (the k8s control plane). Each gives you built-in multi-AZ HA, automated backups, and patching behind an API. The **shared-responsibility line** moves: AWS runs and secures the service itself; you configure and secure *your use* of it.



3 The nuance: you trade cost, control, and lock-in for ops relief

Managed costs more per unit and hides tuning knobs — you can't SSH in and you inherit their upgrade cadence. Choose **self-host** when you need a capability/version the managed one won't give, or at a scale where the premium dwarfs an ops hire. The architect's move is to **name the shared-responsibility boundary explicitly** and weigh portability: sticking to open protocols (the Postgres wire protocol, the S3 API) keeps an exit and limits lock-in.

SELF-HOST VS MANAGED

SELF-HOST

Full control, cheapest unit cost — you own all the ops and the pager.

MANAGED

Less control + some lock-in, higher unit cost — AWS owns the ops.

🏨 ANALOGY — A SERVICED APARTMENT

You could build and maintain a whole house (self-host), or rent a serviced apartment where plumbing, cleaning, and repairs are handled — you pay a premium to never think about the boiler.

💡 PAY TO DELETE NON-DIFFERENTIATING WORK

The core cloud value proposition is buying back engineering time from work that doesn't distinguish you. Reserve self-hosting for where control genuinely *is* your differentiator.

🔗 THE BILL IS A DESIGN INPUT

Managed convenience shows up on the invoice — autoscaling, storage, and egress add up fast. That's why FinOps (11·6) treats cost as an architecture property, not an afterthought.

📝 CHEAT-SHEET

- Managed = built-in HA/backups/patching behind an API.
- Shared responsibility: AWS runs the service, you secure your use.
- Trade: higher unit cost + less control + lock-in for ops relief.
- Self-host only where control is a real differentiator.
- Prefer open protocols (Postgres/S3 API) to keep an exit.

✅ SELF-CHECK

1. What is “undifferentiated heavy lifting”?
2. What exactly do you trade for a managed service?
3. How do open protocols reduce lock-in?

11·4 Terraform (IaC)

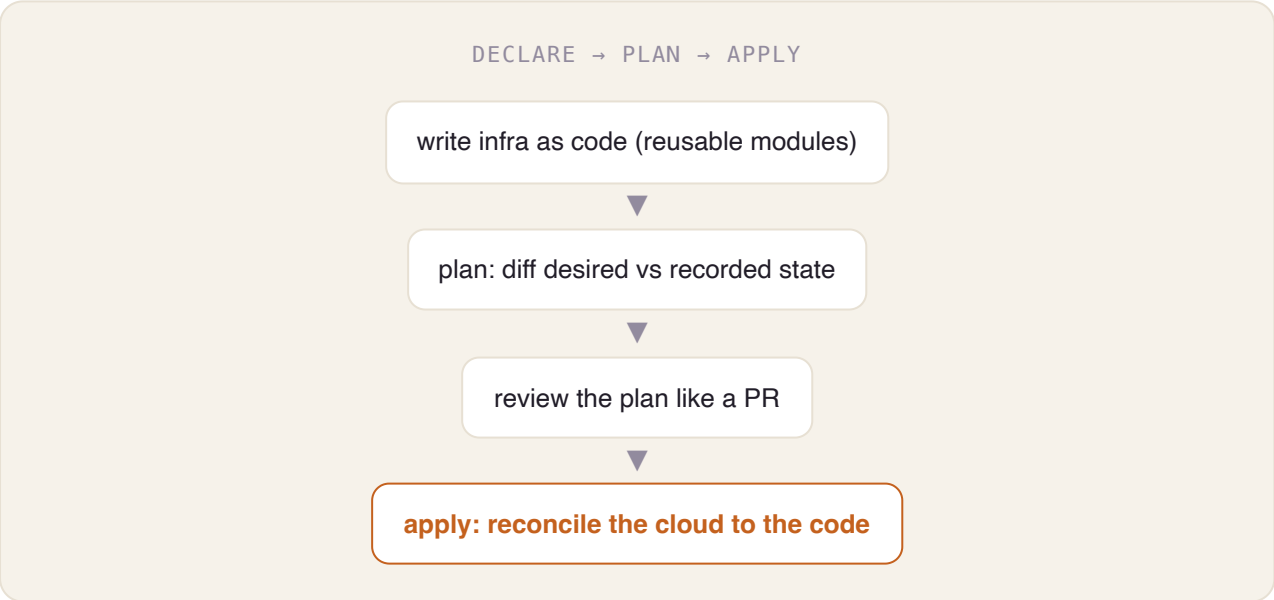
infrastructure as versioned code

1 The problem: console clicking makes snowflake infrastructure

Building infra by clicking in the console produces **snowflakes** — undocumented, unreproducible, un-reviewable resources nobody can recreate in a second region or after a disaster. Over time reality *drifts* from anyone's mental model, and there's no history of who changed what or why.

2 The mechanism: declare desired state; plan then apply

You declare the **desired** infrastructure as code. Terraform compares it against a recorded **state file** (its record of what exists), shows a `plan` (the diff), and on `apply` reconciles the real cloud to match. **Remote state** (e.g. S3 + a lock table) shares that state safely across a team, and **modules** package reusable, composable infra so a “VPC” or “service” is defined once and stamped out.



3 The nuance: the state file is both crown jewel and footgun

State holds the mapping to real resources *and* often secrets — so it must never be committed, must be **locked** (a concurrent apply corrupts it), and drifts must be reconciled (`import`, drift detection). Declarative + plan-review gives infra the same rigor as application code. But a plan is not a guarantee — provider races and eventual consistency happen — so **pin provider versions** and test modules (Terratest).

CONSOLE VS IAC

CONSOLE CLICKING

Snowflakes, silent drift, no history or review.

INFRASTRUCTURE AS CODE

Reproducible, peer-reviewed, versioned, diffable.

📐 ANALOGY — A BLUEPRINT

IaC is a blueprint you can rebuild the entire building from, exactly, at any time — and you diff and review the blueprint before you move a single wall.

💡 IT'S THE RECONCILIATION LOOP AGAIN

Desired-state → observe actual → converge is the same engine behind Kubernetes controllers and Argo CD. Terraform is that loop applied to cloud infrastructure — one mental model covers all three.

⚠️ PROTECT AND LOCK THE STATE

State is sensitive and single-writer: never commit it, always use locked remote state. Two unlocked applies racing on the same state can corrupt or destroy live infrastructure.

📝 CHEAT-SHEET

- Declarative, versioned, reviewable infrastructure.
- plan = the diff (desired vs state); apply = converge reality.
- Remote, locked state shared across the team; modules for reuse.
- State is sensitive + single-writer — never commit, always lock.
- Pin providers; test modules (Terratest); reconcile drift.

✅ SELF-CHECK

1. What does a Terraform plan actually show you?
2. Why must remote state be locked?
3. Why is a plan not an absolute guarantee of the result?

11·5 Multi-Region (active-passive → active-active)

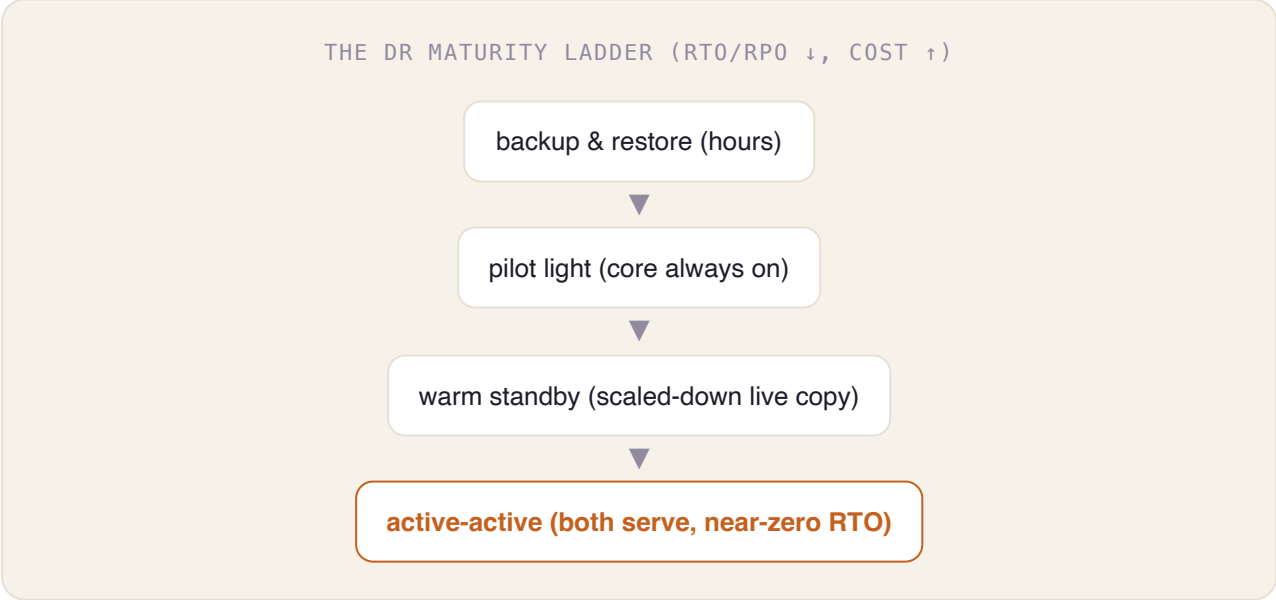
survive a region outage

1 The problem: one region is one fault domain

Multi-AZ survives the loss of a data center, but an entire **region** can fail — including its control plane. Some workloads must survive that. Multi-region is expensive and genuinely hard, though, so you justify it with concrete **RPO/RTO** targets, not reflex — most systems don't need it, and paying for it blindly is over-engineering.

2 The mechanism: replicate data, route around failure

You replicate data to another region and steer traffic away from failure. **Active-passive**: one region serves while a warm standby holds replicated data and health-based routing (Route 53) fails over on outage. **Active-active**: both regions serve live. There's a maturity ladder — *backup/restore → pilot light → warm standby → active-active* — each rung lowering RTO/RPO at higher cost and complexity.



3 The nuance: the hard part is data, not compute

Spinning up compute elsewhere is easy; keeping *data* correct is the problem. **Async** replication → a non-zero **RPO** (you can lose the last few seconds on failover); **sync** replication removes that but taxes every write with cross-region latency. Active-active forces you to resolve **write conflicts** (last-writer-wins, CRDTs, or region-sharded ownership). And failover you never rehearse won't work — **game-day** it, and design against split-brain.

ACTIVE-PASSIVE VS ACTIVE-ACTIVE

ACTIVE-PASSIVE

One region serves; a standby holds replicated data and takes over on failover.

ACTIVE-ACTIVE

Both regions serve live traffic; requires real data-conflict handling.

🏢 ANALOGY — A BACKUP HQ

Active-passive is a fully-staffed backup HQ in another city ready to take over; active-active is a second HQ that *also* serves customers every day. The second is far harder to keep in sync.

💡 RPO/RTO PICK THE ARCHITECTURE

Set how much data you can lose (RPO) and how fast you must recover (RTO) *first*, then buy the cheapest ladder rung that meets them. The targets drive the design — not the other way around.

⚠️ UNTESTED FAILOVER IS THEATER

Replication lag (RPO), split-brain, and stale DNS TTLs all surface only during a real outage. If you haven't run a game day, assume your failover is broken.

📝 CHEAT-SHEET

- One region = one fault domain; justify multi-region by RPO/RTO.
- Ladder: backup/restore → pilot light → warm standby → active-active.
- Async replication → non-zero RPO; sync → latency tax.
- Active-active must resolve write conflicts (LWW/CRDT/sharding).
- Rehearse failover (game days); design against split-brain.

✅ SELF-CHECK

1. Why justify multi-region with RPO/RTO rather than defaulting to it?
2. Why is data the hard part, not compute?
3. What must active-active solve that active-passive doesn't?

11·6 FinOps

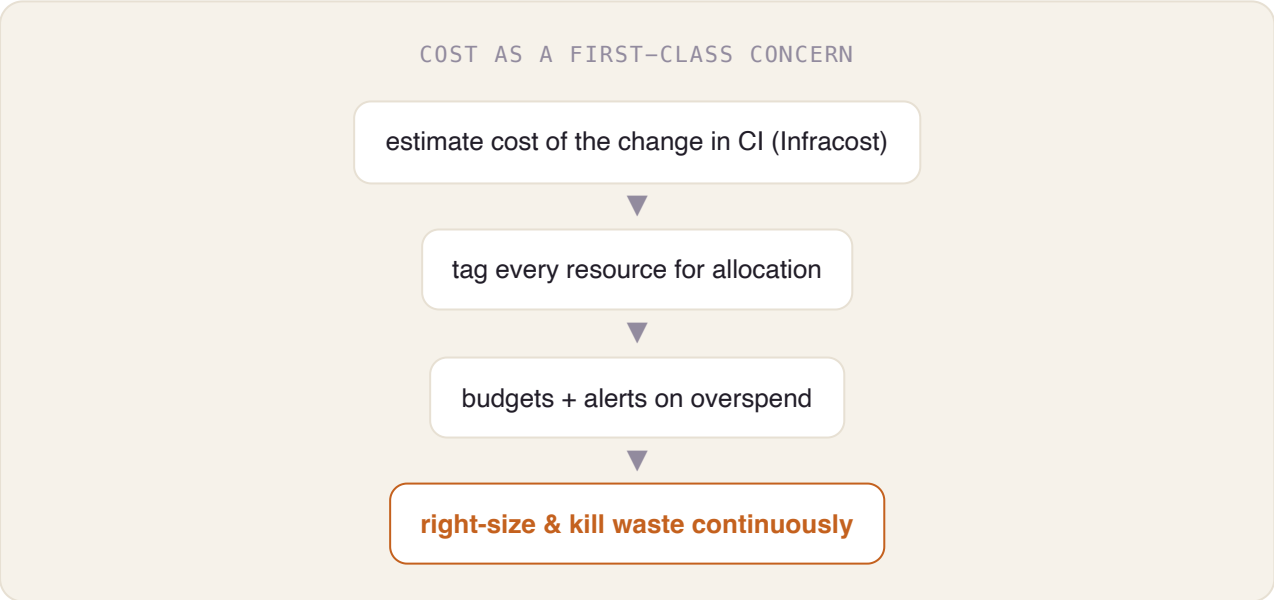
make cloud cost visible

1 The problem: cloud spend is emergent and silent

In the cloud every engineer can provision resources, cost accrues by the second, and the bill only arrives at month-end — aggregated past the point of recognition. Waste (idle capacity, over-provisioned instances, forgotten Stage-10 preview envs) hides inside the total until someone gets a nasty surprise.

2 The mechanism: make cost a first-class, visible signal

FinOps pulls cost *left* and makes it observable: **estimate the cost of a change in CI** (Infracost on the PR), **tag every resource** (team / env / feature) so the bill can be allocated, set **budgets and alerts**, and right-size continuously. It's as much a culture as a tool — engineering, finance, and product jointly owning spend through *inform* → *optimize* → *operate*.



3 The nuance: cost is an architecture property, measured in unit economics

Cost is the Well-Architected **Cost Optimization** pillar, not an afterthought. Levers: right-sizing, autoscaling to demand, **Savings Plans/Reserved** for steady load + **Spot** for interruptible work, storage tiering, and killing zombies (Stage-10 teardown, cross-AZ/region egress from 11·1/11·5). The metric that matters is **unit economics** — $ per job or per tenant — because that, not total spend, tells you whether growth is profitable.

REACTIVE BILLING VS FINOPS

UNTAGGED / REACTIVE

Month-end surprise; waste hidden; no attribution.

FINOPS

Attributable, forecasted, right-sized; unit economics tracked.

🔌 ANALOGY — A SMART METER

FinOps is a smart meter for the cloud bill, broken down per team and per feature in real time — so nobody is shocked at month-end and waste shows up while it's still cheap to fix.

💡 TRACK $/JOB, NOT JUST THE TOTAL

A falling total can still hide a rising cost *per unit of value*. Unit economics ($ per job/tenant) is the number that says whether you scale profitably — architects optimize that, not the invoice.

⚠️ THE SILENT KILLERS

Forgotten preview environments (Stage 10), cross-AZ/region egress, and “just-in-case” over-provisioning quietly dominate real bills. Default TTLs + mandatory tagging are what catch them early.

📝 CHEAT-SHEET

- Estimate cost before merge (Infracost); shift cost left.
- Tag everything for allocation; set budgets + alerts.
- It's the Well-Architected Cost Optimization pillar.
- Levers: right-size, autoscale, Savings Plans/Spot, tiering, kill zombies.
- Optimize unit economics ($/job or $/tenant), not just total spend.

✅ SELF-CHECK

1. Why estimate cost in CI instead of at month-end?
2. What does tagging enable that a raw bill can't?
3. Why are unit economics more useful than total spend?

◆ STAGE 12 — OBSERVABILITY, SRE & CAPSTONE ⭐ ◆

12·1 The Three Pillars

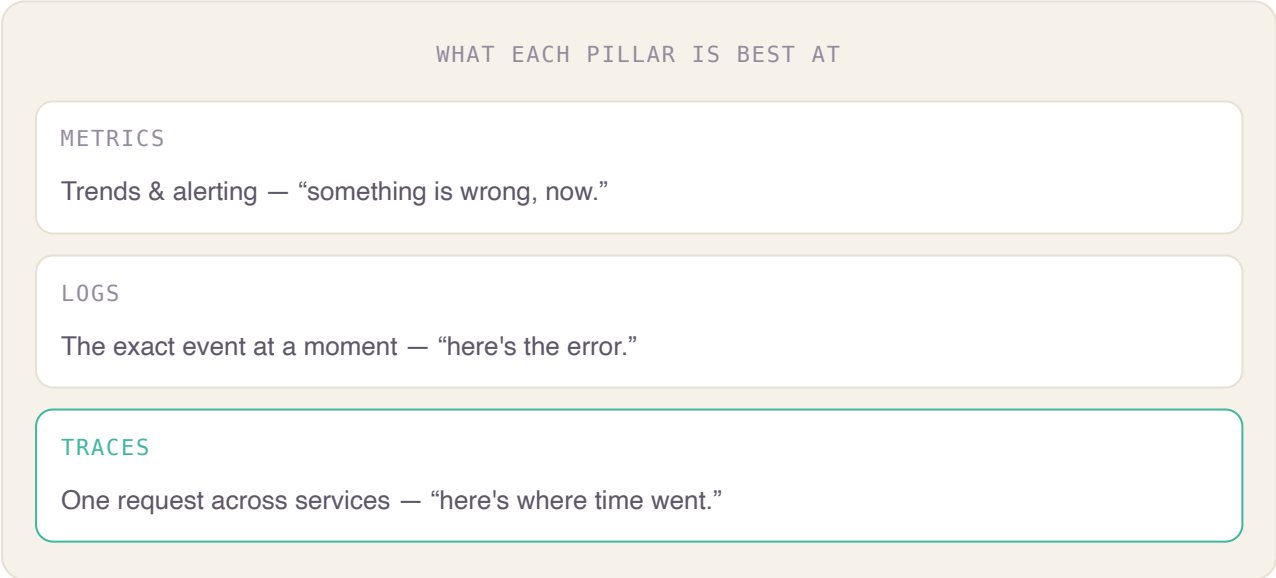
metrics, logs, traces

1 The problem: no single signal answers both “is it broken?” and “why?”

Metrics tell you something's wrong but not why; logs explain one moment but hide trends and cross-service flow; traces show a request's path but aren't where you set alarms. Lean on one alone and every incident stalls at the handoff between “detect” and “diagnose.” You need all three, and — crucially — **correlated**.

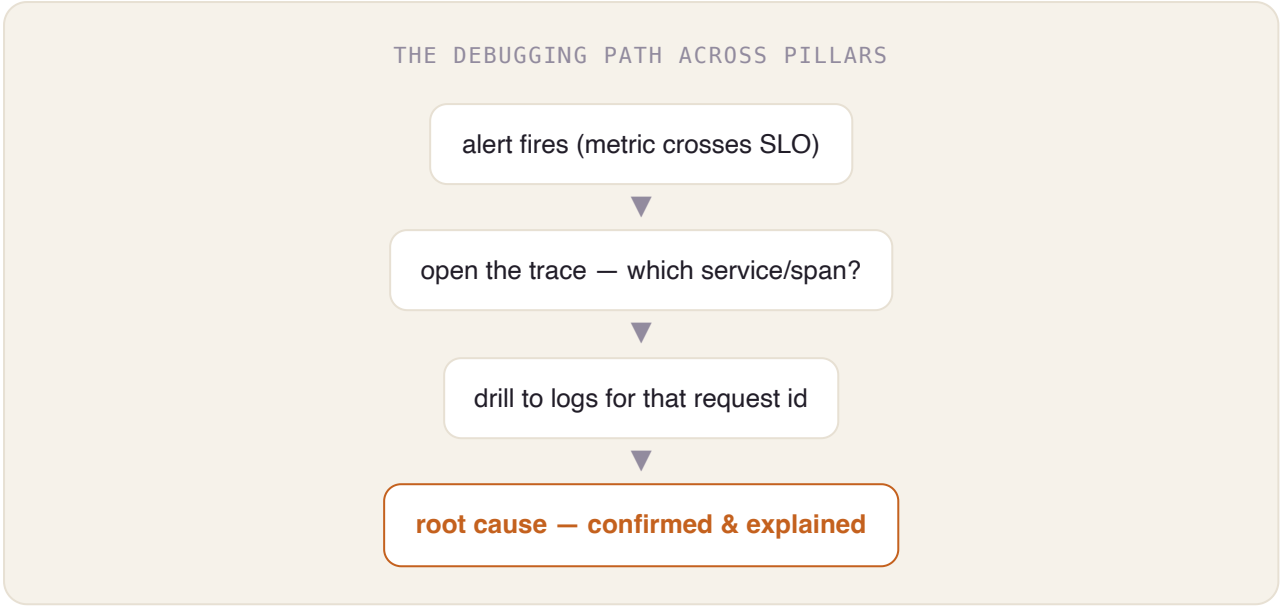
2 The mechanism: three signals, each cheapest at one job

Metrics are cheap aggregated numeric time-series — detection, alerting, dashboards. **Logs** are high-detail discrete events — the exact error at a moment. **Traces** are a request's span tree across services — where the latency went. The glue is a shared **request/trace id** (your Stage-0 correlation ID): it stitches a metric spike to the trace to the log line.



3 The nuance: the design axis is cost × cardinality × retention

Each signal is priced differently. Metrics are cheapest but **explode with high-cardinality labels** (never label by `user_id` / `job_id`). Logs are mid-cost — index vs raw-volume trade. Traces are expensive, so you **sample**. The mature move is to correlate rather than duplicate: an *exemplar* links a metric bucket to a representative trace, so you jump signal-to-signal instead of paying to store everything three ways.



🏥 **ANALOGY — A PATIENT'S CHART**

Metrics are the vitals monitor (trends, alarms), logs are the diary of what happened, traces are the map of the patient's journey through the hospital. A doctor needs all three.

💡 **CORRELATION IS THE WHOLE GAME**

Without a shared id tying them together you have three disconnected tools and a manual hunt. With it, one click walks alert → trace → log. Instrumenting the id (Stage 0) is what makes observability, not just monitoring.

⚠️ **CARDINALITY IS THE METRICS KILLER**

A single unbounded label (user, job, request id) multiplies your time-series into millions and wrecks TSDB cost/performance. Keep metric labels low-cardinality; push the high-cardinality detail into logs/traces.

📝 **CHEAT-SHEET**

- Metrics = detect/alert; Logs = explain a moment; Traces = follow a request.
- A shared request/trace id correlates all three (Stage 0).
- Design by cost × cardinality × retention per signal.
- Exemplars link a metric spike → a trace → logs.
- Never put high-cardinality values in metric labels.

✅ SELF-CHECK

1. What is each pillar the cheapest answer for?
2. What single thing makes the three usable together?
3. Why is a `user_id` metric label dangerous?

12·2 OpenTelemetry

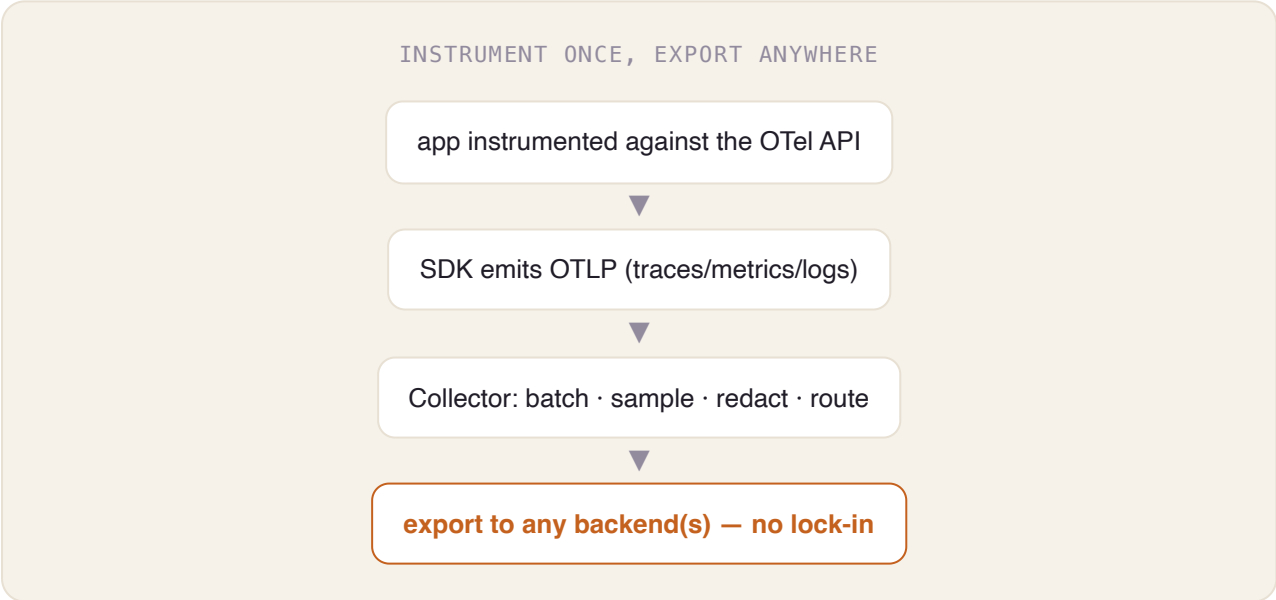
vendor-neutral instrumentation

1 The problem: vendor SDKs lock you in and don't agree

Instrument with a vendor's SDK and switching backends means re-instrumenting the entire codebase. Worse, if every language/agent propagates context its own way, a request's trace breaks at service boundaries — you lose the end-to-end view exactly where distributed systems need it most.

2 The mechanism: one API, the OTLP protocol, and the Collector

OpenTelemetry is a vendor-neutral *API/SDK* + wire protocol (**OTLP**) + the **Collector**. You instrument once against the OTel API; the SDK emits traces/metrics/logs; **context propagation** (W3C `traceparent`) carries the trace across every service hop. The Collector receives, processes, and exports to any backend — so the app never knows or cares where signals land.



3 The nuance: the Collector is where policy lives

The Collector is the architectural leverage point: it does **tail-based sampling** (keep the interesting traces), **PII redaction**, and **fan-out** to several backends — centrally, so app teams never touch export config. **Semantic conventions** make a “http.status_code” mean the same thing everywhere, so signals are comparable across services and languages. Auto-instrumentation gets you coverage fast; manual spans add the business-meaningful detail.

VENDOR SDK VS OTEL

VENDOR SDK

Re-instrument the whole codebase to switch backends.

OPENTELEMETRY

Instrument once; change the backend in the Collector.

🔌 ANALOGY — A UNIVERSAL ADAPTER

One adapter that fits every monitoring socket: wire your code once, and plug into whichever backend you like — swap it without rewiring the house.

💡 CENTRALIZE POLICY IN THE COLLECTOR

Sampling, redaction, and routing belong in the Collector, not scattered through app code. Move that policy once and every service inherits it — the same “decouple apps from infrastructure” instinct as a service mesh.

🔗 BUILT ON STAGE-0 CORRELATION IDS

Your hand-rolled request id becomes a standardized, propagated W3C trace context — the same idea, now portable across every service and vendor.

📝 CHEAT-SHEET

- OTel = vendor-neutral API/SDK + OTLP protocol + Collector.
- Instrument once; swap backends without touching app code.
- Context propagation (traceparent) keeps traces whole across hops.
- Collector = policy point: tail-sampling, redaction, fan-out.
- Semantic conventions make signals comparable everywhere.

✅ SELF-CHECK

1. What does OTel free you from?
2. Why is the Collector the right place for sampling/redaction?
3. What keeps a trace intact across service boundaries?

12·3 Prometheus & Alerting

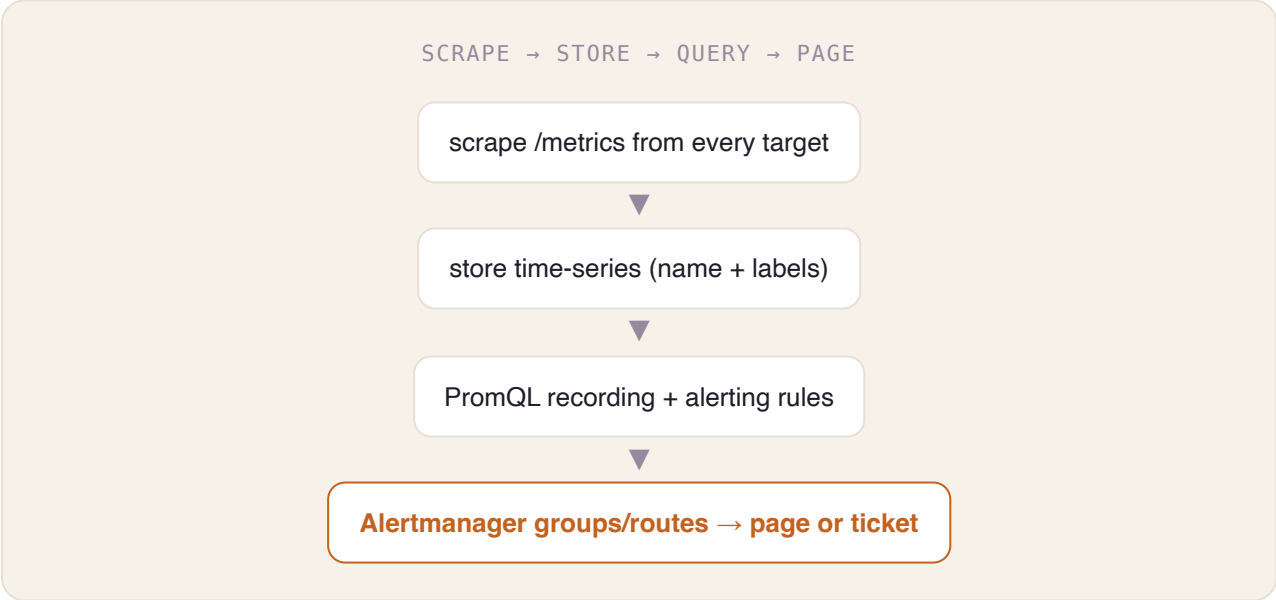
metrics + rules that page

1 The problem: queryable health + alerting, at scale, decoupled from apps

You need numeric health you can query and alert on, for a fleet that's constantly scaling, without every app knowing where the monitoring system is or pushing to it reliably. And you need alerts that route to the right people without drowning them.

2 The mechanism: pull, a labelled TSDB, PromQL, and Alertmanager

Prometheus *pulls* — it scrapes each service's `/metrics` on a schedule (targets found via service discovery). It stores **time-series keyed by name + labels**; **PromQL** queries them; recording rules precompute; alerting rules evaluate and hand firing alerts to **Alertmanager**, which dedupes, groups, silences, and routes to pager/Slack.



3 The nuance: pull-vs-push, durability, and the cardinality ceiling

Pull gives you a free liveness signal (`up`) and easy dedup, but short-lived batch jobs can't be scraped — those *push* to a Pushgateway. Prometheus's local TSDB isn't HA or long-term, so **remote-write** to Thanos/Mimir/Cortex for durability + a global view. And the recurring failure mode is **cardinality explosion** — labels are a dimensional data model, powerful but the same footgun as 12·1. Alertmanager (grouping/inhibition) is where you kill noise — pair it with burn-rate (12·7).

PULL VS PUSH

PULL (DEFAULT)

Liveness (up), easy dedup, standard for long-lived services.

PUSH (PUSHGATEWAY)

For short-lived batch jobs that can't be scraped.

🎛️ ANALOGY — A CONTROL ROOM OF GAUGES

Walls of gauges you can query, with an alarm wired to ring only when a reading crosses a genuine danger line — not for every flicker.

💡 LABELS ARE A DIMENSIONAL MODEL

name+labels is a mini query dimension — slice by service/route/code in PromQL. That power is exactly why an unbounded label detonates your series count.

⚠️ LOCAL STORAGE ISN'T HA OR LONG-TERM

A single Prometheus loses data on restart and can't see across clusters. Remote-write to Thanos/Mimir for durability, global query, and long retention.

📝 CHEAT-SHEET

- Pull-based scraping of /metrics; service discovery finds targets.
- Time-series = name + labels; PromQL queries + recording rules.
- Alertmanager dedupes/groups/routes; where noise control lives.
- Push (Pushgateway) only for short-lived jobs.
- Remote-write (Thanos/Mimir) for HA + long-term + global view.

✅ SELF-CHECK

1. Why does pull give you a liveness signal for free?
2. When must you push instead of scrape?
3. What does remote-write solve that local TSDB doesn't?

12·4 Grafana & Loki

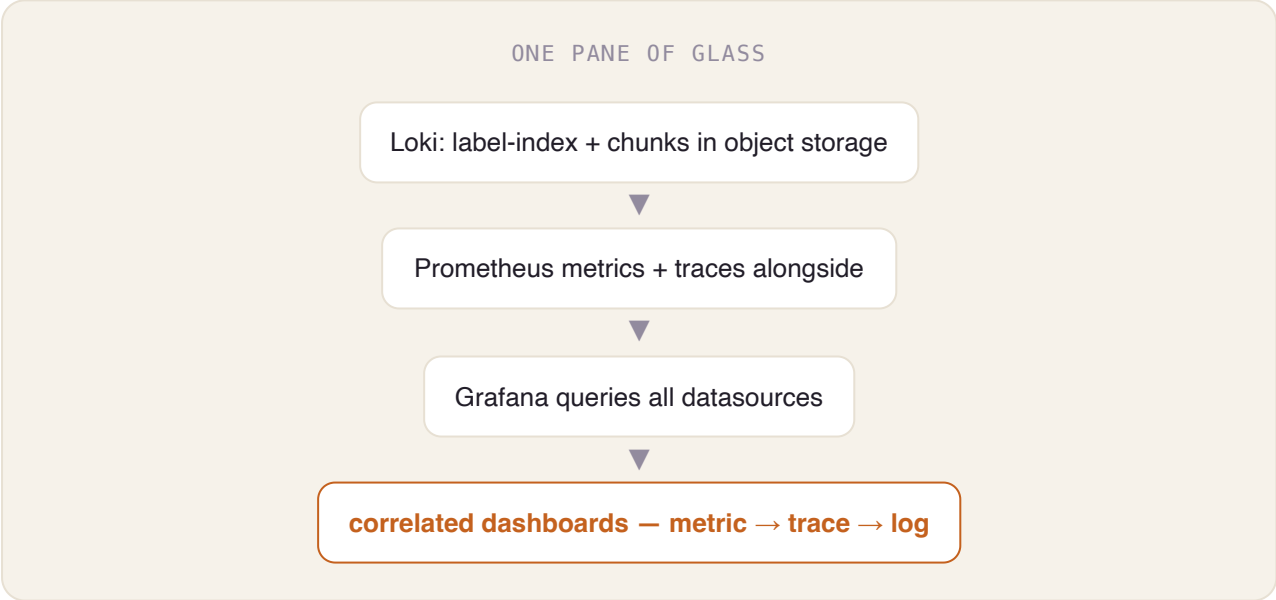
dashboards + a single pane

1 The problem: signals in separate stores, and full-text logs cost a fortune

Metrics, logs, and traces live in different systems; hopping tools mid-incident is slow and error-prone. And indexing every word of every log line (classic ELK) gets ruinously expensive at real volume — you pay to make searchable a haystack you rarely search.

2 The mechanism: a query layer + a label-indexed log store

Grafana is a query + visualization layer over many datasources — one pane of glass, with **correlation**: click a metric exemplar → jump to the trace → jump to the logs for that label set. **Loki** indexes only *labels* and stores compressed log *chunks* in object storage; you narrow by labels then grep the stream with LogQL, instead of indexing content.



3 The nuance: cheap because it indexes metadata, not content

Loki's economy comes from indexing *labels*, not words — so it rewards disciplined labels and punishes high-cardinality ones and broad unfiltered greps (the 12·1/12·3 footgun again). Build dashboards **as code** (provisioned JSON) around the **RED** (rate/errors/duration) and **USE** (utilization/saturation/errors) methods — not a “wall of graphs” nobody reads. Shared labels + trace ids are what make the single pane actually connect.

FULL-TEXT INDEX VS LOKI

FULL-TEXT (ELK)

Fast arbitrary search — but indexes everything, costly at scale.

LOKI

Indexes labels only, greps within a stream — far cheaper.

🎭 ANALOGY — DASHBOARDS ON THE WALL + A JOURNAL

Big status boards you read at a glance (Grafana) next to a searchable event journal (Loki) — and a thread tying an entry on the board to the exact page in the journal.

💡 INDEX METADATA, NOT CONTENT

Loki's whole trick — and its cost story — is indexing labels rather than every word. That single decision is what makes centralized logging affordable at fleet scale.

⚠️ SAME CARDINALITY TRAP, AND DON'T BUILD GRAPH WALLS

High-cardinality labels and broad greps kill Loki too; and a dashboard nobody reads is worse than none. Design to RED/USE and provision dashboards as code.

📝 CHEAT-SHEET

- Grafana = query+viz over many datasources → one pane of glass.
- Correlate: metric exemplar → trace → logs for that label set.
- Loki indexes labels + stores chunks in object storage (cheap).
- LogQL greps within a label-narrowed stream, not full text.
- Dashboards as code; design to RED/USE, not graph walls.

✅ SELF-CHECK

1. What makes Loki cheaper than a full-text store?
2. How does Grafana connect the three pillars?
3. What do the RED and USE methods each cover?

12·5 Profiling (Pyroscope)

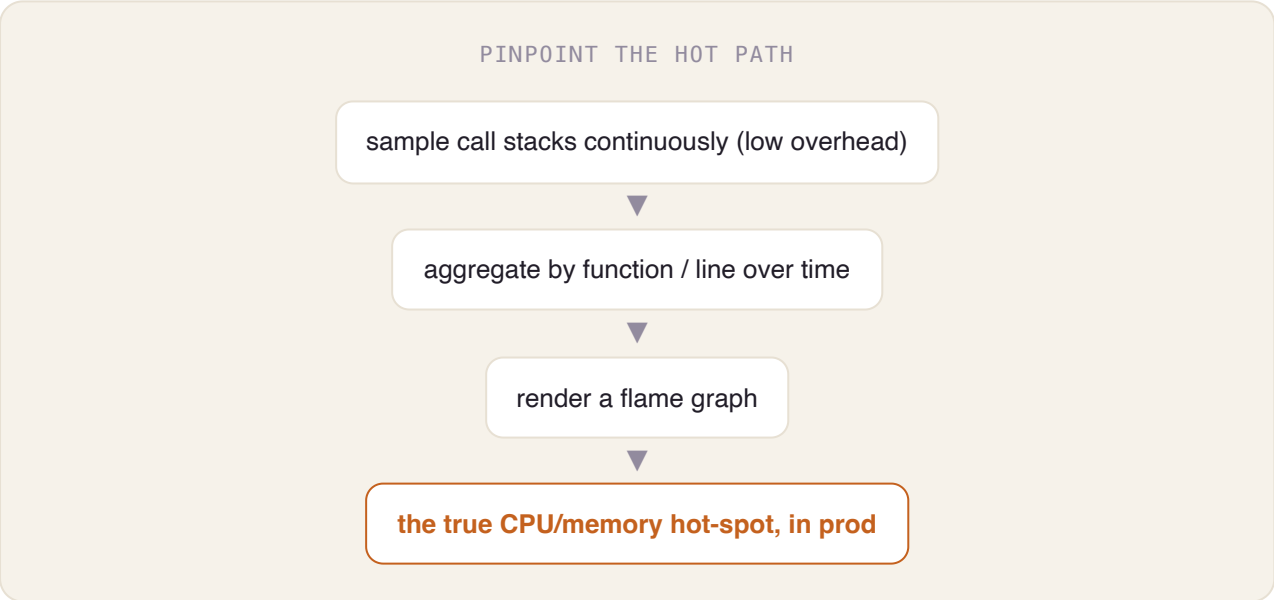
find the hot code

1 The problem: traces name the slow span, not the slow line

Metrics and traces tell you *which* service or endpoint is CPU-hungry or slow — but not *which function or line* is burning the cycles. Reproducing production performance on your laptop is unreliable, so you end up guessing at optimizations. You need line-level truth from prod itself.

2 The mechanism: continuous statistical sampling → flame graphs

Continuous profiling samples the running app's call stacks at a low frequency in production, aggregates by function/line over time, and renders **flame graphs**. Because it's statistical sampling (not instrumenting every call), overhead is small enough to run always-on — so you can correlate a CPU or latency spike to the exact code path that caused it.



3 The nuance: the “fourth pillar” that explains a slow span

Profiling closes the gap the other signals leave: link a **trace span to its flame graph** and you see *why* that span was slow, down to the line. Use it to prioritize optimization by **measured prod cost** (not intuition), to catch performance **regressions** across releases, and to find allocation hotspots driving GC pauses. The only real knob is sampling frequency — resolution vs overhead.

WHAT EACH TOOL LOCALIZES

METRICS / TRACES

Which service or endpoint is slow.

PROFILING

Which function / line burns CPU or memory.

🩻 ANALOGY — AN X-RAY OF THE CODE

Instead of “the patient is tired” (the endpoint is slow), an X-ray shows precisely which muscle is straining — which function is doing the work.

💡 OFTEN CALLED THE FOURTH PILLAR

Wired to traces, profiling answers the question traces raise: “this span took 400ms — doing *what*?” It turns a slow span into a specific line to fix.

⚠️ OPTIMIZE BY MEASUREMENT, NOT INTUITION

The real hot path is rarely where you'd guess. Profile first; a day spent optimizing a cold path is a day wasted while the true bottleneck stands.

📝 CHEAT-SHEET

- Continuous, low-overhead statistical sampling in production.
- Aggregates by function/line → flame graphs.
- Localizes hot-spots to the line — beyond metrics/traces.
- Link a trace span to its profile to explain the slowness.
- Finds regressions + GC-driving allocation hotspots.

✅ SELF-CHECK

1. What does profiling localize that traces don't?
2. Why is it safe to run continuously in prod?
3. Why profile before optimizing?

12·6 SLO / SLI / Error Budgets

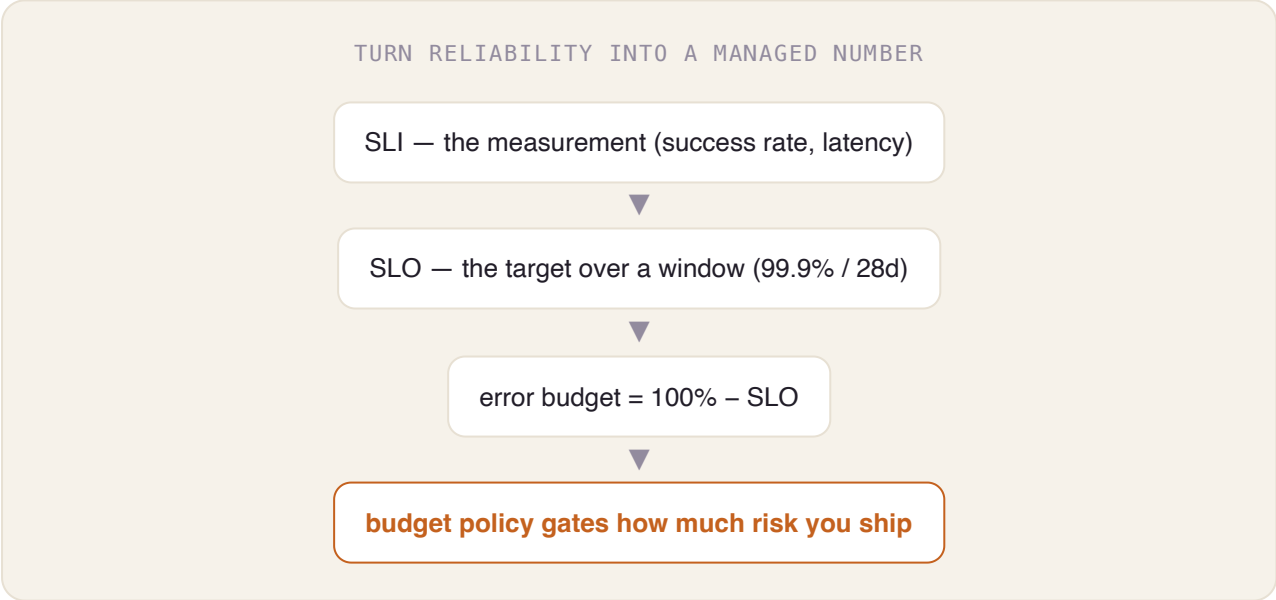
define “good enough”

1 The problem: “make it reliable” is unmeasurable, and 100% is a trap

“Reliable” with no number can't be managed, and **100% is impossible and infinitely expensive** — the last nines cost more than they're worth and users can't tell. Without an agreed target, every incident becomes a subjective argument and you can't decide when reliability work should beat features.

2 The mechanism: measure, target, and budget the failure

An **SLI** is a precise measurement of user-visible reliability (e.g. % of requests that are 2xx and <300ms). An **SLO** is the target over a window (99.9% over 28 days). The **error budget** is `100% − SLO` — the failure you're *allowed*. Budget healthy → ship fast, take risks; budget spent → freeze features and invest in reliability.



3 The nuance: measure from the user, and treat the budget as policy

SLIs must be **user-centric** — measured at the edge, from the user's vantage, not internal CPU that can look green while users suffer. Choose the right SLI (availability, a latency *percentile*, freshness, correctness) and window (rolling vs calendar). The error budget's real power is as a **policy tool**: it turns reliability into a shared, spendable currency that depoliticizes “ship or stabilize?” — the same shape as your Stage-9 experiment guardrails.

AIM-FOR-100% VS SLO + BUDGET

AIM FOR 100%

Infinite cost, still fails, no decision rule.

SLO + ERROR BUDGET

Explicit target, spendable budget, aligns eng + product.

⏱ ANALOGY — AN ON-TIME PROMISE WITH AN ALLOWANCE

“99.9% of trains on time” is the SLO; the small allowance of lateness is the error budget. Spend the allowance and you must slow down and fix the timetable.

💡 THE BUDGET IS A CURRENCY

It converts “should we ship or stabilize?” from an argument into a data decision — spend left, ship; spend gone, stabilize. That alignment is the point, more than the number itself.

⚠️ SERVER-SIDE SLIS LIE

A metric measured inside your service can read 100% while users hit CDN/LB/DNS failures. Measure the SLI from the user's edge or you're grading the wrong exam.

📝 CHEAT-SHEET

- SLI = measurement; SLO = target over a window; budget = 100−SLO.
- 100% is the wrong goal — the last nines aren't worth it.
- Budget healthy → ship; budget spent → freeze & stabilize.
- Measure SLIs user-side (edge), not internal CPU.
- The budget is a policy currency that aligns eng + product.

✅ SELF-CHECK

1. Define SLI, SLO, and error budget in one line each.
2. Why is 100% the wrong target?
3. Why must the SLI be measured from the user's side?

12·7 Burn-Rate Alerting

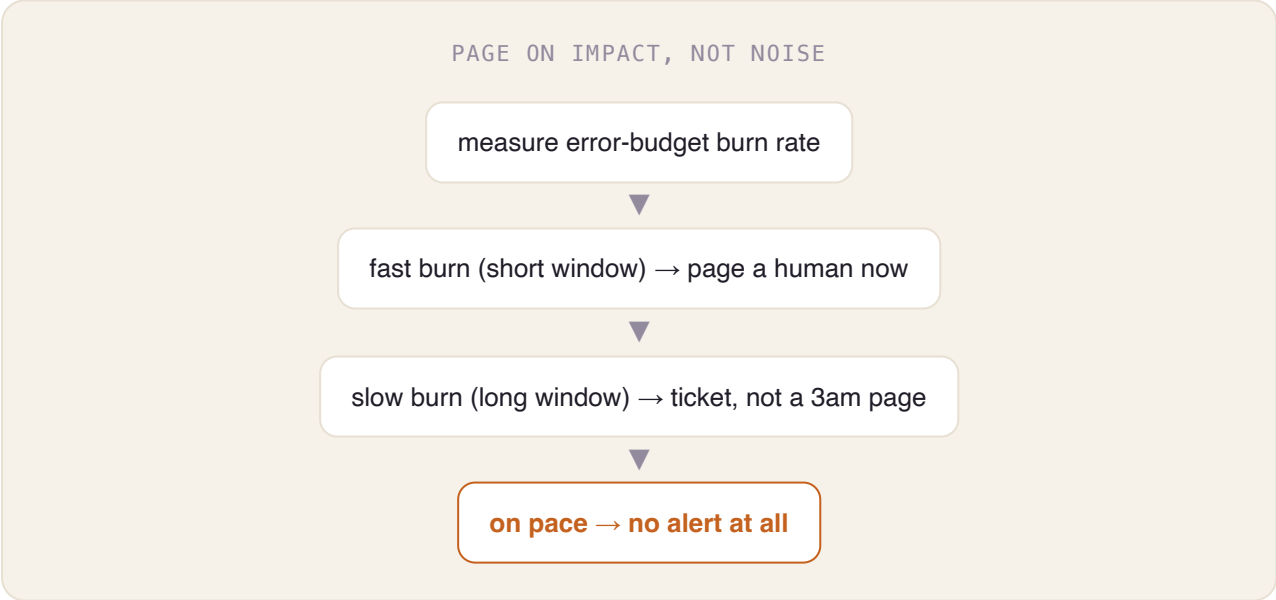
alert on what matters

1 The problem: alert-on-everything trains people to ignore alerts

Paging on every error spike or threshold breach floods humans with alerts for blips that never hurt users. The result is **alert fatigue** — and when everything pages, the one page that matters gets muted with the rest. You want to wake someone only when reliability is *genuinely* at risk.

2 The mechanism: alert on how fast you burn the budget

Burn rate is how fast you're consuming the error budget relative to on-pace (1× spends it exactly over the window; 14.4× spends a month's budget in ~2 days). Alert on *burn rate*, not raw errors. **Multi-window, multi-burn-rate**: a fast-burn rule (high rate over a short window) pages *now*; a slow-burn rule (moderate rate over a long window) opens a ticket.



3 The nuance: symptom-based, and thresholds derive from the budget

Pick thresholds from **how much budget you'll let an incident spend** before engaging humans; the short+long window pair gives fast detection with few false positives (the long window confirms the short one isn't a blip). And prefer **symptom-based** alerting (user-visible SLO burn) over **cause-based** (“CPU high”) — page on impact, keep causes as *diagnostics*. This is 12·6's budget turned into a paging policy.

ALERT-ON-EVERYTHING VS BURN-RATE

ALERT ON EVERYTHING

Noisy, cause-based, breeds fatigue.

MULTI-WINDOW BURN-RATE

Few, symptom-based, meaningful pages.

🔥 ANALOGY — SPEND-RATE ALARM

The alarm rings only when you're spending your allowance dangerously fast — not every time one light flickers. A slow drip gets a note; a gush gets a call.

💡 SYMPTOM OVER CAUSE

Page on the user-visible SLO burn; make CPU, queue depth, and GC diagnostics you consult *after* the page. Cause-based pages are most of the noise and rarely map to real harm.

🔗 THE BUDGET, OPERATIONALIZED

12·6 defined the error budget; burn-rate alerting is how you actually watch it — the policy currency becomes a concrete paging rule.

📝 CHEAT-SHEET

- Burn rate = budget-consumption speed vs on-pace.
- Alert on burn rate, not raw error counts.
- Multi-window (short+long) = fast detection, few false positives.
- Fast burn → page; slow burn → ticket; on pace → nothing.
- Symptom-based (SLO) over cause-based (CPU) alerting.

✅ SELF-CHECK

1. What does burn rate actually measure?
2. Why combine a short and a long window?
3. Why prefer symptom-based over cause-based alerts?

12·8 Chaos Engineering

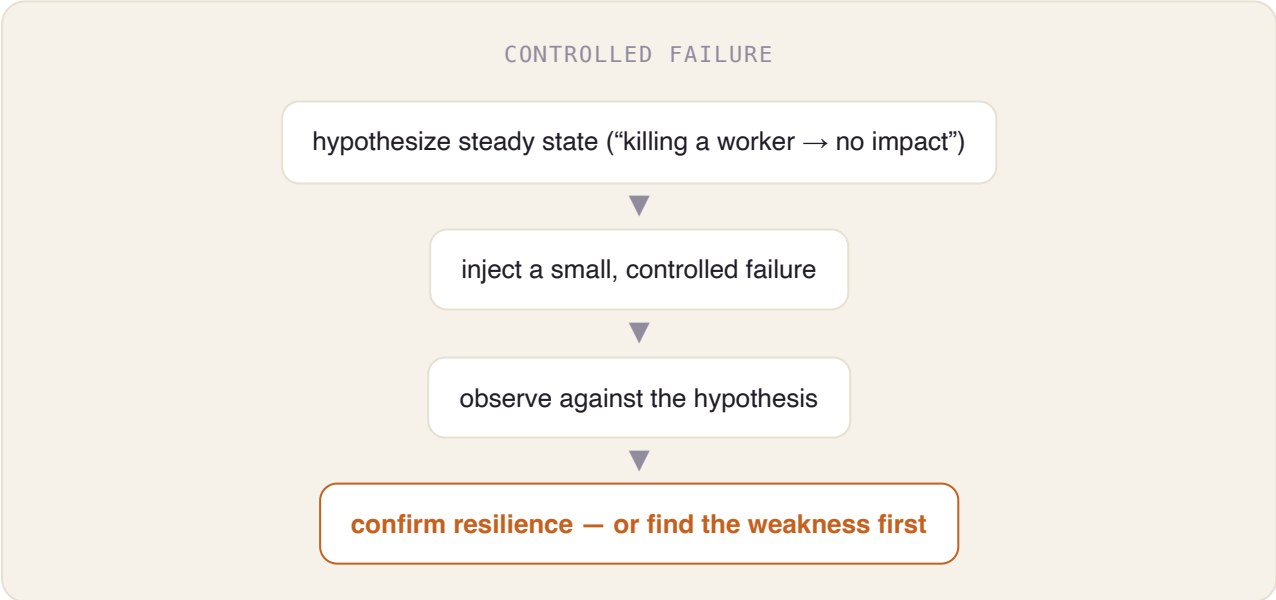
prove resilience on purpose

1 The problem: unverified resilience is just hope

You built retries, circuit breakers, and multi-AZ/region failover (Stages 4 and 11) — but you don't *know* they work until a real failure exercises them, and by then it's a live outage. A circuit breaker never tripped and a failover never triggered are unproven assumptions, not features.

2 The mechanism: the scientific loop with a small blast radius

Chaos engineering is an experiment: state a **steady-state hypothesis** ("kill a worker → no user-visible impact"), **inject a controlled failure** (kill pods, add latency/errors, sever a dependency), **observe** against the hypothesis, and either confirm resilience or discover a weakness. You start with a tiny **blast radius**, off-peak, with an abort switch — a rehearsal, not a gamble.



3 The nuance: GameDays, governance, and validating what you designed

Mature practice runs as scheduled **GameDays** with governance: a capped blast radius, steady-state metrics wired to **auto-abort** (SLO-guarded), and a progression from staging → prod off-peak → continuous. It validates precisely the patterns you designed — breakers, retries with *backoff + jitter*, failover (11·5) — and the culture must be **blameless**: the goal is learning, not blame.

HOPE VS TESTED

HOPE IT'S RESILIENT

Untested failure paths = unknown behavior in the real outage.

CHAOS-TESTED

Weaknesses found on purpose, off-peak, before users hit them.

🧯 ANALOGY — A FIRE DRILL

You start a small, controlled fire to prove the sprinklers actually work — on your schedule, with the fire brigade standing by — rather than discovering they're broken during a real blaze.

💡 IT VALIDATES THE VERY PATTERNS YOU BUILT

Resilience engineering without chaos testing is a set of untested claims. The breaker, the retry, the failover only count once you've watched them catch a real injected failure.

⚠️ BOUND THE BLAST RADIUS OR THE DRILL BECOMES THE OUTAGE

Always cap scope and wire an auto-abort to your SLOs (12·6). Chaos without a kill-switch and a small radius is just self-inflicted downtime.

📝 CHEAT-SHEET

- Inject failures deliberately to verify resilience claims.
- Scientific loop: steady-state hypothesis → inject → observe.
- Start tiny blast radius, off-peak, with an abort switch.
- GameDays with SLO-guarded auto-abort; staging → prod progression.
- Validates breakers/retries/failover; culture stays blameless.

✅ SELF-CHECK

1. Why inject failures on purpose?
2. What governs a safe chaos experiment?
3. Which earlier-stage patterns does it validate?

12·9 Supply-Chain Security (SBOM/cosign/SLSA)

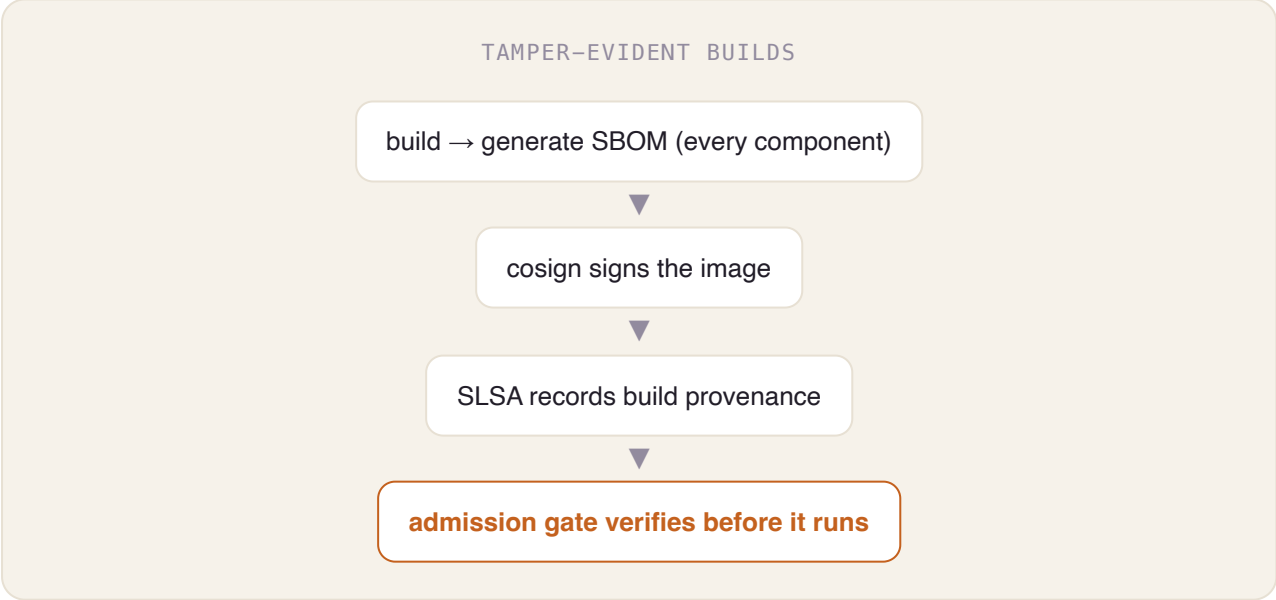
trust what you ship

1 The problem: you can't trust an artifact just because your pipeline built it

A modern build pulls hundreds of transitive dependencies and base images off the internet, and runs on a build server that's itself attackable. A compromised dependency or builder (SolarWinds, the xz backdoor) ships malware straight to prod under your own signature. “Our CI produced it” is no longer proof it's safe.

2 The mechanism: know it, sign it, prove how it was built

Three layers. **SBOM** — a machine-readable bill of materials listing every component + version, so you know what's inside and can scan it. **cosign** — cryptographically sign the artifact (ideally *keyless*, via OIDC identity) so you can verify authenticity + integrity. **SLSA** — a framework of **provenance** attestations proving *how* and *from what source* it was built, at graded maturity levels.



3 The nuance: verification must be an admission gate, not a report

The point isn't the paperwork — it's **enforcement**. An admission controller (Stage 8) *rejects* unsigned images or ones failing policy, so verification is a gate on the way into the cluster, not a dashboard nobody reads. **SLSA levels** are a maturity ladder (source → build → provenance integrity). And SBOM + provenance together let you answer “are we affected by CVE-X?” across the whole fleet in minutes instead of days.

BLIND TRUST VS VERIFIED

TRUST THE PIPELINE

Tamper-blind — a poisoned dep ships silently.

SIGNED + ATTESTED

Verify before run — tamper-evident, CVE-answerable.

🏷️ **ANALOGY — INGREDIENTS, SEAL, CERTIFICATE**

An SBOM is the ingredients list, cosign is the tamper seal, and SLSA is the certificate of origin — together you can prove the package is exactly what the factory made.

💡 **ENFORCE AT ADMISSION**

A signature you never check is theatre. The value lands only when an admission gate refuses to run anything unsigned or non-compliant — verification as policy, not as a PDF.

⚠️ **TRANSITIVE DEPS + BASE IMAGES ARE THE REAL SURFACE**

Your own code is the small part; the risk is the hundreds of things it pulls in. An SBOM is what turns the next zero-day CVE from a week of grepping into a fleet-wide query.

📝 **CHEAT-SHEET**

- SBOM = every component/version → scannable inventory.
- cosign = sign + verify artifacts (prefer keyless/OIDC).
- SLSA = graded build provenance/attestation.
- Enforce at an admission gate — reject unsigned/non-compliant.
- SBOM + provenance = answer “affected by CVE-X?” fleet-wide fast.

✅ **SELF-CHECK**

1. Why isn't “our pipeline built it” sufficient trust?
2. What does each of SBOM / cosign / SLSA give you?
3. Why must verification be an admission gate?

12·10 Load Testing (k6)

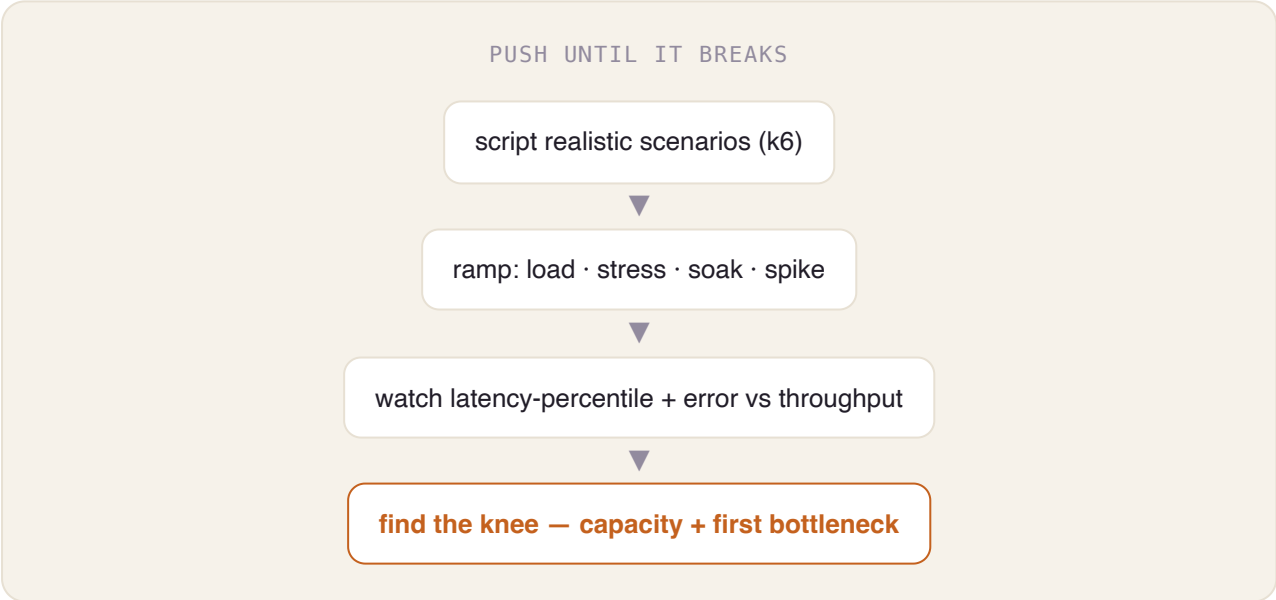
find the breaking point

1 The problem: you don't know your ceiling until traffic finds it

Without testing, your capacity limit and first bottleneck are guesses — and the worst possible time to discover them is during a launch or an organic traffic spike. “How many requests per second can we take, and what breaks first?” must be answered *deliberately*, before real users answer it for you.

2 The mechanism: script realistic load and ramp until it breaks

k6 scripts realistic scenarios (in JavaScript), models virtual users, and ramps load while you watch throughput, **latency percentiles**, and error rate. Test types map to questions: *load* (survive expected peak), *stress* (ramp past it to find the break), *soak* (sustained, to expose leaks), *spike* (sudden surge). The point where latency/errors turn upward is your **capacity knee**.



3 The nuance: model open arrivals, and find the FIRST bottleneck

The subtle trap is the workload model. A **closed** model (fixed pool of virtual users that wait for a response) hides queue collapse, because slow responses throttle the load itself. Real users arrive independently — an **open** model (constant *arrival rate*, k6's arrival-rate executors) exposes the collapse a closed test masks. Test against a prod-like env; the first bottleneck is usually **DB connections/locks**, not CPU; feed results into capacity planning, autoscaling thresholds, and your SLOs (12·6).

CLOSED VS OPEN WORKLOAD MODEL

CLOSED MODEL

Fixed VUs wait for replies — slow responses throttle load, hiding collapse.

OPEN MODEL

Constant arrival rate — realistic; exposes queue collapse.

🌉 ANALOGY — STRESS-TESTING A BRIDGE

Roll on more and more trucks before the bridge opens to the public — you learn the limit on purpose, safely, instead of finding it with a rush-hour jam.

💡 FIND THE FIRST BOTTLENECK

Systems break at their weakest link, and it's rarely CPU — it's usually a database connection pool, a lock, or a downstream quota. Load testing tells you which ceiling you actually hit first.

⚠️ A CLOSED-MODEL TEST FLATTERS YOU

Fixed virtual users that wait for each response quietly self-limit, so the test looks stable right up to a collapse real (open) traffic would trigger. Model arrival rate, not a VU count.

📝 CHEAT-SHEET

- k6 = scriptable load tests; VUs, ramps, thresholds.
- Types: load (peak), stress (break), soak (leaks), spike (surge).
- Watch latency percentiles + error rate vs throughput → the knee.
- Open (arrival-rate) model exposes collapse a closed model hides.
- First bottleneck is usually DB connections/locks, not CPU; feeds capacity + SLOs.

✅ SELF-CHECK

1. Why load-test before a launch instead of after?
2. What does an open (arrival-rate) model reveal that a closed one hides?
3. Where's the first bottleneck usually found?